
Computing Pairings over BLS Curves

Miklós Dániel Vadász, 202402632

Master's Thesis, Computer Science

June 25, 2026

Advisor: Diego F. Aranha



AARHUS
UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

Abstract

Pairing-based cryptography has emerged as a cornerstone of modern public-key cryptographic constructions, since the seminal work of Joux[1] introducing a one-round three-party Diffie-Hellman protocol using bilinear pairings. [2]

Since the seminal work of Joux[1], subsequently, the field of pairings became a priority research area. At their core, pairings builds on bilinear maps defined on elliptic curves over finite fields, transferring hard problems from elliptic-curve groups to multiplicative subgroups of finite fields while preserving sufficient hardness assumptions for cryptographic security.

In the last four decades such extensive research provided a fundamental tool for designing cryptosystems, pairings such as the Weil and Tate-Lichtenbaum pairings have found widespread application in identity-based encryption[3], short digital signatures, zero-knowledge proof systems[4], and advanced privacy-preserving protocols.

We begin with a concise theoretical foundation, covering elliptic curves over finite fields, divisors, and the mathematical structure underlying Weil and Tate-Lichtenbaum pairings. Special emphasis is placed on the concept of extension field tower, embedding degrees, and curve families used in industrial practice. [5, 6, 7]

This work provides a hybrid contribution that is both a survey and an example-oriented study on pairing constructions which enjoy industry interest, such as the BLS12-381 curve. As such, the practical component of this work includes SageMath [8] implementations and experiments, illustrating pairing computations, subgroup operations, and performance characteristics using existing libraries and examples motivated by the referenced academic sources.

Notations

(f)	divisor of the function f
$(f_{m,P})$	function with divisor $(f_{m,P}) = m(P) - ([m]P) - (m-1)(\mathcal{O})$
$\deg(D)$	degree of divisor f
$f(D)$	function evaluated at a divisor
$\sim$	equivalence of divisors
$[n]P$	scalar multiplication (exponentiation) of P by $n \in \mathbb{Z}$
$\mathbb{A}^n(K)$	affine n -space over the field K
D_P	divisor $(P) - (\mathcal{O})$
D_Q	divisor $(Q) - (\mathcal{O})$
$\varepsilon(D)$	effective part of the divisor D
$\mathbb{F}_q$	finite Field with q elements
$\mathbb{F}_q^k$	full extension field
$\mathbb{Z}$	integer numbers
$\mathbb{Z}_n$	integers modulo n
$\mathbb{Z}_p$	when p is a prime $\mathbb{Z}_p$ denotes field rather than a group or ring. $\mathbb{Z}_p$ does not denote the p -adic integers
$\mathbb{Q}$	rational numbers
$\mathbb{Q}_p$	p -adic rational numbers
$\mathbb{R}$	real numbers
$\mathbb{C}$	Complex numbers
K	arbitrary field
K^*	denotes the multiplicative group for field K
K^+	denotes the additive group for field K
$\bar{K}$	if K is a field, then $\bar{K}$ denotes an algebraic closure of K
$\mathbb{P}^n(K)$	projective n -space over the K field
$\mathbb{A}^n(K)$	affine n -space over the arbitrary K field
$\text{char}(\mathbb{K})$	characteristics of field K
$\text{Gal}(L/K)$	Galois group of L over K
$\ell_{R,Q}$	indeterminate Miller line function regarding the addition of R and Q
$\ell_{R,R}$	indeterminate Miller line function regarding the doubling of R
$\mathbb{G}_1$	base field subgroup: $E[r] \cap \ker[\pi - [1]]$ (in Type 3 pairings)

$\mathbb{G}_2$	base field subgroup: $E[r] \cap \ker[\pi - [q]]$ (in Type 3 pairings)
$\mathbb{G}_T$	order r subgroup of $\mathbb{F}_{q^k}^*$, also the r -th roots of unity μ_k
d	degree of twist
$\text{End}(E)$	endomorphism ring of Elliptic Curve E
$\text{Aut}(E)$	automorphism of E elliptic curve
ϕ	occurs as the distortion map on supersingular curves and as the GLV endomorphism
Φ_i	the i -th cyclotomic polynomial
π	q -power Frobenius endomorphisms: $(x, y) \rightarrow (x^q, y^q)$
ψ	occurs as both the isomorphism from $\mathbb{G}_2$ to $\mathbb{G}_1$ and as the GLS isomorphism
$\psi_l(x)$	the l -th division polynomial on E (for odd l)
Ψ	the (un)twisting isomorphism
ρ	the ratio between base field size and subgroup size for pairing-friendly curve
$\text{Pic}^0(E)$	Picard group of E
$\text{Prin}(E)$	group of principal divisors on E
$\text{QR}(q)$	set of quadratic residues modulo q
$\text{Pic}^0(E)$	support of the divisor D
C	an arbitrary curve
C_g	an (imaginary quadratic) hyperelliptic curve with genus g
E	an elliptic curve
E'	a twisted elliptic curve defined over $\mathbb{F}_{q^{k/d}}$
$E[r]$	the entire r -torsion
E/K	elliptic curve defined over field K
D	occurs as both divisor on E and the CM discriminant of E
g	genus of the curve
r	order of the large prime subgroup in $E(\mathbb{F}_q)$
k	embedding degree of the E elliptic curve (with respect to q and r)
ρ	ratio between base field size and subgroup size for a pairing-friendly curve
$\#E$	number of points on E
$\mathcal{O}$	points at infinity on the E elliptic curve
nP	multiplicity of point P in associated divisor
P	generator of $\mathbb{G}_1$
P'	twist of the point P
Q	generator of $\mathbb{G}_2$
Q'	twist of the point Q
e	a general pairing
$e(P, Q)$	pairing of P and Q (paired value)

η_T	eta (T) pairing
a_T	ate pairing
t	trace of Frobenius
aTr	anti-trace map
T	ate pairing loop parameter ($T = t - 1$)
$t_r(P, Q)$	order- r Tate pairing
$T_r(P, Q)$	order- r reduced Tate pairing
$w_r(P, Q)$	order- r Weil pairing

Acronyms

CM Complex Multiplication method

DLP *discrete logarithm problem*

DLP Discrete Logarithm Problem

ECC Elliptic Curve Cryptography

ECDDHP Elliptic Curve Decision Diffie-Hellman Problem

ECDHP Elliptic Curve Diffie-Hellman Problem

ECDLP Elliptic Curve Discrete Logarithm Problem

HMAC Hash-based Message Authentication Code

MAC Message Authentication Code

*Miklós Dániel Vadász,
Aarhus, June 25, 2026.*

Contents

Abstract	iii
Notations	v
1 Introduction	1
1.1 Motivation	1
1.1.1 Objectives	2
1.2 Organization of the document	2
1.3 Contributions	4
2 Preliminaries	7
2.1 Introduction to groups, rings and fields	7
2.1.1 Groups and subgroups	7
2.1.2 Basic definitions and properties from Ring and general Field Theory	9
2.1.3 Fields	11
2.2 Overview of Finite Field Arithmetic	11
2.2.1 Introduction to finite fields	11
2.3 Public-key Cryptography using Elliptic Curves	14
2.3.1 Diffie-Hellman Public-Key Encryption/Decryption Scheme . .	16
2.3.2 Security assumptions related to Elliptic Curve-based Cryptosystems	18
2.4 Elliptic Curve operations over Finite Fields	18
2.4.1 Introduction to Elliptic Curves	19
2.4.2 Point Representations over general K field	22
2.4.3 The chord-and-tangent rule: Group law	25
2.4.4 Representing the point at infinity in projective space	26
2.4.5 Speeding up the group law computations	30
2.5 Divisor Theory	32
2.5.1 The divisor class group and consequence of the Riemann-Roch Theorem	36
2.6 Elliptic curves as pairing groups	39
2.6.1 The r -torsion subgroup, embedding degree, Frobenius endomorphism	40
2.6.2 Pairing types	43
2.6.3 Twist of elliptic curves	44

3	Computational Aspects of Pairings	47
3.1	Miller’s algorithm for the Weil and Tate pairings	47
3.1.1	Theoretic overview of Weil Pairing	47
3.1.2	Theoretic overview of Tate Pairing	48
3.2	Merging the Theory with Practice	49
3.2.1	Miller’s Algorithm for Weil and Tate pairing	50
3.2.2	Working example for Weil Pairing	53
3.2.3	Working example for Tate Pairing	53
4	BLS curves for high-security pairings	57
4.1	Pairing-friendly curve selection and the balancing act	57
4.1.1	The balancing act	58
4.1.2	Review the construction of ordinary pairing-friendly curves .	59
4.1.3	Overview of Barreto-Lynn-Scott Curve family	60
4.1.4	Line function	63
4.2	Optimizations	64
4.2.1	Denominator elimination	64
4.2.2	Low hamming weights	65
4.2.3	Implementation Summary: Reduced Tate Pairing on BLS12-381	66
	Bibliography	71
A	SageMath Code examples	77
A.1	(<i>Section 2.1</i>) Introduction to groups rings and fields	77
A.1.1	Group operations	77
A.1.2	Ring operations	90
A.1.3	Field operations	95
A.2	Elliptic Curve operations over Finite Fields	106
A.2.1	(<i>Subsection 2.4.1</i>) Introduction to Elliptic Curves	106
A.2.2	(<i>Subsection 2.4.2</i>) Elliptic Curve Point representations . . .	111
A.2.3	(<i>Subsection 2.4.3</i>) Elliptic curve group law computation . .	114
A.3	(<i>Section 2.5</i>) Divisors	116
A.3.1	Divisor class groups and the consequence of Riemann-Roch theorem	118
A.4	(<i>Section 2.6</i>) Elliptic curves as pairing groups	123
A.5	(<i>Section 3.1</i>) Computational aspects of pairings	133
A.6	Pairing-friendly curve selection and balancingact	138
A.7	Tate pairing implementation over BLS Curve	141
A.7.1	(Reduced) Tate Pairing Computation over BLS12-381 curve .	141
B	Theoretical Appendices	151
B.1	Weil Pairing	151
B.1.1	Provable properties of Weil pairing	151
B.1.2	Rational functions and divisors	152
B.1.3	BLS curve	155

Chapter 1

Introduction

1.1 Motivation

Pairing-based cryptography has emerged as a cornerstone of modern public-key cryptographic constructions, since the seminal work of Joux[1] introducing a one-round three-party Diffie-Hellman protocol using bilinear pairings. [2] The general assumption was and has been proven in some cases (e.g., see [9, 10]), that the Discrete Logarithm Problem (DLP) reduces in polynomial time to the Diffie-Hellman Problem (DHP).[2] Thus, the underlying Diffie-Hellman Problem believed to be intractable for specifically chosen groups over a finite field, such as the multiplicative group of a finite field or the group of points of an elliptic curve defined over a finite field. [2]

Since the seminal work of Joux[1], subsequently, the field of pairings became a priority research area. At their core, pairings builds on bilinear maps defined on elliptic curves over finite fields, transferring hard problems from elliptic-curve groups to multiplicative subgroups of finite fields while preserving sufficient hardness assumptions for cryptographic security.

In the last four decades such extensive research provided a fundamental tool for designing cryptosystems, pairings such as the Weil and Tate-Lichtenbaum pairings have found widespread application in identity-based encryption[3], short digital signatures, zero-knowledge proof systems[4], and advanced privacy-preserving protocols. However, developments in quantum computing have led to a generally accepted recognition that a future large-scale quantum computer would be able to break any public-key cryptosystem used today (e.g. based on ECC).[11]

In response to this, several globally recognized government agencies have taken steps to standardize post-quantum cryptographic (PQC) systems , focusing primarily on digital signatures and key exchange/public key encryption systems. Despite the current efforts, such existing solutions suffer from various disadvantages in terms of efficiency and compactness to gain the necessary trust in the academic and industrial domains [11]. Therefore, it is believed that before transition to purely quantum-safe cryptography an intermediate step shall occur where current classically secure protocols and quantum-safe solutions will co-exist.[11] This is confirmed by the Commercial National Security Algorithm Suite 2.0 report[12], which makes the transition to quantum-resistant cryptographic algorithms mandatory by 2033 and recommends the use of 192-bit security level ECC until then.[11, 12]

1.1.1 Objectives

This work provides a hybrid contribution that is both a survey and an example-oriented study on pairing constructions which enjoy industry interest, such as pairing over BLS curves.[13, 11]

Therefore we begin with a concise theoretical foundation, covering elliptic curves over finite fields, divisors, and the mathematical structure underlying Weil and Tate-Lichtenbaum pairings. Special emphasis is placed on the concept of extension field towering, embedding degree, and how one can choose parameters securely which are aligned industrial best practices. [5, 6, 7]

1.2 Organization of the document

Chapter 1

The introductory chapter of my thesis introduces my motivation (*Subsection 1.1*), describes the objectives (*Subsection 1.1.1*). Within *Subsection 1.2* the Reader get an overview of the organization the document itself, lastly the chapter ends with *Subsection 1.3* outlining my contributions to the subject matter.

Chapter 2

Accordingly for a more straightforward didactic structure and in accordance with the objectives of *Subsection 1.1.1* , *Chapter 2* starts with a brief introduction to groups, rings and fields in *Subsection 2.1* . The subsection then closes with the justification for Galois field and why is it essential for cryptographic applications, such as pairings.

For the sake of efficient computation, it is of paramount importance that the chosen finite field possess good arithmetic properties. Therefore *Subsection 2.2* follows with the overview of finite field arithmetic, including field operations, the existence and uniqueness criteria and field characteristics.

Section 2.3 provides the connection between pairing-based cryptography and elliptic curve based cryptosystems with respect to the related security assumptions . Specifically, we review several assumptions based on discrete log problem that play a critical role in pairing based cryptosystem security.

Within *Section 2.4* we review the elliptic curve operations over finite fields, which are serves us the building blocks for pairing computation. The simplification of the Weierstrass Equation introduced in *Equation 2.7* provides us the ability to differentiate between three distinct cases in relation with field characteristics. Yields us with the notation of non-supersingular (ordinary) and supersingular elliptic curves.

Depending on whether we choose a supersingular or a standard elliptic curve, additional options become available for making the calculations more efficient in terms of point representation. *Subsection 2.4.2* introduces the techniques for point mapping based on the given formulas, thereby establishing a connection between point mapping in affine and projective spaces.

Based on the methodology presented in 2.4.3 and 2.4.4, the subsections analyze how point addition and doubling operations can be performed, taking into account the distinguished point at infinity.

In practical cryptographic applications we have to deal with rational function. However, instead of working with f directly, we can uniquely determine it by its divisor $\text{div}(f)$ as introduced in *Section 2.5*, followed by some important properties outlined within *Subsection 2.5.1*.

Section 2.6 focuses on elliptic curves as pairing groups with the primary objective on how to define them for pairing purposes. In *Subsection 2.6.1* we focus on r -torsion subgroups and their structures, leading us with the definition of the embedded degree.

As it turns out in *Subsection 2.6.2* the placing of $\mathbb{G}_1$ and $\mathbb{G}_2$ in different subgroups of $E[r]$ will have practical implications, therefore we can distinguish between four pairing types. Lastly in *Subsection 2.6.3* we discuss the notion of *twists* of elliptic curves, which can be used pull computations back into the subfield $\mathbb{F}_{q^{k/d}}$ of $\mathbb{F}_{q^k}$.

Chapter 3

In *Chapter 3* we review the theoretical foundations of Weil and Tate pairings with regards to Miller's algorithm on how to compute them. First we start with the Theoretical overview of the Weil pairing in *Subsection 3.1.1*, however we note that finding explicit expressions presented in *Equation 3.1* is infeasible for cryptographically useful pairings, hence we will use Miller's algorithm.

In the followup *Subsection 3.1.2* we review we consider the Tate pairing. Recall we wish to compute $f_P(Q)$ where f_P has divisor $(P)r$. We can view our current situation as a special case of the above with $R = \mathcal{O}$.

We conclude the chapter with *Section 3.2* reviewing some examples motivated by the applications of pairings based on [5].

Chapter 4

In the last chapter we review the process behind selecting pairing friendly curves, namely the balancing act (refer to *Subsection 4.1.1*).

We revisit the ρ -value calculation, which indicates how much (ECDLP) security a curve offers for its field size. The reason behind it is the fact that pairings are fundamentally different to traditional number-theoretic primitives, in that they require multiple groups that are defined in different settings. Which means that all three groups must be secure against the respective instances of the discrete logarithm problem.

In the following subsection *Subsection 4.1.2* we briefly review the complex multiplication method (CM for short) of Atkin and Morain [14], which can be used to find the elliptic curve equations provided the CM discriminant. Building upon the technique presented in *Subsection 4.1.2*, we review the constructions of ordinary pairing-friendly curves, following up with the overview of BLS Curve Family (*Subsection 4.1.3*)

Lastly, in *Subsection 4.2.3* we conclude the chapter with the implementation of Reduced Tate pairing over BLS12-381 curve, with further optimization possibilities outlined in *Subsection 4.2*

Appendix

Due to the breadth of the subject matter, it is not possible to include every example here for reasons of space constraints therefore, illustrative examples and definitions

that aid in understanding each topic have been included within the Appendix as can be followed in the subsequent *Section 1.3*.

1.3 Contributions

The practical component of this work includes SageMath-based[8] implementations and experiments, illustrating pairing computations, subgroup operations, and performance characteristics using existing libraries and selected examples motivated by the referenced materials.

- *Chapter 2 , Section 2.1* Code Examples:
 - **Definition 2.1.1, Definition 2.1.2, Definition 2.1.3:** Refer to *Code Snippet A.1* in *Appedix A.1.1*
 - **Definition 2.1.4, Definition 2.1.5, Definition 2.1.6:** Refer to *Code Snippet A.2* in *Appedix A.1.1*
 - **Definition 2.1.7, Definition 2.1.8:** Refer to *Code Snippet A.3* in *Appedix A.1.1*
 - **Definition 2.1.9, Definition 2.1.10, Definition 2.1.11:** Refer to *Code Snippet A.4* in *Appedix A.1.1*
 - **Definition 2.1.12:** Refer to *Code Snippet A.5* in *Appedix A.1.1*
 - **Definition 2.1.13, Definition 2.1.14:** Refer to *Code Snippet A.6* in *Appedix A.1.2*
 - **Definition 2.1.15:** Refer to *Code Snippet A.7* in *Appedix A.1.2*
 - **Definition 2.1.16:** Refer to *Code Snippet A.8* in *Appedix A.1.2*
 - **Definition 2.1.17, Definition 2.1.18, Definition 2.1.19:** Refer to *Code Snippet A.9* in *Appedix A.1.3*
 - **Definition 2.1.20:** Refer to *Code Snippet A.10* in *Appedix A.1.3*
 - **Definition 2.1.21:** Refer to *Code Snippet A.11* in *Appedix A.1.3*
 - **Example 2.2.1:** Refer to *Code Snippet A.12* in *Appedix A.1.3*
 - **Example 2.2.2:** Refer to *Code Snippet A.13* in *Appedix A.1.3*
 - **Example 2.2.3:** Refer to *Code Snippet A.14* in *Appedix A.1.3*
- *Chapter 2 , Section 2.4.1* Code Examples:
 - **Example 2.4.1:** Refer to *Code Snippet A.15* in *Appedix A.2.1*
 - **Example 2.4.2:** Refer to *Code Snippet A.16* in *Appedix A.2.2*
 - **Example 2.4.3:** Refer to *Code Snippet A.17* in *Appedix A.2.2*
 - **Example 2.4.4:** Refer to *Code Snippet A.18* in *Appedix A.2.2*
 - **Example 2.4.5:** Refer to *Code Snippet A.19* in *Appedix A.2.3*
- *Chapter 2 , Section 2.5* Code Examples:
 - **Example 2.5.1:** Refer to *Code Snippet A.20* in *Appedix A.3*

- **Example 2.5.3:** Refer to *Code Snippet A.22* in *Appedix A.3*
- **Example 2.5.5:** Refer to *Code Snippet A.23* in *Appedix A.3*
- **Example 2.5.6:** Refer to *Code Snippet A.22* in *Appedix A.3*
- **Example 2.5.7:** Refer to *Code Snippet A.22* in *Appedix A.3*
- **Example 2.5.8:** Refer to *Code Snippet A.22* in *Appedix A.3*
- *Chapter 2 , Section 2.6* Code Examples:
 - **Example 2.6.1:** Refer to *Code Snippet A.27* in *Appedix A.5*
 - **Example 2.6.2:** Refer to *Code Snippet ??* in *Appedix A.4*
 - **Example 2.6.3:** Refer to *Code Snippet A.29* in *Appedix A.4*
 - **Example 2.6.4:** Refer to *Code Snippet A.30* in *Appedix A.4*
- *Chapter 3 , Section 3.2* Code Examples:
 - **Example 3.2.1:** Refer to *Code Snippet A.31* in *Appedix A.5*
 - **Example 3.2.2:** Refer to *Code Snippet A.32* in *Appedix A.5*
- *Chapter 4 , Section 4.1* Code Examples:
 - **Example 4.2.1:** Refer to *Code Snippet A.33* in *Appedix A.5*
 - **Example 4.2.3:** Refer to *Code Snippet A.34* in *Appedix A.5*

Chapter 2

Preliminaries

2.1 Introduction to groups, rings and fields

In this first section I review the fundamental definitions from the field of Abstract Algebra based on [15]. First I begin by defining groups in *Subsection 2.1.1*, afterwards I will consider rings and fields in *Section 2.1.2* and 2.1.3 respectively.

2.1.1 Groups and subgroups

Definition 2.1.1. A group $(G, \cdot)$ is a set G together with binary operator $\cdot$ (that is, a function) $G \times G \rightarrow G$ that satisfies the following four properties:

1. **(Closure)** If $a, b \in G$, then $a \cdot b \in G$.
2. **(Associativity)** If $a, b, c \in G$, then $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. **(Identity)** For every $a \in G$ there exists identity element $e \in G$, such that $e \cdot a = a \cdot e = a$.
4. **(Inverse)** Every $a \in G$ has an inverse element $a^{-1} \in G$, such that $a \cdot a^{-1} = a^{-1} \cdot a = e$.

Definition 2.1.2. An abelian group $(G, \cdot)$ is a group $(G, \cdot)$ satisfying the additional commutativity property in addition to closure, associativity, identity and inverse properties:

5. **(Commutativity)** If $a, b \in G$, then $a \cdot b = b \cdot a$.

Definition 2.1.3. Let G given a group. Then the *order* of G is the cardinality of the set G , denoted by

$$|G| = \#G.$$

Using *Corollary 2.6.6* from [15] and *Definition 2.1.2* the followings are equivalent: G is cyclic (refer to *Definition 2.1.9*); for any integer $d \mid n$, G group has exactly d elements such that order divides d .

Definition 2.1.4. Given groups G and H . Their *direct product* denoted as $G \times H$ is the group:

$$G \times H = \{(g, h) \mid g \in G, h \in H\},$$

with group operation defined componentwise as $(g_1, h_1) \cdot (g_2, h_2) = (g_1 g_2, h_1 h_2)$.

Definition 2.1.5. Let G be a group and let A and B be subsets of G . Then the product AB is the subset of G given by

$$AB = \{ab \mid a \in A, b \in B\}.$$

The following definition is the direct consequence of the associative law $(ab)c = a(bc)$ for multiplication of individual elements of G :

Definition 2.1.6. Let G be a group and let $A, B, C \subseteq G$. Then we can formulate

$$(AB)C = A(BC).$$

Given G, H groups. As a next step we want to consider mapping $\varphi : G \rightarrow H$ between groups, which are taking into account the operations on given groups as well.

Definition 2.1.7. Let G and H groups, then homomorphism $\varphi : G \rightarrow H$ from a group G to a group H is a function for all $a, b \in G$, with the property:

$$\Phi(ab) = \Phi(a)\Phi(b)$$

Two important properties of homomorphisms are the following: $\varphi(e) = e$, and $a \in G, \Phi(a^{-1}) = \Phi(a)^{-1}$.

Example 2.1.1. As an example let us consider homomorphisms between abelian groups, using additive notation.

- (i) For any fixed integer n , we have a homomorphism $\phi : \mathbb{Z} \rightarrow \mathbb{Z}$ defined by $\phi(i) = ni$. Then the following observation can be made regarding ϕ as homomorphism as follows: $\phi(i + j) = n(i + j) = ni + nj = \phi(i) + \phi(j)$
- (ii) We have a homomorphism $\varphi : \mathbb{Z} \rightarrow \mathbb{Z}_n$ for any fixed integer n defined by: $\varphi(i) = [i]_n$. Then one may observe φ is a homomorphism by the definition of addition in $\mathbb{Z}_n$ as:

$$\varphi(i + j) = [i + j]_n = [i]_n + [j]_n = \varphi(i) + \varphi(j),$$

- (iii) For any fixed integers m and n , we have a homomorphism $\varphi : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ defined by $\varphi([i]_n) = [mi]_n$. With a similar observation as before we conclude that

$$\begin{aligned} \varphi([i]_n + [j]_n) &= \varphi([i + j]_n) = [m(i + j)]_n \\ &= [mi + mj]_n = [mi]_n + [mj]_n \\ &= \varphi([i]_n) + \varphi([j]_n). \end{aligned}$$

- (iv) Lastly, we have a homomorphism $\varphi : \mathbb{Z}_n \rightarrow \mathbb{Z}_m$ defined by $\varphi([i]_n) = [i]_m$. To ensure that φ is well defined, we need to know that the value of $\varphi([i]_n)$ does not depend on the choice of representative of the congruence class $[i]_n$. We refer *Example 2.2.4* furthermore in [15].

Definition 2.1.8. Given G and H groups, then the homomorphism $\varphi : G \rightarrow H$ is an isomorphism if there exists an inverse homomorphism defined as:

$$\varphi^{-1} : H \rightarrow G.$$

On the otherhand an isomorphism $\varphi : G \rightarrow H$ exists, then the groups G and H are said to be isomorphic. Furthermore, the isomorphism $\varphi : G \rightarrow G$ is also called the automorphism of G .

With that being said the automorphism group $\text{Aut}(G)$ of G is the group of all automorphisms $\varphi : G \rightarrow G$.

Definition 2.1.9. A group G is cyclic if it consists of the powers of some element $a \in G$. Such an element is called a generator of G . A finite cyclic group G is isomorphic to $\mathbb{Z}_n$, where n is the order of G .

Later, subgroups will play an important role. Lastly in this subsection groups will be written multiplicatively.

Definition 2.1.10. Let G be a group, then H is a subgroup of G denoted if $H \subseteq G$ and H is a group with the same operation as in G . In case G is abelian, the group's *torsion subgroup* can be defined as: $G_{\text{tor}} = \{x \in G \mid x \text{ is an element of finite order}\}$.

Note, we are using the notation G_{tor} only for this given section. Later on torsion subgroups will noted simply as r -torsion subgroups.

Definition 2.1.11. Let G be a finite group and let H be a subgroup of G . Then $|H|$ divides $|G|$. (Also known as Lagrange's theorem) Consequently, if $|G| = n$, and a has order m , then $H = \{e, a, \dots, a^{m-1}\}$ is a subgroup of G , and so $|H| = m$ divides $|G| = n$.

Now, the question arises, how does the subgroups relate to homomorphism. *Definition 2.1.12* gives us the definition of *image* and *kernel* as follows.

Definition 2.1.12. Given $\phi : G \rightarrow H$ group homomorphism, then image of ϕ is defined as $\text{Im}(\phi) = \{h \in H \mid h = \phi(g) \text{ for some } g \in G\}$. While the kernel of ϕ homomorphism is $\text{Ker}(\phi) = \{g \in G \mid \phi(g) = e\}$. Using *Lemma 2.3.29* [15] yields us with the following two observations: $\text{Im}(\phi)$ is also a subgroup of H , $\text{Ker}(\phi)$ is recognized as the normal subgroup of G .

2.1.2 Basic definitions and properties from Ring and general Field Theory

Let us continue with rings. The difference oppose to groups is that groups have one binary operation (denoted as $\cdot$), while rings have two related operations, *addition* and *multiplication*. Please note, [16] mentions numerous interesting properties, definitions and techniques from ring theory; however, due to space constraints, only those definitions that are absolutely necessary for understanding the content of the following chapters have been included in this subsection.

Definition 2.1.13. Given R set with addition $(+)$ and multiplication $(\cdot)$ operations. Given $(R, +)$ form an abelian group, then we have the following properties under multiplication operation:

1. **(Closure)** R is closed under multiplication for every $a, b \in R$: $a \cdot b \in R$.
2. **(Associativity)** for every $a, b, c \in R$ Multiplication is associative, $a \cdot (b \cdot c) = (a \cdot b) \cdot c$.
3. **(Distributivity over addition)** For all $a, b, c \in R$ multiplication distributes over addition as: $a(b + c) = ab + ac$ and $(b + c)a = ba + ca$.
4. **(Commutativity)** Multiplication is commutative, $a \cdot b = b \cdot a$, for every $a, b \in R$.

We also call R a ring with identity, if such $a \cdot 1 = 1 \cdot a = a$ exists for every $a \in R$ with condition $1 \neq 0$. (Note here, 1 denotes the multiplicative identity.)

On the otherhand R is called commutative ring with identity if R is both a ring with identity and multiplicative.

Definition 2.1.14. Let S denote the subset of ring R . Then S is also a subring of R if S is a ring with the same operations as R .

Similarly, if R is a ring with identity element, the the subset S of R is a subring with identity element as long as it has the same operation as R , and $1_S = 1_R$ ($1_R, 1_S$ here denotes the multiplicative identity element).

Definition 2.1.15. Next, we continue with ring homomorphisms. Depending on wheter we have rings with identity elements we may distinct the following cases

- (i) Ring homomorphism between rings R and S is a map $\phi : R \rightarrow S$ satisfying:
 $\phi(r_1 + r_2) = \phi(r_1) + \phi(r_2)$, and $\phi(r_1 r_2) = \phi(r_1)\phi(r_2)$ for all $r_1, r_2 \in R$
- (ii) A homomorphism of rings with 1 between R and S rings with 1 is the map $\phi : R \rightarrow S$ such that $\phi(1_R) = 1_S$. We also note the special case if R and S are *fields*, then $\phi : R \rightarrow S$ homomorphism of rings with $\phi(1_R) = 1_S$
- (iii) In either case, an invertible homomorphism is called an *isomorphism*. If there exists an isomorphism $\phi : R \rightarrow S$, then R and S are said to be *isomorphic*.

For the rest of this subsection we take a look at some basic properties of $\mathbb{Z}$ and polynomial rings.

Definition 2.1.16. Let R be a commutative ring with 1, and let $f(x) \in R[x]$ be a nonzero polynomial. Then the then the *degree* of $f(x)$ is n , if $f(x) = a_0 + a_1x + \cdots + a_nx^n$. Also the polynomial is called *monic* if the coefficient of its highest-degree term is equal to 1.

2.1.3 Fields

Definition 2.1.17. Using *Definition 2.1.13*, a field $\mathbb{F}$ is a commutative ring with multiplicative identity in which every nonzero element is invertible, denoted as $\mathbb{F}^* = \mathbb{F} - \{0\}$ (such that every nonzero element of the field is a unit). For example, given p prime, then $\mathbb{Z}_p = \{0, 1, \dots, p-1\}$ is a field with addition $(\text{mod } p)$, and multiplication $(\text{mod } p)$, hence we use notation $\mathbb{F}_p$.

Definition 2.1.18. The *characteristic* $\text{char}(\mathbb{F})$ of given $\mathbb{F}$ field is the smallest positive integer n such that $n \cdot 1 = 0 \in \mathbb{F}$ or 0 if no such positive integer exists. Using *Lemma 4.1.6* from [15] $\text{char}(\mathbb{F}) = 0$ or is a prime number. For example, let p denote a prime then $\mathbb{F}_p$ is a field of characteristic of p .

Definition 2.1.19. Let F be a field. A homomorphism $\varphi : \mathbb{F} \rightarrow \mathbb{F}$ is called an *endomorphism* of $\mathbb{F}$. If φ is an isomorphism, then it is called an *automorphism* of $\mathbb{F}$.

Definition 2.1.20. Let F be a field of characteristic p . The map $\Phi : \mathbb{F} \rightarrow \mathbb{F}$ given by $\Phi(a) = a^p$ for all $a \in \mathbb{F}$ is an endomorphism of $\mathbb{F}$ (called the Frobenius endomorphism, or automorphism, as the case may be).

Lastly we recall the definition of irreducible polynomial based on *Theorem 4.2.3* [15] as follows:

Definition 2.1.21. Let F be a field and let $p(x) \in \mathbb{F}[x]$ be an irreducible polynomial. Then the quotient ring $\mathbb{F}[x]/\langle p(x) \rangle$, where $\langle p(x) \rangle$ is the ideal (refer *Section 3.2* in [15]) generated by $p(x)$ in $F[x]$, is a field.

In the following *Section 2.2* we review the properties of finite field arithmetic in more detail.

2.2 Overview of Finite Field Arithmetic

Finite field arithmetic plays an important role as in elliptic-curve based cryptosystems because curve operations are performed using arithmetic operations in the underlying field. In present section building upon the previous *Section 2.1*, thus I review the formal definitions, theorems and lemmas related to finite fields: prime fields, binary fields and extension fields.

Please note, for this section I refer primarily [17], with the exception of those subsections where additional references are also cited.

2.2.1 Introduction to finite fields

Building upon *Subsection 2.1.3* we can think about fields as abstractions of number systems (such as the rational numbers $\mathbb{Q}$, the real numbers $\mathbb{R}$, and the complex numbers $\mathbb{C}$) and their essential properties. They consist of a set $\mathbb{F}$ together with two binary operations: addition (denoted by $+$) and multiplication (denoted by $\cdot$). Both operations satisfy the usual arithmetic properties as follows:

1. $(\mathbb{F}, +)$ forms an abelian group with (additive) identity denoted by 0.

2. $(\mathbb{F} \setminus \{0\}, \cdot)$ forms an abelian group with (multiplicative) identity denoted by 1.
3. The distributive law can be expressed as: $(a + b) \cdot c = a \cdot c + b \cdot c$ for all $a, b, c \in \mathbb{F}$.

If the set $\mathbb{F}$ is finite, then we call the field *finite*.

Field Operations

As I mentioned before, the field $\mathbb{F}$ is equipped with two binary operations, addition and multiplication. This also opens up the possibility to express subtraction of field elements in terms of addition: For $a, b \in \mathbb{F}$, $a - b = a + (-b)$ where $-b$ is the unique element in $\mathbb{F}$ such that $b + (-b) = 0$, where $-b$ is also called the *negative* of b .

Similarly, division of field elements is defined in terms of multiplication: for $a, b \in \mathbb{F}$ with $b \neq 0$, $a/b = a \cdot b^{-1}$, where b^{-1} is the unique element in $\mathbb{F}$ such that $b \cdot b^{-1} = 1$. In the expression b^{-1} is called the *inverse* of b .

Existence and Uniqueness criteria and Field Characteristics

The *order* of a finite field is the number of elements in the field. Then, there exists a finite field $\mathbb{F}$ of order q if and only if q is a prime power i.e., $q = p^m$ where p is a prime number and m is a positive integer. If $m = 1$, then $\mathbb{F}$ is called a *prime field*, while $m \geq 2$, then $\mathbb{F}$ is called an *extension field*.

For any prime power q , there is only one finite field of order q . On the other hand, this means that any two finite fields of order q are structurally the same, with the exception that the labeling used to represent the field elements may differ. We also say that any two finite fields of order q are *isomorphic* and denote such a field by $\mathbb{F}_q$.

Prime fields, Binary fields, Extension fields

Let p be a prime number. The integers modulo p (consisting of the integers $\{0, 1, 2, \dots, p-1\}$) with addition and multiplication (performed modulo p), is a finite field of order p . We denote this field by $\mathbb{F}_p$, furthermore we call p the *modulus* of $\mathbb{F}_p$. Furthermore we introduce the operation *reduction modulo p* as follows: for any integer a , $a \bmod p$ denote the unique integer remainder r , $0 \leq r \leq p-1$, obtained upon dividing a by p .

Finite fields of order 2^m are called *binary fields*, also known as *characteristic-two finite fields*. One way to construct $\mathbb{F}_{2^m}$ is to use a polynomial basis representation. The elements of $\mathbb{F}_{2^m}$ are the binary polynomials (polynomials whose coefficients are in the field $\mathbb{F}_2 = \{0, 1\}$) of degree at most $m-1$ as can be followed in *Equation 2.1*:

$$\mathbb{F}_{2^m} = \{a_{m-1}z^{m-1} + a_{m-2}z^{m-2} + \dots + a_2z^2 + a_1z + a_0 : a_i \in \{0, 1\}\}. \quad (2.1)$$

We note that (based on *Appendix A.1*[17]) an irreducible binary polynomial $f(z)$ of degree m exists for any m and can be efficiently found. By irreducibility of $f(z)$ we mean that $f(z)$ cannot be factored as a product of binary polynomials each of degree less than m , similarly as we seen before in *Definition 2.1.21*.

Example 2.2.1 (Binary field $\mathbb{F}_{2^4}$). Let us now follow with an example using *Equation 2.1*. The elements of $\mathbb{F}_{2^4}$ are the 16 binary polynomials of degree at most 3, can be represented as follows [17]:

$$\mathbb{F}_{2^4} = \left\{ \begin{array}{cccc} 0, & 1, & z, & z+1, \\ z^2, & z^2+1, & z^2+z, & z^2+z+1, \\ z^3, & z^3+1, & z^3+z, & z^3+z+1, \\ z^3+z^2, & z^3+z^2+1, & z^3+z^2+z, & z^3+z^2+z+1 \end{array} \right\}. \quad (2.2)$$

Taking into consideration the operations, addition of field elements may performed as the usual addition of polynomials, with coefficient arithmetic performed modulo 2. Multiplication of field elements on the other hand performed modulo the reduction polynomial $f(z)$. For any binary polynomial $a(z)$, $a(z) \bmod f(z)$ will denote therefore the unique remainder polynomial $r(z)$ of degree less than m obtained upon long division of $a(z)$ by $f(z)$. In the literature this operation is commonly referred as *reduction modulo $f(z)$* .

Example 2.2.2 (Examples of arithmetic operations in $\mathbb{F}_{2^4}$). Based on [17], let the reduction polynomial be $f(z) = z^4 + z + 1$, then

- **Addition** performed as: $(z^3 + z^2 + 1) + (z^2 + z + 1) = z^3 + z$.
- **Subtraction** performed as: $(z^3 + z^2 + 1) - (z^2 + z + 1) = z^3 + z$. We can use fact that $-1 = 1$ in $\mathbb{F}_2$, thus we have $-a = a$ for all $a \in \mathbb{F}_{2^m}$.
- **Multiplication**: $(z^3 + z^2 + 1) \cdot (z^2 + z + 1) = z^5 + 1$. As a first step we $(z^3 + z^2 + 1) \cdot (z^2 + z + 1) = z^5 + z + 1$, than $(z^5 + z + 1) \bmod (z^4 + z + 1) = z^2 + 1$.
- **Inversion**: can be performed as: $(z^3 + z^2 + 1)^{-1} = z^2$, since $(z^3 + z^2 + 1) \cdot z^2 \bmod (z^4 + z + 1) = 1$.

Extension Fields

We briefly mentioned field extensions at the end of *Subsection 2.1.2*, now let us continue with more precision. We will use the fact that the polynomial basis representation for any binary field can be generalized to all extension fields.

Let p be a prime and $m \geq 2$. Then, we introduce the notation $\mathbb{F}_p[z]$, which denotes the set of all polynomials in the variable z with coefficients from $\mathbb{F}_p$. Let $f(z)$, the *reduction polynomial*, be an irreducible polynomial (as we previously mentioned in *Subsection 2.2.1*) of degree m in $\mathbb{F}_p[z]$. The elements of $\mathbb{F}_{p^m}$ are the polynomials in $\mathbb{F}_p[z]$ of degree at most $m - 1$ can be expressed in accordance with *Equation 2.3*:

$$\mathbb{F}_{p^m} = \{a_{m-1}z^{m-1} + a_{m-2}z^{m-2} + \dots + a_2z^2 + a_1z + a_0 : a_i \in \mathbb{F}_p\}. \quad (2.3)$$

Furthermore the addition operation of field elements is the usual addition of polynomials, with coefficient arithmetic performed in $\mathbb{F}_p$. Multiplication of field elements is performed modulo the polynomial $f(z)$.

Bases of a Finite Field

The finite field $\mathbb{F}_{q^n}$ can be viewed as a vector space over its subfield $\mathbb{F}_q$. In this abstraction the vectors are elements of $\mathbb{F}_{q^n}$, scalars are elements of $\mathbb{F}_q$. Vector addition corresponds the addition operation in $\mathbb{F}_{q^n}$, while scalar multiplication corresponds the multiplication in $\mathbb{F}_{q^n}$ of $\mathbb{F}_q$ -elements with $\mathbb{F}_{q^n}$ -elements. The vector space has dimension n and has many bases.

If $\mathcal{B} = \{b_1, b_2, \dots, b_n\}$ is a basis, then $a \in \mathbb{F}_{q^n}$ can be uniquely represented by an n -tuple $(a_1, a_2, \dots, a_n)$ of $\mathbb{F}_q$ -elements where $a = a_1b_1 + a_2b_2 + \dots + a_nb_n$. In such representation for instance the polynomial basis representation of the field $\mathbb{F}_{p^m}$ is an m -dimensional vector space over $\mathbb{F}_p$ and $\mathcal{B} = \{z^{m-1}, z^{m-2}, \dots, z^2, z, 1\}$ is a basis for $\mathbb{F}_{p^m}$ over $\mathbb{F}_p$.

Example 2.2.3 (Polynomial basis representation of $\mathbb{F}_{2^4}$). Let us consider the field $\mathbb{F}_{2^4}$ with reduction polynomial $f(z) = z^4 + z + 1$ based on [17]. The polynomial basis over $\mathbb{F}_2$ is described as $\mathcal{B} = \{z^3, z^2, z, 1\}$. With every element $a \in \mathbb{F}_{2^4}$ can be uniquely written as $a = a_3z^3 + a_2z^2 + a_1z + a_0$, $a_i \in \{0, 1\}$.

For instance, the element $z^3 + z + 1$ has coordinate representation $(a_3, a_2, a_1, a_0) = (1, 0, 1, 1)$ with respect to $\mathcal{B}$, corresponding to the 4-bit string 1011.

Before me move to *Section 2.3* we close this introductory section with the multiplicative group definition for a given finite field. Let the nonzero elements of a $\mathbb{F}_q$ finite field denoted as $\mathbb{F}_q^*$. $\mathbb{F}_q^*$ the also form a cyclic group under multiplication, hence there exist elements $b \in \mathbb{F}_q^*$ called *generators* such that

$$\mathbb{F}_q^* = \{b^i : 0 \leq i \leq q-2\}. \quad (2.4)$$

The *order* of $a \in \mathbb{F}_q^*$ is the smallest positive integer t such that $a^t = 1$. Since $\mathbb{F}_q^*$ is a cyclic group, it follows that t is a divisor of $q-1$.

Before we dive into introduction of the elliptic curve operations over finite field (refer to *Section 2.4*) in detail, we will briefly gave a context about the elliptic curve security assumptions related to *Public-Key Cryptographic* systems.

2.3 Public-key Cryptography using Elliptic Curves

Elliptic curves have a rich and beautiful history, and having been studied by mathematicians for over a hundred years. They have been used to solve a diverse range of problems. For instance the congruent number problem asks for a classification of the positive integers occurring as the area of some right-angled triangle, the lengths of whose sides are rational numbers. Another example is proving *Fermat's Last Theorem* which states that the equation $x^n + y^n = z^n$ has no nonzero integer solutions for x, y and z when the integer n is greater than 2.

In 1985, Neal Koblitz and Victor Miller independently proposed their solution on how to design public-key cryptographic systems using elliptic curves. During the late 1990's elliptic curve systems started receiving commercial acceptance when accredited standards organizations specified elliptic curve protocols and private companies started to include these protocols in their commercial products.

However, there are usually several criterias that need to be addressed when selecting a family of public-key schemes for such domain-specific application. In general, these security principals are *functionality*, *security* and *performance*. The first one answers the question *does the chosen public-key family provide the desired capabilities ?*, while the second principal examines the assurance protocol security. Lastly, the third principle evaluates that the primary objective are meeting the desired level of security from the performance perspective.

In practice other factors (outside of the previous three mentioned) that may influence the decision are for instance the existence of best-practice standards (usually developed by accredited standards organizations), perhaps the availability of commercial cryptographic products, patent coverage, etc.

Let us depict the *Base Communication* model as shown in *Figure 2.1*. Given two entities *A* (Alice) and *B* (Bob) who are communicating over an *unsecured channel*. Such specific scenario may be arise when for example *A* and *B* are communicating over a cellular telephone network, or an individual person *A* who is in the process of purchasing a product from an online store *B* represented by its web site *B*. Another practical example when *A* is sending let say an email message to *B* over the Internet.

In every case, *E* could attempt to read the traffic from *A* being send to *B* from the unsecured channel, or modify selected portions of such message.

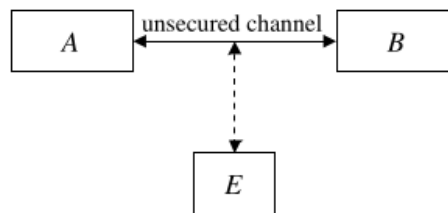


Figure 2.1: *Base Communication Model*. Entities *A* (Alice) and *B* (Bob) communicates over an unsecured channel. *E* (Eve) represents the malicious entity, who is able to read, modify messages and/or impersonate entites.

Given that specific scenario we are being presented by fundamental objectives of secure communications: Firstly, we can speak about *confidentiality* with the purpose of keeping the data secret from all but authorized party members to see it, for example the messages being sent by *A* to *B* should not be readable by *E*.

Next, we might think about *data integrity* which ensures that data has not been altered by any unauthorized means, therefore *B* should be able to detect when data sent by *A* has been modified by *E*. Similarly, this should yield the same result from the perspective of the other party member: *B* should be able to verify that data purportedly sent by *A* indeed originated with *A* (commonly referred as *entity authentication*).

The last objective is referred as *non-repudiation*, with goal to prevent an entity from denying previous commitments or actions. For instance when *B* receives a message purportedly from *A*, not only is *B* convinced that the message originated with *A*, but *B* can convince a neutral third party of this, thus *A* cannot deny having sent the message to *B*.

Based on the consideration how the communicating entities agree upon keyring

material during their communication, cryptographic systems can be broadly divided into two kinds: symmetric-key and public-key schemes as depicted in *Figure 2.2*. With the previously demanded security objectives in mind, and given the method of use it can be a decisive factor in choosing between symmetric and public-key cryptographic systems.

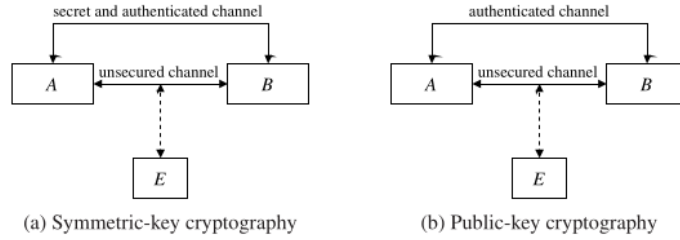


Figure 2.2: Comparison of Symmetric-key and Public-key Cryptography Schemes

In symmetric-key schemes (depicted in the left subfigure of *Figure 2.2*), the communicating entities first agree upon keying material that is both secret and authentic. Subsequently, they may use a symmetric-key encryption scheme such as the *Advanced Encryption Standard* (AES) [18] to achieve confidentiality, in conjunction with the use a *message authentication code* (MAC) algorithm such as HMAC[19] to achieve data integrity and to preserve origin authentication.

The major advantage of symmetric-key cryptography is its high efficiency. However, there are some significant drawbacks to these systems given our scenario. One primary drawback is the infamous *key distribution problem*, with the requirement for a channel to be both secret and authenticated for the distribution of keying material is essential. On top of that in a network of N entities, each entity may have to maintain different keying material with each of the other $N - 1$, as commonly referred in the *key management problem*.

2.3.1 Diffie-Hellman Public-Key Encryption/Decryption Scheme

In 1967 Diffie and Hellman showed that two parties can establish a shared secret over an insecure channel (such as depicted in *Figure 2.1*) without any prior knowledge of one another. Since one cannot gain any intuition towards their groundbreaking result without being aware of the existence of *one-way functions*, I will primarily refer [7, 3, 20, 17] in this subsection and for the following *Subsection 2.3.2*.

A *one-way function* is one that is easy to compute in "one direction", but which is believed infeasible to compute from the other direction. In their paper Diffie and Hellman used the exponentiation in finite fields as an example to introduce the notion of public-key cryptography. More specifically, they showed that a party member A can bury their secret value a by raising some publicly known generator g to the power of a in $\mathbb{F}_q$: for instance, by computing $g_a = g^a \in \mathbb{F}_q$. Then A 's public key (denoted as g_a) can then be sent over the insecure channel to another party B , whose secret value is b , to which B can exponentiate g_a and compute $g_{ab} = g_b^a$. Conversely, b can compute their public key $g_b = g^b$ and send it over an insecure channel to A , who can compute the shared secret $gab = g_a^b$.

In contrast to symmetric-key schemes, public-key schemes (as can be followed in *Figure 2.2*) only requires the authenticity of the exchange keying material of communicating entities, but which does not have to be secret. Each entity then selects a single key pair (e, d) consisting of a public key e , and a related private key d (that the entity keeps secret). Such keys have the property that it is computationally infeasible to determine the private key solely from knowledge of the public key.

Let say an entity A wishes to send entity B a confidential message noted as m , she she will obtain an authentic copy of B 's public key e_B . As a next step she uses the encryption function of a public-key encryption scheme to compute the ciphertext, formulated as $c = \text{ENC}_{e_B}(m)$. A then transmits c to B , who uses the decryption function on his private key d_B to recover the plaintext, formulated as $m = \text{DEC}_{d_B}(c)$.

In such relation it is essential only that A obtain an authentic copy of e_B , thus confidentiality can be satisfied. Furthermore to satisfy non-repudiation objective *Digital Signature Schemes* may be devised for data origin authentication and data integrity.

In both cases the individual secrets of both parties are assumed to be safe under the *discrete logarithm problem* (DLP) in $\mathbb{F}_q$, which is defined as follows: given $g \in \mathbb{F}_q$ and $h = g^x \in \mathbb{F}_q$, find x . If q is large enough, so that solving the DLP is infeasible, then the exponentiation is the *one-way function* that keeps a and b out of reach for eavesdroppers and attackers. In addition, we assume that computing $g_{ab} = g^{ab}$ is infeasible for adversaries that know g, g_a and g_b , yielding us with the Diffie-Hellman problem in $\mathbb{F}_q$. [7, 3]

Discrete Logarithm Problem

The first published public-key construction by Diffie and Hellman was based on the discrete logarithm problem in the finite field $\mathbb{F}_p$, where p denotes a (large) prime. Using the *Primitive Root Theorem*[21] tells us, that given p prime number, then there exists an element $\mathbb{F}_p^*$ whose powers give every element of $\mathbb{F}_p^*$ in the form of *Equation 2.5*:

$$\mathbb{F}_p^* = \{1, g, g^2, g^3, \dots, g^{p-2}\} \quad (2.5)$$

The elements of *Equation 2.5* with this property are called the *primitive roots* of $\mathbb{F}_p$ or generators of $\mathbb{F}_p^*$, as we saw previously in *Definition 2.4*. Such generators can be seen as the elements of $\mathbb{F}_p^*$ having order of $p - 1$. This means that every nonzero element of $\mathbb{F}_p$ is equal to some power of g . Now, with the definition of primitive root for $\mathbb{F}_p$, we can reformulate the *Discrete Logarithm Problem* as follows [21].

Definition 2.3.1. Let g be a primitive root for $\mathbb{F}_p$, while h be a nonzero element of $\mathbb{F}_p$. Than the *Discrete Logarithm Problem* (DLP) is the problem to find an exponent of x such that:

$$g^x \equiv h \pmod{p}, \quad (2.6)$$

where x is called the *discrete logarithm of h to the base g* and is denoted by $\log_g(h)$.

2.3.2 Security assumptions related to Elliptic Curve-based Cryptosystems

For the sake of completeness, we must have mention the case of the discrete logarithm problem discussed in *Subsection 2.3.1* as it pertains to elliptic curves as well. However, we note that there are several known formulations of the discrete logarithm problem (refer to [22]), but this subsection includes only those definitions that, based on sources [17] and [3], aid in understanding the subsequent sections.

Since the hardness of the elliptic curve discrete logarithm problem is essential for the security of all elliptic curve cryptographic schemes first we start the generalization of the discrete logarithm problem for elliptic curves.

Definition 2.3.2. The elliptic curve discrete logarithm problem (ECDLP) is as follows. Given an elliptic curve E defined over a finite field $\mathbb{F}_q$, a point $P \in E(\mathbb{F}_q)$ of order n , and a point $Q \in \langle P \rangle$. The goal is to find the integer $l \in [0, n-1]$ such that $Q = lP$. Where the integer l is called the discrete logarithm of Q to the base P , denoted as $l = \log_P Q$.

While hardness of the ECDLP is necessary for the security of any elliptic curve cryptographic scheme, it is generally not sufficient. As the referenced articles presents, some problems related to the ECDLP whose hardness is assumed in the security proofs for some elliptic curve protocols.

Definition 2.3.3. The (computational) elliptic curve Diffie-Hellman problem (ECDHP) is: given an elliptic curve E defined over a finite field $\mathbb{F}_q$, a point $P \in E(\mathbb{F}_q)$ of order n and points $A = aP, B = bP \in \langle P \rangle$. Then the problem can be expressed as to find the point $C = abP$.

If the ECDLP in $\langle P \rangle$ can be efficiently solved, then the ECDHP in $\langle P \rangle$ can also be efficiently solved by first finding a from (P, A) and then computing $C = aB$. Thus the ECDHP is no harder than the ECDLP.

The elliptic curve decision Diffie-Hellman problem (ECDDHP) is formulated as follows:

Definition 2.3.4. Given an elliptic curve E defined over a finite field $\mathbb{F}_q$, a point $P \in E(\mathbb{F}_q)$ of order n , and points $A = aP, B = bP, C = cP \in \langle P \rangle$. The goal is to determine whether $C = abP$ or $c \not\equiv ab \pmod{n}$.

If the ECDHP in $\langle P \rangle$ can be efficiently solved, then the ECDDHP in $\langle P \rangle$ can also be efficiently solved by first finding $C' = abP$ from (P, A, B) and then comparing C' with C . Thus the ECDDHP is no harder than the ECDHP.

In the following *Section 2.4*, we continue with the introduction to elliptic curves starting with curve operations over finite fields and point representations.

2.4 Elliptic Curve operations over Finite Fields

Using the basic definitions from *Subsection 2.1.3* and properties related to finite fields ($\mathbb{F}_q$) from *Subsection 2.2*, we can think of an elliptic curve as an abstract type of group. More generally, elliptic curves are groups which are defined on top of fields. Even though elliptic curve groups permit only one binary operation (the so called *group law*), the operation itself is computed within the underlying field, which by definition permits two operations (and their inverses).

Throughout the following subsections I refer mainly refer [5, 17].

2.4.1 Introduction to Elliptic Curves

For a given general field K , the group elements of an elliptic curve E are points whose (x, y) coordinates come from $\bar{K}$ (denotes the algebraic closure of K) and which satisfy the following (affine) curve equation for E and can be written as the form in Equation 2.7:

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \quad (2.7)$$

where $a_1, \dots, a_6 \in \bar{K}$. We say that E elliptic curve is defined over K because the coefficients a_1, a_2, a_3, a_4, a_6 of its defining equation are elements of K . Sometimes we write E/K to emphasize that E is defined over K , where K is called the *underlying field*. Note that if E is defined over K , then E is also defined over any extension field of K .

The discriminant of E (denoted as Δ) can be formulated as follows in Equation 2.8:

$$\Delta = -d_2^2d_8 - 8d_4^3 - 27d_6^2 + 9d_2d_4d_6 \quad (2.8)$$

$$\begin{cases} d_2 = a_1^2 + 4a_2 \\ d_4 = 2a_4 + a_1a_3 \\ d_6 = a_3^2 + 4a_6 \\ d_8 = a_1^2a_6 + 4a_2a_6 - a_1a_3a_4 + a_2a_3^2 - a_4^2 \end{cases} \quad (2.9)$$

The condition $\Delta \neq 0$ ensures that the elliptic curve is *smooth*, which means that there are no points at which the curve has two or more distinct tangent lines. Before we dissect the Weierstrass equation in detail, let us unpack what smoothness means more precisely. For simplicity's sake, let's assume that we are not interpreting our curve over a finite field, but it is defined over $\mathbb{Q}$. In that case we can conclude that not every tuple $(a, b) \in K \times K$ gives rise to the curve given by $f(x, y) = y^2 - (x^3 + ax + b) = 0$ being an elliptic curve. For instance, if there exists $P = (x_P, y_P)$ point of f such that both partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ vanishes simultaneously at P , then P is called a singular point and f is also deemed singular. Conversely, if no such point exists on the curve, f is called *non-singular*, or *smooth*, and is then an elliptic curve as can be followed on Figure 2.3

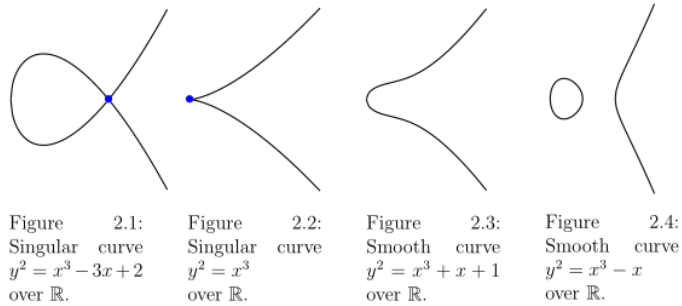


Figure 2.3: Illustrating the difference between singular and non-singular (smooth) elliptic curves (defined on field $\mathbb{Q}$), based on [5]

We also note that singularity occurs if and only if the discriminant of the (*Figure 2.21*) Weierstrass equation equals 0. Namely, if $4a^3 + 27b^2 = 0$. Therefore as long as $4a^3 + 27b^2 \neq 0$ in K , then $E/K : y^2 = x^3 + ax + b$ will be an elliptic curve.

The *Equation 2.7* is also called the general Weierstrass equation for elliptic curves. Aside from all the $(x, y) \in \bar{K}$ solutions to general Weierstrass equation, there exists one extra point which can not be defined using the affine equation, but which must be included to complete the group definition: given L extension field of K , then the set of L -rational points on E is formulated in accordance with *Equation 2.10*.

$$E(L) = \{(x, y) \in L \times L : y^2 + a_1xy + a_3y - x^3 - a_2x^2 - a_4x - a_6 = 0\} \cup \{\mathcal{O}\} \quad (2.10)$$

This point denoted by $\mathcal{O}$ is called the point at infinity (refer *Section 2.4.4*), which is the only point on the line at infinity that satisfies the projective form of the Weierstrass equation (refer *Subsection 2.4.4*). The L -rational points on E are the points (x, y) that satisfy the equation of the curve and whose coordinates x and y belong to L . The point at infinity is considered an L -rational point for all extension fields L of K .

We consider separately the cases where the underlying field K has characteristic different from 2 and 3, or has characteristic equal to 2 or 3.

1. If the characteristic of K is not equal to 2 or 3, then the admissible change of variables

$$(x, y) \longrightarrow \left(\frac{x - 3a_1^2 - 12a_2}{36}, \frac{y - 3a_1x}{216} - \frac{a_1^3 + 4a_1a_2 - 12a_3}{24} \right) \quad (2.11)$$

transforms E to the curve

$$y^2 = x^3 + ax + b \quad (2.12)$$

where $a, b \in K$. The discriminant of this curve is $\Delta = -16(4a^3 + 27b^2)$.

2. If the characteristic of K is 2, then there are two cases to consider.

- (a) If $a_1 \neq 0$, then the admissible change of variables

$$(x, y) \longrightarrow \left(a_1^2x + \frac{a_3}{a_1}, a_1^3y + \frac{a_1^2a_4 + a_3^2}{a_1^3} \right) \quad (2.13)$$

transforms E to the curve

$$y^2 + xy = x^3 + ax^2 + b \quad (2.14)$$

where $a, b \in K$. Such a curve is said to be *non-supersingular* and has discriminant $\Delta = b$.

- (b) If $a_1 = 0$, then the admissible change of variables

$$(x, y) \longrightarrow (x + a_2, y) \quad (2.15)$$

transforms E to the curve

$$y^2 + cy = x^3 + ax + b \quad (2.16)$$

where $a, b, c \in K$. Such a curve is said to be *supersingular* and has discriminant $\Delta = c^4$.

3. If the characteristic of K is 3, then there are two cases to consider.

(a) If $a_1^2 \neq -a_2$, then the admissible change of variables

$$(x, y) \longrightarrow \left(x + \frac{d_4}{d_2}, y + a_1x + \left(a_1 \frac{d_4}{d_2} + a_3 \right) \right) \quad (2.17)$$

where $d_2 = a_1^2 + a_2$ and $d_4 = a_4 - a_1a_3$, transforms E to the curve

$$y^2 = x^3 + ax^2 + b \quad (2.18)$$

where $a, b \in K$. Such a curve is said to be *non-supersingular* and has discriminant $\Delta = -a^3b$.

(b) If $a_1^2 = -a_2$, then the admissible change of variables

$$(x, y) \longrightarrow (x, y + a_1x + a_3) \quad (2.19)$$

transforms E to the curve

$$y^2 = x^3 + ax + b \quad (2.20)$$

where $a, b \in K$. Such a curve is said to be *supersingular* and has discriminant $\Delta = -a^3$.

Let us consider the case where the field characteristic (refer *Definition 2.1.18*) is not equal to 2 or 3. Then divisions by 2 and 3 in K permit the substitution $y \mapsto \left(\frac{y - a_1x - a_3}{2} \right)$, yielding $E : y^2 = 4x^3 + b_2x^2 + 2b_4x + b_6$. Furthermore using the map $(x, y) \mapsto \left(\frac{x - 3b_2}{36}, \frac{y}{108} \right)$, yields us the simplified Weierstrass form for elliptic curves as can be followed in *Equation 2.21* :

$$E : y^2 = x^3 + ax + b \quad (2.21)$$

Since in practical cryptographic application we will be working over large prime fields, the short Weierstrass presented in *Equation 2.21* covers all possible isomorphism classes of elliptic curves (refer *Definition 2.4.1*) for our purpose, meaning the curves we use later on will always be an instance of the short Weierstrass equation.

Definition 2.4.1 (Isomorphic elliptic curves). Given elliptic curves E_1 and E_2 defined over K and given by the Weierstrass equations

$$E_1 : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad (2.22)$$

$$E_2 : y^2 + a'_1xy + a'_3y = x^3 + a'_2x^2 + a'_4x + a'_6 \quad (2.23)$$

are said to be *isomorphic over K* if there exist $u, r, s, t \in K$, $u \neq 0$, such that the change of variables

$$(x, y) \longrightarrow (u^2x + r, u^3y + u^2sx + t) \quad (2.24)$$

transforms equation E_1 into equation E_2 . The transformation (2.24) is called an *admissible change of variables*.

Example 2.4.1. Lastly, let consider Equation 2.21 over $\mathbb{F}_{11}$ in the form of $y^2 = x^3 + 4x + 3$ based on [5].

The form the quadratic extension $\mathbb{F}_{q^2} = \mathbb{F}_q(i)$ with $i^2 + 1 = 0$ then considering E over $\mathbb{F}_{q^2}$ gives us many more solutions and give many more points (denoted as $\#E(\mathbb{F}_{q^2})$ in this case). In addition to the points in $E(\mathbb{F}_q)$, $E(\mathbb{F}_{q^2})$ will also contain those points with x -coordinates in $\mathbb{F}_q$ that did not give $x^3 + 4x + 3$ as a quadratic residue in $\mathbb{F}_q$, but it does in $\mathbb{F}_{q^2}$ (and more with both coordinates in $\mathbb{F}_{q^2} \setminus \mathbb{F}_q$).

Which leads to the following (later useful) general insight: $\#E(\mathbb{F}_q) | \#E(\mathbb{F}_{q^2})$, since $E(\mathbb{F}_q)$ is a subgroup of $E(\mathbb{F}_{q^2})$

2.4.2 Point Representations over general K field

The formulas for adding two points on the elliptic curves will be introduced later (refer Subsection 2.4.3). For now we assume that E elliptic curve $y^2 = x^3 + ax + b$ defined over a field K , furthermore we assume such K field has a characteristic that is neither 2 nor 3, and for $y^2 + xy = x^3 + ax^2 + b$ the curve is defined over a binary field K .

For both curves, the formulas for point addition (i.e., adding two distinct finite points that are not negatives of each other) and point doubling require a field inversion and several field multiplications. If inversion in K is significantly more expensive than multiplication, then it may be advantageous to represent points using *projective coordinates*.

We also note that in the literature more variants are mentioned next to *Affine*, *Projective (Standard or Homogeneous, Jacobian)* projections, such as *Chudnovsky* ($P = (X, Y) = (x/z^2, y/z^3) = (x, y, z, z^2, z^3)$), *Lopez-Dahab* ($P = (X, Y) = (x/z, y/z^2) = (x, z, y)$) or various mixed notation. However, we restrict our attention to Projective (Standard, Jacobian) cases, since these are the two substitution that we are going to use later on.

As a start, we first introduce projective coordinate representations for elliptic-curve points referring [5, 17].

Projective coordinates

Let K be a field, and let c and d be positive integers. One can define an equivalence relation (denoted as $\sim$) on the set $K^3 \setminus \{(0, 0, 0)\}$ of nonzero triples over K by

$$(X_1, Y_1, Z_1) \sim (X_2, Y_2, Z_2) \quad \text{if} \quad X_1 = \lambda^c X_2, Y_1 = \lambda^d Y_2, Z_1 = \lambda Z_2 \quad \text{for some } \lambda \in K^*. \quad (2.25)$$

The equivalence class containing $(X, Y, Z) \in K^3 \setminus \{(0, 0, 0)\}$ is

$$(X : Y : Z) = \{(\lambda^c X, \lambda^d Y, \lambda Z) : \lambda \in K^*\}. \quad (2.26)$$

$(X : Y : Z)$ is called a *projective point*, and (X, Y, Z) is called a *representative* of $(X : Y : Z)$. The set of all projective points is denoted by $\mathbb{P}(K)$. We can notice that if $(X', Y', Z') \in (X : Y : Z)$ then $(X' : Y' : Z') = (X : Y : Z)$; meaning, any element of an equivalence class can serve as its representative. In particular, if $Z \neq 0$, then $(X/Z^c, Y/Z^d, 1)$ is a representative of the projective point $(X : Y : Z)$, and in fact is the only representative with Z -coordinate equal to 1. Thus we have a 1-1 correspondence between the set of projective points as can be followed in 2.27 Equation 2.28:

$$\mathbb{P}(K)^* = \{(X : Y : Z) : X, Y, Z \in K, Z \neq 0\} \quad (2.27)$$

$$(K) = \{(x, y) : x, y \in K\}. \quad (2.28)$$

The set of projective points

$$\mathbb{P}(K)^0 = \{(X : Y : Z) : X, Y, Z \in K, Z = 0\} \quad (2.29)$$

is called the *line at infinity* since its points do not correspond to any of the affine points.

The projective form of Weierstrass equation (2.31) of an elliptic curve E defined over K is obtained by replacing x by X/Z^c and y by Y/Z^d , and clearing denominators. Now, if $(X, Y, Z) \in K^3 \setminus \{(0, 0, 0)\}$ satisfies the projective equation then so does any $(X', Y', Z') \in (X : Y : Z)$.

Therefore we say that the projective point $(X : Y : Z)$ lies on E . We thus have a one-to-one correspondence between the affine points in (K) that lie on E and the projective points in $\mathbb{P}(K)^*$ that lie on E . The projective points in $\mathbb{P}(K)^0$ which lie on E are the *points at infinity* on E .

Example 2.4.2 (Standard projective coordinates, based on [5]). Given $c = 1$ and $d = 1$ constants. Then the projective form of the Weierstrass equation

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad (2.30)$$

defined over K can be rewritten as

$$Y^2Z + a_1XYZ + a_3YZ^2 = X^3 + a_2X^2Z + a_4XZ^2 + a_6Z^3. \quad (2.31)$$

The only point on the line at infinity that also lies on E is $(0 : 1 : 0)$. This projective point corresponds to the point $\mathcal{O}$ as introduced before in *Equation 2.10*.

Formulas that do not involve field inversions for adding and doubling points in projective coordinates can be derived by first converting the points to affine coordinates, then using the formulas from 2.35 to add the affine points, and finally clearing denominators as can be followed in *Equation 2.36*. Also of use in point multiplication methods (refer 2.38 and *Equation 2.39*) is the addition of two points can be achieved in *mixed coordinates* - where the two points are given in different coordinate systems.

Example 2.4.3 (Addition formulas using Jacobian coordinates, based on [5]). Let $c = 2$ and $d = 3$. The projective point $(X : Y : Z)$, $Z \neq 0$, corresponds to the affine point $(X/Z^2, Y/Z^3)$. The projective form of the Weierstrass equation

$$E : y^2 = x^3 + ax + b \quad (2.32)$$

defined over K is

$$Y^2 = X^3 + aXZ^4 + bZ^6. \quad (2.33)$$

The point at infinity, $\mathcal{O}$ corresponds to $(1 : 1 : 0)$, while the negative of $(X : Y : Z)$ is $(X : -Y : Z)$.

Point doubling. Let $P = (X_1 : Y_1 : Z_1) \in E$, and suppose that $P \neq -P$. Since $P = (X_1/Z_1^2 : Y_1/Z_1^3 : 1)$, we can use the doubling formula for E in affine coordinates to compute $2P = (X'_3 : Y'_3 : 1)$, obtaining

$$X'_3 = \left(\frac{3\frac{X_1^2}{Z_1^4} + a}{2\frac{Y_1}{Z_1^3}} \right) - \left(2\frac{X_1}{Z_1^2} \right) = \frac{(3X_1^2 + aZ_1^4)^2 - 8X_1Y_1^2}{4Y_1^2Z_1^2} \quad (2.34)$$

$$Y'_3 = \left(\frac{3\frac{X_1^2}{Z_1^4} + a}{2\frac{Y_1}{Z_1^3}} \right) \left(\frac{X_1}{Z_1^2} - X'_3 \right) - \frac{Y_1}{Z_1^3} = \frac{3X_1^2 + aZ_1^4}{2Y_1Z_1} \left(\frac{X_1}{Z_1^2} - X'_3 \right) - \frac{Y_1}{Z_1^3} \quad (2.35)$$

To eliminate denominators in the expressions for X'_3 and Y'_3 , we set $X_3 = X'_3 \cdot Z_3^2$ and $Y_3 = Y'_3 \cdot Z_3^3$ where $Z_3 = 2Y_1Z_1$, and obtain the following formulas for computing $2P = (X_3 : Y_3 : Z_3)$ in Jacobian coordinates:

$$\begin{cases} X_3 = (3X_1^2 + aZ_1^4)^2 - 8X_1Y_1^2 \\ Y_3 = (3X_1^2 + aZ_1^4)(4X_1Y_1^2 - X_3) - 8Y_1^4 \\ Z_3 = 2Y_1Z_1 \end{cases} \quad (2.36)$$

By storing some intermediate elements, X_3 , Y_3 and Z_3 can be computed using six field squarings and four field multiplications as follows:

$$\begin{aligned} A &\leftarrow Y_1^2, & B &\leftarrow 4X_1 \cdot A, & C &\leftarrow 8A^2, \\ D &\leftarrow 3X_1^2 + a \cdot Z_1^4, \\ X_3 &\leftarrow D^2 - 2B, \\ Y_3 &\leftarrow D \cdot (B - X_3) - C, & Z_3 &\leftarrow 2Y_1 \cdot Z_1. \end{aligned} \quad (2.37)$$

Point addition using mixed Jacobian-affine coordinates. Let $P = (X_1 : Y_1 : Z_1) \in E$, $Z_1 \neq 0$, and $Q = (X_2 : Y_2 : 1)$, and suppose that $P \neq \pm Q$. Since $P = (X_1/Z_1^2 : Y_1/Z_1^3 : 1)$ we can use the addition formula for E in affine coordinates to compute $P + Q = (X'_3 : Y'_3 : 1)$, obtaining

$$X'_3 = \left(\frac{Y_2 - \frac{Y_1}{Z_1^3}}{X_2 - \frac{X_1}{Z_1^2}} \right)^2 - \frac{X_1}{Z_1^2} - X_2 = \left(\frac{Y_2Z_1^3 - Y_1}{(X_2Z_1^2 - X_1)Z_1} \right)^2 - \frac{X_1}{Z_1^2} - X_2, \quad (2.38)$$

$$Y'_3 = \left(\frac{Y_2 - \frac{Y_1}{Z_1^3}}{X_2 - \frac{X_1}{Z_1^2}} \right) \left(\frac{X_1}{Z_1^2} - X'_3 \right) - \frac{Y_1}{Z_1^3} = \left(\frac{Y_2Z_1^3 - Y_1}{(X_2Z_1^2 - X_1)Z_1} \right) \left(\frac{X_1}{Z_1^2} - X'_3 \right) - \frac{Y_1}{Z_1^3}. \quad (2.39)$$

To eliminate denominators in the expressions for X'_3 and Y'_3 , we set $X_3 = X'_3 \cdot Z_3^2$ and $Y_3 = Y'_3 \cdot Z_3^3$, where $Z_3 = (X_2Z_1^2 - X_1)Z_1$.

To obtain the following formulas for computing $P + Q = (X_3 : Y_3 : Z_3)$ in Jacobian coordinates:

$$\begin{cases} X_3 = (Y_2 Z_1^3 - Y_1)^2 - (X_2 Z_1^2 - X_1)^2 (X_1 + X_2 Z_1^2), \\ Y_3 = (Y_2 Z_1^3 - Y_1)(X_1(X_2 Z_1^2 - X_1) - X_3) - Y_1(X_2 Z_1^2 - X_1)^2, \\ Z_3 = (X_2 Z_1^2 - X_1)Z_1. \end{cases} \quad (2.40)$$

By storing some intermediate elements, X_3 , Y_3 , and Z_3 can be computed using three field squarings and eight field multiplications in accordance with the referenced sources as follows:

$$\begin{array}{ll} A \leftarrow Z_1^2, & G \leftarrow E^2, \\ B \leftarrow Z_1 \cdot A, & H \leftarrow G \cdot E, \\ C \leftarrow X_2 \cdot A, & I \leftarrow X_1 \cdot G, \\ D \leftarrow Y_2 \cdot B, & X_3 \leftarrow F^2 - (H + 2I), \\ E \leftarrow C - X_1, & Y_3 \leftarrow F \cdot (I - X_3) - Y_1 \cdot H, \\ F \leftarrow D - Y_1, & Z_3 \leftarrow Z_1 \cdot E. \end{array}$$

At the beginning of this subsection, we started with elliptic curves defined over the general field K . In the following *Subsection 2.4.3*, however, we will examine in more detail how the projection can be applied to curves defined over a finite field taking into consideration the operations of point addition and multiplication based on [17, 5].

2.4.3 The chord-and-tangent rule: Group law

The *group law* exploits the fact that, over any field, a line (represented by a degree one equation in x and y) intersects a degree three equation in x and y (represented by a cubic curve) in three places: Given a line $\ell : y = \lambda x + \nu$ equation between $P = (x_P, y_P)$ points and $Q = (x_Q, y_Q)$ on E elliptic curve.

After substituting the line into Equation 2.21 $E : y^2 = x^3 + ax + b$ will give a cubic polynomial in x . The roots then will be represented as the x -coordinates of the three points of intersection between ℓ and E . After we get to know x_P and x_Q (the two roots) allows us to determine a unique third root that corresponds to the third and only other point in the affine intersection $\ell \cap E$ (denoted by $\ominus R$). Then using the "flipping" operation on the point $\ominus R$ over the x -axis will result the point R . In general, the elliptic curve composition law $\oplus$ is defined by $R = P \oplus Q$.

In case we want to compute $R = P \oplus Q$, the line ℓ is the tangent to E at P . Thus, the derivatives of ℓ and E are matched at P , meaning that ℓ intersects E "twice" at P as can be followed on *Figure 2.4*

After the definition of the general group operation, let us now take into consideration the role of the point at infinity $\mathcal{O}$. One can think of it as being a point that simultaneously sits infinitely high and infinitely low in the y direction. This allows us to informally conceptualise two properties of elliptic curve groups. Namely, the point at infinity denotes the *identity* of the group, secondly, the unique inverse of a point is its reflected image over the x -axis (similar to the "flipping" of $\ominus R$ as can be seen on *Figure 2.4*).

For instance, let's compute $R \oplus (\ominus R)$. The vertical line that connects R and $\ominus R$ intersects E twice at the point at infinity $\mathcal{O}$, which is then reflected back onto itself,

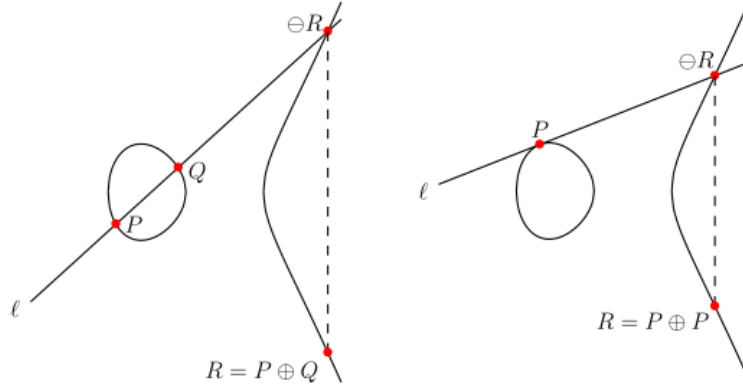


Figure 2.4: Illustration of Elliptic curve addition (left) and curve doubling (right) in accordance of the chord-and-tangent rule, based on [5]

yielding $R \oplus (\ominus R) = \mathcal{O}$. Consequently, if the identity of the group is chosen to be $\mathcal{O}$, then then the inverse of any element $R = (x_R, y_R)$ can be calculated as $\ominus R = (x_R, -y_R)$

As a justification, one can follow the geometrical representation on *Figure 2.5* why (in cryptography) we only ever use elliptic curves over finite field. However, we note that since we will be using enormously large p primes, such geometrical representation has little use from practical point of view.

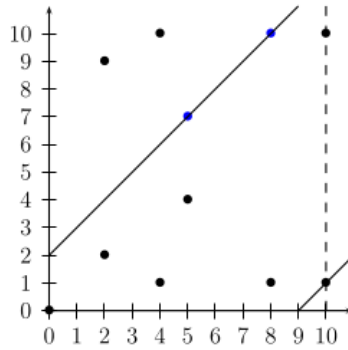


Figure 2.5: Representation of points on $E(\mathbb{F}_{11})$ elliptic curve from *Example 2.4.1*, excluding the point at infinity. The image is based on [5]

2.4.4 Representing the point at infinity in projective space

So far elliptic curves were described in affine space as a set of affine points together with the point of infinity until *Subsection 2.4.1*. We can summarize more formally as $E = \{(x, y) \in \mathbb{A}^2(\overline{K}) : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$.

A more precise way however to unify or include points at infinity with the affine points is to work in projective space for instance, choose *lines* that pass through the origin in $(n+1)$ -space. Furthermore this can be interpreted as the affine points in 2-space become lines in 3-space. Thus $(x, y) \in \mathbb{A}^2(\overline{K})$ corresponds to the line defined

by points of the form $(\lambda x, \lambda y, \lambda) \in \mathbb{A}^2(\bar{K})$, where $\lambda \in \bar{K}^*$. Even more precisely, the $\mathbb{P}^2$ projective plane is $\{(0,0,0)\} \setminus \mathbb{A}^3$ modulo the $(x_1, y_1, z_1) \sim (x_2, y_2, z_2)$ congruence condition if there exists such $\lambda \in \bar{K}^*$ such that $(x_1, y_1, z_1) = (\lambda x_2, \lambda y_2, \lambda z_2)$

As can be followed on *Figure 2.6* we will use capital letters and colons to denote a (representative of a) congruence class in projective coordinates. In general $(X : Y : Z)$ represents the set of all points on the line in $\mathbb{P}^2$ that corresponds to $(x, y) \in \mathbb{A}^2$. Traditionally the affine point $(x, y) \in \mathbb{A}^2$ is mapped to projective space via the inclusion $(x, y) \mapsto (x : y : 1)$, and for any $(X : Y : Z) \neq \mathcal{O} \in \mathbb{P}^2$ will be mapped back to $\mathbb{A}^2$ via the $(X : Y : Z) \mapsto (X/Z, Y/Z)$ mapping. This latter case also corresponds to the *Projective (Standard)* mapping we have mentioned at the introduction of *Section 2.4.2*.

Lastly the $\mathcal{O}$ point at infinity will be represented by $(0 : 1 : 0)$ in projective space, for which we can note that the map back to $\mathbb{A}^2$ is ill-defined.[5]

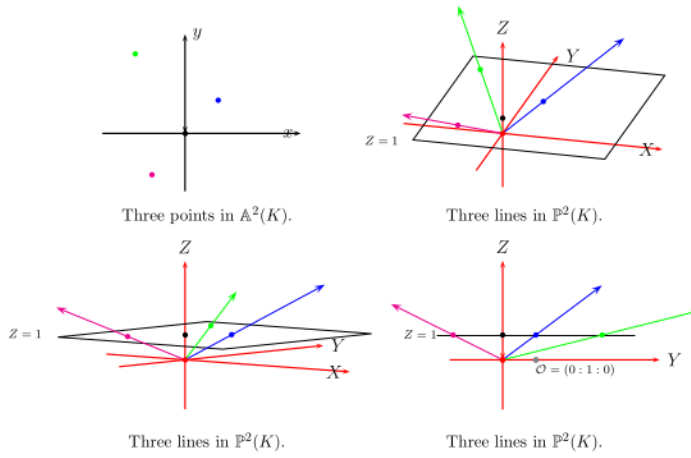


Figure 2.6: Identification of points in $\mathbb{A}^2$ affine plane with lines in $\mathbb{P}^2$ projective plane, based on [5]

Another way to define the collection of points in projective space can be achieved through homogenization of the $E : y^2 = x^3 + ax + b$ Weierstrass equation. By the technique we are using substitution $x = X/Z$, $y = Y/Z$ while multiplying the equation by Z^3 (which corresponds to denominator clearing) as can be expressed more formally in accordance with *Equation 2.41*.

$$E_{\mathbb{P}} : Y^2 Z = X^3 + aXZ^2 + bZ^3 \quad (2.41)$$

We also note, that the set of points (X, Y, Z) with coordinates in $\bar{K}$ that also satisfies *Equation 2.41* is called the projective closure of E . On the other hand $(0, \lambda, 0)$ is in the projective closure for all $\lambda \in \bar{K}$ and using the fact that such points cannot be mapped into $\mathbb{A}^2$ justifying the representative of point at infinity being $\mathcal{O} = (0 : 1 : 0)$.

The projective coordinates X, Y, Z used to replace the x, y affine coordinates above are called homogenous projective coordinates, because the projective version of the curve equation in *Equation 2.41* is homogeneous. In *Equation 2.41* we used coordinate substitutions as $x = X/Z, y = Y/Z$, but we are not restricted to this choice of substitution.

Explicit formula derivation for group law computations

Taking into consideration the chord-and-tangent process that has been previously summarised *Figure 2.4*, we are able to derive explicit formulas for computing the elliptic curve group law.

First, we have to derive the formulas in affine space, then transferring them into projective space. The derivation of the formulas for point additions ($R = P \oplus Q$) and for point doublings ($R = P \oplus P$) are following the same steps with exception the calculation of the gradient of the λ of the line: $\ell : y = \lambda x + v$.

Next we have to distinguish between the general and special cases of point additions. In the general case ($R = P \oplus Q$) during group law operations between points neither point is $\mathcal{O}$. This means that the line $\ell : y = \lambda x + v$ that intersects $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ has the gradient $\lambda = (y_Q - y_P) / (x_Q - x_P)$ (refer to *Figure 2.4*).

At this step v can be calculated as $v = y_P - \lambda x_P$ or as $v = y_Q - \lambda x_Q$ or as the average of the two: $\lambda = (y_Q x_P - y_P x_Q) / (x_P - x_Q)$. As we did before, substituting line equation $\ell : y = \lambda x + v$ into $E : y^2 = x^3 + ax + b$, yields us the third affine point of the intersection (denotes as $\ominus R$ in $\ell \cap E$). Finding the coordinates of $\ominus R$ also reveals the coordinates of $R = (x_R, y_R)$, since $\ominus R = (x_R, -y_R)$. The roots of the cubic will be x_P , x_Q and x_R as can be followed in *Equation 2.42*.

$$\begin{aligned} (x - x_P)(x - x_Q)(x - x_R) &= (x^3 + ax + b) - (\lambda x + v)^2 \\ &= x^3 - \lambda^2 x^2 + (a - 2\lambda v)x + b - v^2 \end{aligned} \quad (2.42)$$

To determine x_R we need to look at the coefficient of x^2 . Since the coefficient on the left hand side of *Equation Figure 2.42* is $-(x_P + x_Q + x_R)$, recovering the y -coordinate is can be achieved in accordance with *Equation 2.43*.

$$x_R = \lambda^2 - 2x_P; \quad y_R = -(\lambda x_R + v) \quad (2.43)$$

This finishes the description of point addition in the general case. In case of point doubling (e.g. adding P to itself) the line equation $\ell : y = \lambda x + v$ represents the tangent to E at point P . The λ gradient is than equivalent to the derivative function dy/dx evaluated at point P as follows:

$$\begin{aligned} \frac{d}{dx} (y^2) &= \frac{d}{dx} (x^3 + ax + b) \\ \frac{dx}{dy} (y^2) \frac{dy}{dx} &= 3x^2 + a \\ \frac{dy}{dx} &= \frac{3x^2 + a}{2y} \end{aligned}$$

The gradient evaluation at point P can be achieved as $\lambda = \frac{dy}{dx} (P) = (3x_P^2 + a) / (2y_P)$, thus x_R and y_R can be obtained as follows in *Equation 2.44*:

$$x_R = \lambda^2 - 2x_P \quad y_R = -(\lambda x_R + v) \quad (2.44)$$

With *Equation 2.43* and *2.44* we have now the group law description, therefore we are able to take a look at the special cases.

Our second scenario is, where the point at infinity is the identity or neutral element is trivial. Otherwise any operation between elements P and Q with different x -coordinates employs the general addition. This leaves the following remaining cases of $x_P = x_Q$: If $y_P = -y_Q$, then P and Q are inverses of each other and $P \oplus Q = \mathcal{O}$, which also includes $y_P = y_Q = 0$. The second case is when $y_P = y_Q \neq 0$, then $P = Q$ and the point doubling formulas can be used.

In [5] it is noted that in optimised cryptographic implementations however, this is not the way that the group law operation is coded. The groups we use are so large that the chances of running into a special case (that is not general doubling or general addition) randomly is negligible. Moreover, the parameters are usually chosen so that we are guaranteed not to run into these cases. The major operations we are concerned with are point additions ($R = P \oplus Q$) and point doublings ($R = P \oplus P$), as referred as *affine addition* (refer to *Equation 2.45*) and *addine doubling* (*Equation 2.46*) respectively, where $[2]P$ denotes the scalar multiplication of point P .

$$\begin{aligned} \text{Affine addition} \quad \lambda &= \frac{y_Q - y_P}{x_Q - x_P} & v &= y_P - \lambda x_P \\ (x_P, y_P) \oplus (x_Q, y_Q) &= (x_R, y_R) = (\lambda^2 - x_P - x_Q, -(\lambda x_R + v)) \end{aligned} \quad (2.45)$$

$$\begin{aligned} \text{Affine doubling} \quad \lambda &= \frac{3x_P^2 + a}{2y_P} & v &= y_P - \lambda x_P \\ [2](x_P, y_P) &= (x_R, y_R) \oplus (x_P, y_P) = (x_R, y_R) = (\lambda^2 - x_P - x_Q, -(\lambda x_R + v)) \end{aligned} \quad (2.46)$$

Lastly in this subsection, we examine the *Multiplication-by- m* map on E as defined in accordance with *Equation 2.47*.

$$[m] : E \rightarrow E, \quad P \mapsto [m]P \quad (2.47)$$

The operation in *Equation 2.47* is analogous to exponentiation $g \mapsto g^m$ in $\mathbb{Z}_q^*$. It is the one-way operation that buries discrete logarithm problems in $E(\mathbb{F}_q)$ (refer to *Subsection 2.3.2*).

To efficiently compute the exponentiation g^m in $\mathbb{Z}_q^*$ we square-and-multiply, whilst to compute multiplication $[m]P$ in $E(\mathbb{F}_q)$ we use *double-and-add*.

Example 2.4.4. Motivated by *Example 2.1.7* and *example 2.1.8* from [5]. Refer to *Appendix A.18* in *Appendix Section A*

To do *double-and-add*, we write the binary representation of m as $(m_9, \dots, m_0)_2$, initialising $T \leftarrow P$ and starting from the second most significant bit (denoted by m_8 in our case). As a last step we can compute $T \leftarrow [2]T$ for each bit down to m_0 and whenever $m_i = 1$ we compute $T \leftarrow T + T$. In general then, this straightforward double-and-add algorithm will take $\log_m(2)$ doublings and roughly half as many additions to compute $[m]P$, in case m is randomly chosen.

2.4.5 Speeding up the group law computations

Group law computations on elliptic curves are more complex than computations in traditional groups that facilitate discrete logarithm based protocols like $\mathbb{F}_q^*$. However, in our case elliptic curve groups of a relatively small size may achieve the same conjectured security as multiplicative groups in much larger finite fields, for instance $E(\mathbb{F}_{q_1})$ achieves similar security as $\mathbb{F}_{q_2}^*$ when $q_2 \gg q_1$.

Before we consider the latter case as highlighted in Table 2.1 based on [23], let us review how to compare algorithm strengths as proposed within [24]. Cryptographic algorithms can provide different “strengths” of security, depending on the algorithm and the key size used (when keys are required by the algorithm). A security strength is a number associated with the amount of work (i.e., the number of operations) that is required to break the cryptographic algorithm or system. Such security strengths of the approved algorithms are for example 128, 192, and 256 bits.

Furthermore, an algorithm that has a Y -bit key and an estimated maximum security strength that is comparable to a symmetric-key algorithm with an X -bit key is said to have an *estimated maximum security strength of X bits* or to be able to provide X bits of security, refer to [24] for in-depth details about Symmetric Block Cipher and Asymmetric-Key algorithm comparison.

Security (bits)	RSA	Discrete Log		ECC
		field size	subfield	
48	480	480	96	96
56	640	640	112	112
64	816	816	128	128
80	1248	1248	160	160
112	2432	2432	224	224
128	3248	3248	256	256
160	5312	5312	320	320
192	7936	7936	384	384
256	15424	15424	512	512

Table 2.1: Comparison of fields size based on keysize equivalence according to [23, 24]

This case for us means, that although more field operations are required to perform a group law computation, operations that take place in a field whose operational complexity is much less provides us the possibility to tip the balance in the favour of elliptic curves.

In addition, the smaller group elements in $E(\mathbb{F}_{q_1})$ implies much smaller key sizes, which in return greatly reduces storage and bandwidth requirements. One key observation that has been made in order to minimize the field operations was that performing computations in projective space avoids field inversions, which are extremely costly in practice.

Example 2.4.5 (Cost of field operations, based on [5]). Let us consider a general Weierstrass curve over the field $E(\mathbb{F}_q)$ where q is a some large prime.

Then, the Weierstrass form is given as: $y^2 = x^3 + ax + b$. For the sake of easier readability let **M** denote the cost of computing multiplications, **S** the cost of squarings and **I** be the cost of inversions in $\mathbb{F}_q$ respectively.

Using the just introduced notations, the cost of a general affine point doubling $(x_R, y_R) = [2](x_P, y_P)$ costs $2\mathbf{M} + 2\mathbf{S} + \mathbf{I}$ using Equation 2.46, and to a general affine point addition using Equation 2.45 $(x_R, y_R) = (x_P, y_P) \oplus (x_Q, y_Q)$ costs $2\mathbf{M} + \mathbf{S} + \mathbf{I}$.

On the other hand as showed in Subsection 2.4.4 before, we can transform the formulas into homogeneous projective space substitutions formulas as $x = X/Z$, $y = Y/Z$. As before, now we can consider computing

$(X_R : Y_R : Z_R) = [2](X_P : Y_P : Z_P)$ and $(X_R : Y_R : Z_R) = ((X_P : Y_P : Z_P)) \oplus ((X_Q : Y_Q : Z_Q))$ on $E_{\mathbb{P}} : Y^2Z = X^3 + aXZ^2 + bZ^3$.

For the addition case, substituting $x_i = X_i/Z_i$ and $y_i = Y_i/Z_i$ for $i \in \{P, Q, R\}$ into the affine formulas from Equation 2.45 yields:

$$x_R = \left(\frac{y_Q - y_P}{x_Q - x_P} \right)^2 - x_P - x_Q; \quad y_R = \left(\frac{y_Q - y_P}{x_Q - x_P} \right) (x_P - x_R) - y_P$$

, thus:

$$\frac{X_R}{Z_R} = \left(\frac{\frac{Y_Q}{Z_Q} - \frac{Y_P}{Z_P}}{\frac{X_Q}{Z_Q} - \frac{X_P}{Z_P}} \right)^2 - \frac{X_P}{Z_P} - \frac{X_Q}{Z_Q}; \quad \frac{Y_R}{Z_R} = \left(\frac{\frac{Y_Q}{Z_Q} - \frac{Y_P}{Z_P}}{\frac{X_Q}{Z_Q} - \frac{X_P}{Z_P}} \right) \left(\frac{X_P}{Z_P} - \frac{X_R}{Z_R} \right) - \frac{Y_P}{Z_P}$$

Now, to be able to get acquire X_R, Y_R, Z_R coordinates we can set Z_R to be the smallest value that contains both denominators above, and update the numerators accordingly to give:

$$\begin{aligned} X_R &= (X_P Z_Q - X_Q Z_P)(Z_P Z_Q (Y_P Z_Q - Y_Q Z_P)^2 - (Z_P Z_Q - X_Q Z_P)^2 (X_P Z_Q + X_Q Z_P)); \\ Y_R &= Z_P Z_Q (X_Q Y_P - X_P Y_Q)(X_P Z_Q - X_Q Z_P)^2 - (Y_P Z_Q - Y_Q Z_P) \\ &\quad ((Y_P Z_Q - Y_Q Z_P)^2 Z_P Z_Q - (X_P Z_Q + X_Q Z_P)(X_P Z_Q - X_Q Z_P)^2); \\ Z_R &= Z_P Z_Q (X_P Z_Q - X_Q Z_P)^3 \end{aligned}$$

We note that the report from the explicit formulas database (EFD, refer [25]) reports that X_R, Y_R, Z_R can be computed in a total of $12\mathbf{M} + 2\mathbf{S}$. On top of that, adopting projective coordinates for computations provides the advantage for optimized implementations of $\mathbb{F}_q$, where arithmetic have $\mathbf{I} \gg 20\mathbf{M}$, and the multiplication to inversion ratio is commonly reported to be 80:1 or higher.

Therefore in accordance with [25] $12\mathbf{M} + 2\mathbf{S}$ used for additions in projective space will be much faster than $2\mathbf{M} + \mathbf{S} + \mathbf{I}$ the for affine additions. Since the main topic of this thesis is not faster group operations we leave this subsection with closing remark that the research to develop such operation are field on it's own, therefore are many more algorithms to choose from as outlined within [26].

In the following Section 2.5 we introduce the basic definition for divisors from algebraic geometry that are fundamental to the understanding of cryptographic pairing computations based on [5, 17].

2.5 Divisor Theory

Before we dive into the topic on divisors more detail, we first recall how quotients of polynomials behaves when two variables are represents points on an elliptic curve based on [27]. Let $\mathbb{F}_{q^k}[X, Y]$ denote the ring of polynomials in variables X, Y with coefficients in $\mathbb{F}_{q^k}$. Furthermore assume we will define E elliptic curve with $Y^2 = X^3 + aX + b$ over $\mathbb{F}_{q^k}$. Then $E(X, Y)$ can be written as the polynomial $Y^2 - X^3 - aX - b$.

In accordance with [27] one can show that a polynomial f satisfies $f(X, Y) = 0$ $E(X, Y) = 0$ if and only if $f(X, Y)$ is a multiple of $E(X, Y)$. This means that from all the possible polynomials we consider exactly only two of them to be equivalent if they take the same values on all points of E , denoted as $\mathbb{F}_{q^k}[E] = \mathbb{F}_{q^k}[X, Y]_{E(X, Y)}$.

This observation provides us with the idea that every element $f(X, Y)$ in $\mathbb{F}_{q^k}[E]$ can be written in the form $f_x(X) + Yf_y(X)$ for polynomials $f_x, f_y \in \mathbb{F}_{q^k}[X]$ because we may replace Y^2 by $X^3 + aX + b$. Precisely, if we define $\mathbb{F}_{q^k}(E)$ to be the field of fractions of $\mathbb{F}_{q^k}[E]$, then the elements of $\mathbb{F}_{q^k}(E)$ can be written in the form $f(X, Y)/g(X, Y)$ where f, g are polynomials in two variables X, Y and g is not a multiple of E .

Recall the naming conventions from [28], we say that a rational function $f \in K(E)$ on an elliptic curve E have a *zero* at a point $P \in E$ if $f(P) = 0$ and a *pole* at P if $f(P) = \infty$. Following the previously outlined idea, it can be noted that the *zeroes* are the roots of f and the *poles* are the roots of g , therefore f/g is determined up to a constant by its zeroes and poles and their multiplicities. Suppose the zeroes and poles are denoted with $\alpha_1, \dots, \alpha_n$. Let $a_1, \dots, a_n$ denote the multiplicities where α_i is a zero of multiplicity a_i . We also consider a zero of negative multiplicity a_i to be a pole of multiplicity as $-a_i$.

Then we write $f(X)/g(X)$ by multiplying equations of appropriate vertical lines:

$$f(X)/g(X) = (X - \alpha_i)^{a_i} \quad (2.48)$$

The main advantage of the generalization shown in *Equation 2.48* is that one can view roots of a polynomial f as places where the curve $Y = f(X)$ intersects with the curve $Y = 0$. Consequently, we can further generalize by replacing $Y = 0$ with an elliptic curve. Thus we can call the points of intersection between $f(X, Y)$ and E zeroes of f , where $f(X, Y) \in \mathbb{F}_{q^k}[E]$.

However, similarly as before, complications will occur due to the point at infinity. We already presented *Section 2.4.4* how one may use projective coordinates to handle the point at infinity. Now, building upon the technique presented before and in conjunction with the idea generalized in *Equation 2.48* it turns out that every line $\alpha X + \beta Y + \gamma$ that intersects a given elliptic curve E has a pole at $\mathcal{O}$ of some multiplicity.

To describe the multiplicities at zeroes and poles one can take advantage by introducing the uniformizing parameter d for P , based on [17]. Given all $P \in E$ there exists a rational function $u \in \overline{K}(E)$, $u(P) = 0$, such that if $f \in \overline{K}(E)^*$, then

$$f = u^d s \quad (2.49)$$

, where $s \in \overline{K}(E)$, $s(P) \neq 0, \infty$. The function u in *Equation 2.49* called a *uniformizing parameter* for P , where the integer d does not depend on the choice of u . However, one question arises regarding how to choose the uniformizing parameter. To answer the question, we can distinguish between three different cases using *Equation 2.49* and *Theorem 2.22* from [28] as follows in *Example 2.5.1*.

Example 2.5.1 (Distinct cases for the uniformizing parameter.). Given $E : y^2 = x^3 + ax + b$ over finite field $K = \mathbb{F}_q$, with $\text{char}(K) \neq 2, 3$. Refer to *Code Snippet A.20* in *Appedix A.3*

- Let $P = (c, d) \notin E[2]$. Then tangent line to E evaluated at P is as follows:

$$(-3c^2 - a)(x - c) + 2d(y - d) = 0$$

, Since $d \neq 0$, a uniformizing parameter for P is $u = x - c$.

- Let $P = (c, 0) \in E$ be a point of order 2. The tangent line to E evaluated at P is as follows:

$$(-3c^2 - a)(x - c) = 0.$$

- To find a suitable uniformizing parameter for $\mathcal{O}$, we recall the coordinate substitution technique presented in *Subsection 2.4.4*. First, we choose the affine coordinates as $v = X/Y$; $w = Z/Y$, therefore the E equation is transformed to $f(v, w) = v^3 + avw^2 + bw^3 - w = 0$. We note, that $\mathcal{O} = (0, 0)$ using (v, w) -coordinate representation, thus:

- First, $\frac{\partial f}{\partial v}(\mathcal{O}) = 0$ and $\frac{\partial f}{\partial w}(\mathcal{O}) = -1$, thus the equation of the tangent line to E evaluated at $\mathcal{O}$ is $w = 0$,
- Secondly, the line $v = 0$ passes through $\mathcal{O}$ and is not the tangent line at $\mathcal{O}$,
- Lastly, reverting back to the original (x, y) coordinates we found that $u = x/y$ is a uniformizing parameter for $\mathcal{O}$.

For instance, if a vertical line (represented as $X - \alpha$) intersects the E elliptic curve then it has a pole at $\mathcal{O}$ of multiplicity 2. Secondly, if a non-vertical line (represented as $\alpha X + \beta Y + \gamma$) intersects E then it has a pole of multiplicity 3 at $\mathcal{O}$. Using the just introduced notation in *Equation 2.49*, we can formulate the order of f at P as $\text{ord}_P(f) = d$. Therefore the point P is a *zero* of f if and only if $\text{ord}_P(f) > 0$, in case its multiplicity is defined to be $\text{ord}_P(f)$. On the other hand, the point P is a *pole* of f if and only if $\text{ord}_P(f) < 0$, then its multiplicity is defined to be $-\text{ord}_P(f)$.

To make our case easier, we will introduce the concept of divisors, which is standard way to notate zeroes, poles and their multiplicities, or orders referring to [27]. Given the scenario where we wish to record zeroes and poles $P_1, \dots, P_n$ of orders $a_1, \dots, a_n$ (where P_i is a zero of order a_i if $a_i > 0$ and is a pole of order $-a_i$ otherwise, with generalization of ord_P notation), then we represent the divisor as follows in *Equation 2.50*:

$$D = a_1 \langle P_1 \rangle + \dots + a_n \langle P_n \rangle \quad (2.50)$$

With the notation introduced in 2.50, if we add the divisors of two functions together, then the resulting divisor reflects the zeroes and poles of the product of the two functions. Similarly, given the scenario where D given the divisor with zeros and poles $P_1, \dots, P_n$ of multiplicities $a_1, \dots, a_n$. Then we can construct D as follows in 2.51:

$$D = (P_1)^{a_1} \dots (P_n)^{a_n} \quad (2.51)$$

We note, that the following statements apply to all curves C over any perfect field K and its closure $\overline{K}$, however, for now we place the discussion in our context and specialise to the case where C is an elliptic curve E defined over a finite field as formulated by $K = \mathbb{F}_q$ as before.

Using the definition of divisor (denoted by D) on E is also a convenient way to denote the formal sum of multi-set of points on E with finitely many zeroes ($n_P \in \mathbb{Z}$) as written in *Equation 2.52*:

$$D = \sum_{P \in E(\overline{\mathbb{F}})_q} n_P(P) \quad (2.52)$$

We call the summarization in *Equation 2.52* formal because the parentheses around P and the absence of square parentheses around the n_P is differentiates from the notation of the actual sum of points on E (see *Section 2.4.3*), where the square brackets denoted the sum of points using the group law. The set of all divisors on E is thereby denoted by $\text{Div}_{\overline{\mathbb{F}}}(E)$. Also, the set of all divisors forms a group, where addition of divisors is natural, and the identity is the divisor with all $n_P = 0$, where the zero divisor $0 \in \text{Div}_{\overline{\mathbb{F}}}(E)$. In addition, we interpret the degree of a divisor denoted by $\text{Deg}(D) = \sum_{P \in E(\overline{\mathbb{F}}_q)} n_P$, and the the set of support can be understood as $\text{supp}(D) = \{P \in E(\overline{\mathbb{F}}_q) : n_P \neq 0\}$.

Example 2.5.2 (Properties of divisors.). Consider $P, Q, R, S \in E(\overline{\mathbb{F}}_q)$. Let $D_1 = 2(P) - 3(Q)$ and $D_2 = 3(Q) + (R) - (S)$, thus the degree of divisors: $\text{Deg}(D_1) = 2 - 3 = -1$, and $\text{Deg}(D_2) = 3 + 1 - 1 = 3$ respectively. The sum of divisors equals to $D_1 + D_2 = 2(P) + (R) - (S)$, furthermore $\text{Deg}(D_1 + D_2) = \text{Deg}(D_1) + \text{Deg}(D_2) = 2$. The supports are $\text{supp}(D_1) = \{P, Q\}$, $\text{supp}(D_2) = \{Q, R, S\}$. The sum of the support then equals to $\text{supp}(D_1 + D_2) = \{P, R, S\}$.

Let f denotes the the function associating divisors on E convenient way to write down the intersection points (and their multiplicities) of f and E . $\text{ord}_P(f)$ count the multiplicity of f at P which is positive if f has zero at P and negative if f has a pole at P . Using the just introduced notation the divisor of a function f as (f) and it is defined as the divisor as can be followed in *Equation 2.53*

$$(f) = \sum_{P \in E(\overline{\mathbb{F}}_q)} \text{ord}_P(f)(P) \quad (2.53)$$

In *Section 2.4.4* we've already seen examples of functions on E used in the chord-and-tangent rule, and it is natural that we are really only interested in the points of intersection of ℓ and E , which is exactly what the divisor (ℓ) tells us.

Example 2.5.3 (Illustrative example based on *Figure 2.7*.). As an illustrative example we use *Figure 2.7*, where the chord ℓ intersects E in P , Q and $-(P+Q)$, all with multiplicity 1. We can also observe that ℓ intersects E with multiplicity -3 at $\mathcal{O}$, since ℓ has a pole of order 3 at $\mathcal{O}$. Thus, ℓ has divisor $(\ell) = (P) + (Q) + (-(P+Q)) - 3(\mathcal{O})$. Refer to *Code Snippet A.21* in *Appendix A.3*.

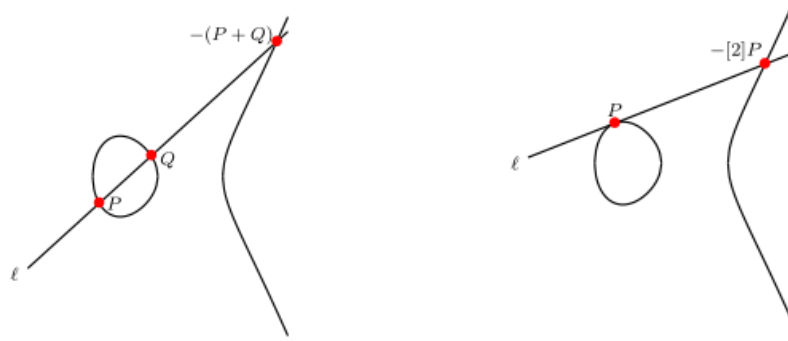


Figure 2.7: The divisor (l) intersects E $(l) = (P) + (Q) + (-(P+Q)) - 3(\mathcal{O})$
 Figure 2.8: Line ℓ intersection with P point-doubling. $(\ell) = 2(P) + (-[2]P) - 3(\mathcal{O})$

Example 2.5.4 (Illustrative example based on Figure 2.8.). Another case is, where the tangent ℓ intersects E with multiplicity 2 at P , with multiplicity 1 at $-[2]P$, and with multiplicity -3 at $\mathcal{O}$ as can be seen Figure 2.8, as in this case $(l) = 2(P) + (-[2]P) - 3(\mathcal{O})$. Refer to Code Snippet A.22 in Appedix A.3.

In both cases, as depicted on 2.7 and Figure 2.8 we have $\deg(\ell) = 0$. As illustrated, for any function f on E we will always have $\deg(\ell) = 0$ as stated in Theorem 7.7.1 [29]. This property follows from observing that the degree of the affine equation that solves for the zeros of f on E matches the degree of the projective equation that determines the multiplicity of the pole of f at $\mathcal{O}$. This corresponds to the projective version of $f = g/h$ where g and h both have the same degree as f .

The algebra between functions naturally translates across to the algebra between their divisors, meaning, that $(fg) = (f) + (g)$ and $(f/g) = (f) - (g)$ therefore $(f) - (g) = 0$ if and only if f is constant. For instance, given $(f) = (g)$ then $(f/g) = 0$, thus if f is a constant multiple of g , it means consequently that the divisor (f) determines f up to non-zero scalar multiples.

Example 2.5.5. From Example 2.5.3 we know that three zeros (counting multiplicities) will always arise from substituting $\ell : y = \lambda x + v$ into $E/\mathbb{F}_q : y^2 = x^3 + ax + b$. However, during the example we only considered ℓ on the affine curve $E \cap \mathbb{A}^2$, where ℓ has no poles. To consider the case ℓ on E at $\mathcal{O} = (0 : 1 : 0)$ (in $\mathbb{P}^2(\mathbb{F}_q)$), we need to apply $x = X/Z$ and $y = Y/Z$ substitutions which gives $\left(\frac{\lambda X + vZ}{Z}\right)^2 = \left(\frac{X}{Z}\right)^3 + a\left(\frac{X}{Z}\right) + b$, therefore we have a pole of order 3 when $Z = 0$.

In the following Subsection 2.5.1 we will now focus on divisor class groups, subgroups of divisors, as well the implications arising from their group structures.

2.5.1 The divisor class group and consequence of the Riemann-Roch Theorem

The set of degree zero divisors forms a proper subgroup, which we write as $\text{Div}^0(E) \subset \text{Div}(E)$, constructed as $\{D \in \text{Div}(E) : \text{Deg}(D) = 0\}$. We will call D *principal* divisor, if a divisor D on E and the set of principal divisors naturally form a group, written as $\text{Prin}(E)$. Principal divisors furthermore have degree zero, but there are also degree zero divisors that are not the divisors of a function, so the degree zero subgroup is strictly larger than the principal divisors i.e $\text{Prin}(E) \in \text{Div}^0(E)$

There is, however, an extra condition on elements of $\text{Div}^0(E)$. Namely, $D = \sum_P n_P(P) \in \text{Div}^0(E)$ is principal if and only if $\sum_P [n_P]P = \mathcal{O}$ on E in accordance with *Theorem IX 2.* from [30]. Before we define the divisor class group of E , we look at the important notion of divisor equivalence in $\text{Div}(E)$. We call the divisors D_1 and D_2 equivalent (denoted as $D_1 \sim D_2$), if $D_1 = D_2 + (f)$ for some function f .

The *divisor class group* (or *Picard group*) of E is defined as the quotient group as can be followed *Equation 2.54*.

$$\text{Pic}^0(E) = \text{Div}^0(E) / \text{Prin}(E) \quad (2.54)$$

Equation 2.54 means that the divisor class group is the group of all degree zero divisors modulo the principal divisors on E . Let us consider *Example 2.5.6* with the just introduced notion to describe the elliptic curve group law in terms of divisors, following from *Chapter IX.2* [30].

Example 2.5.6 (Describe the elliptic curve group law in terms of divisors.). Referring back go *Figure or 2.6* in the case that $Q = P$ the line ℓ joining P, Q has divisor: $(\ell) = (P) + (Q) + (-R) - 3(\mathcal{O})$, whilst the vertical line $v = x - x_R$ has divisor $(v) = (-R) + (R) - 2(\mathcal{O})$. The quotient $\frac{\ell}{v}$ has divisor $(\frac{\ell}{v}) = (P) + (Q) - (R) - (\mathcal{O})$. Thus the equation $R = P + Q$ on E is the same as the divisor equality $(R) - (\mathcal{O}) = (P) - (\mathcal{O}) + (Q) - (\mathcal{O}) - \left(\frac{\ell}{v}\right)$.

Now, to concretely connect this back to *Equation 2.54* both $(R) - (\mathcal{O})$ and $(P) + (Q) - 2(\mathcal{O})$ are in $\text{Div}^0(E)$, thus they represent the same class in $\text{Pic}^0(E)$ because the divisor is principal, and therefore zero in $\text{Pic}^0(E)$.

Following up on the notation of divisor equivalence allows us to reduce divisors of any size $D \in \text{Pic}^0(E)$ into much smaller divisors. To get a glimpse about the size of D divisor, we first have to introduce the *effective* conditionality regarding the divisors. A divisor $(D = \sum_P n_P(P))$ called effective if $n_P \leq 0$ for all $P \in E$. Consequently, the only divisor in $\text{Div}^0(E)$ that is effective is the zero divisor. Thus, we define the effective part of a divisor D as $\varepsilon(D) = \sum_P n_P(P)$ where $n_P \geq 0$.

As an illustrative example, consider the divisor $D = (P) + (Q) - 2(\mathcal{O})$: Such divisor is not effective, but the *effective part* can be formulated as $\varepsilon(D) = (P) + (Q)$. Now, using the effective property we can give a more precise definition about the size of divisor. From now on the degree of the effective part will be recognized as the size of D divisor. As in our example, although $\text{Deg}(D) = 0$ it has size equals exactly 2, since $\text{Deg}(\varepsilon(D)) = 2$.

In order to discuss the Riemann-Roch theory in detail, it would be necessary to introduce additional definitions and their proofs primarily from the fields of complex analysis and algebraic geometry, which would go beyond the scope of this thesis. However, based on [5] the corollary we use is the following: for any given curve C there is a unique minimal integer g of C (called the *genus* of the curve), such that any divisor $D \in \text{Pic}^0(E)$ is equivalent to a divisor D' with $\deg(\varepsilon(D')) \leq g$. The Elliptic curves (denoted by E) we will be referring to are genus $g = 1$ curves, meaning that every $D \in \text{Pic}^0(E)$ can be written as $(P_1) - (Q_1)$, hence we are able to reduce a divisor with given size into an equivalent divisor with reduced size in $\text{Pic}^0(E)$ as follows in *Example 2.5.7* with the aid of *Figure 2.9*.

Example 2.5.7 (Divisor reduction using the Riemann-Roch Theorem.). Consider divisor $D = (P_1) + \dots + (P_{11}) - 11(\mathcal{O})$ as an element of $\text{Pic}^0(E)$ on $E/\mathbb{F}_q : y^2 = x^3 + ax + b$, where the P_i are not necessarily distinct. The degree of the effective part of such divisor is $\text{Deg}(\varepsilon(D)) = 11$. To find a divisor that is equivalent to D , we: construct function $\ell_{10} : y = a_{10}x^{10} + \dots + a_1x + a_0$ to interpolate the distinct points in $\text{supp}(D)$ with appropriate multiplicities. After we substitute ℓ_{10} into E , we will be granted with a degree 20 polynomial in x , meaning that the roots will reveal the 20 affine points of intersection (counting multiplicities) between ℓ_{10} and E . We are using the fact that 11 of these are already known points (the P_i 's), so let $P'_1, \dots, P'_9$ be the other 9. On the other hand we note that these points are not necessarily defined over $\mathbb{F}_q$.

Since

$$(\ell_{10}) = \sum_{i=1}^{11} (P_i) + \sum_{i=1}^9 (P'_i) - 20(\mathcal{O}) \in \text{Prin}(E), \quad D' = - \left(\sum_{i=1}^9 (P'_i) \right) - 9(\mathcal{O})$$

is a divisor equivalent to D in $\text{Pic}^0(E)$, i.e. $D' \sim D$.

We can repeat this process, interpolating the points in $\text{supp}(D')$ with a degree 8 polynomial $\ell_8 : y = a'_8x^8 + \dots + a'_1x + a'_0$, which will intersect E (in the affine sense) 16 times, giving 7 new intersection points, thereby finding a divisor.

We observe that the maximum number of divisors in the consecutive supports decreases each time by two, so that in two more steps we will arrive at $\tilde{D} = (\tilde{P}_1) + (\tilde{P}_2) + (\tilde{P}_3) - 3(\mathcal{O})$. We can interpolate the three points in $\text{supp}(\tilde{D})$ with a quadratic function $\tilde{\ell} : y = \tilde{a}_2x^2 + \tilde{a}_1x + \tilde{a}_0$ that clearly intersects E at one more affine point, say Q .

That is, $(\tilde{\ell}) = (\tilde{P}_1) + (\tilde{P}_2) + (\tilde{P}_3) + (Q) - 4(\mathcal{O})$, and since $(\tilde{\ell}) \in \text{Prin}(E)$, then $(\tilde{D}) \sim (\mathcal{O}) - (Q)$. Lastly, the vertical line $\tilde{v}$ has divisor $(\tilde{v}) = (Q) + (R) - 2(\mathcal{O})$, meaning $(\mathcal{O}) - (Q) \sim (R) - (\mathcal{O})$, which gives $(\tilde{D}) \sim (R) - (\mathcal{O})$. To summarise, we started with a divisor $D = (P_1) + \dots + (P_{11}) - 11(\mathcal{O})$ which had size 11, and reduced to the equivalent divisor $(R) - (\mathcal{O}) \sim D$ in $\text{Pic}^0(E)$ which has size 1.

Using the one-to-one correspondence between the divisor class group $\text{Pic}^0(E)$ and the fact that points on E has the group homomorphisms $P \mapsto (P) - (\mathcal{O})$ we are now able to describe the chord-and-tangent rule without divisors or the definition of the divisor class group.

Thus, within the elliptic curve setting, we can simply talk about the group elements being points, rather than divisors. We note that however for other curves of higher genus or general abelian varieties this does not happen; group elements are no longer points, but rather divisor classes in $\text{Pic}^0(E)$ with multiple elements in their support.

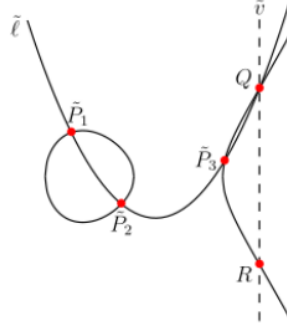


Figure 2.9: Illustration of divisor reduction using the Riemann-Roch Theorem based on *Example 2.5.7*

Despite the abstraction presented above, the language of divisors is essential in the description of elliptic curve pairings, where the objective is to compute very large degree functions on E with prescribed divisors, and then evaluate these functions at other divisors. Such function evaluation can be described as in *Equation 2.55*.

$$f(D) = \prod_{P \in E} f(P)^{n_P} / \quad (2.55)$$

where $f \in \mathbb{F}_q(E)$ and divisor $D = \sum_{P \in E} n_P(P)$ has a natural definition, provided by divisors (f) and D 's disjoint supports. The stipulation of disjoint supports is clearly necessary for $f(D)$ to be non-trivial since $P \in \text{supp}((f))$ implies P is a zero or pole of f on E , meaning $f(P)^{n_P}$ is either zero or infinity respectively.

Example 2.5.8. Consider $E/\mathbb{F}_{163} : y^2 = x^3 - x - 2$, with $P = (43, 154)$, $Q = (46, 38)$, $R = (12, 35)$ and $S = (5, 66)$ all on E . Let $\ell_{P,Q}$, $\ell_{P,P}$ and $\ell_{Q,Q}$ be the lines joining P and Q , tangent to P , and tangent to Q on E respectively, computed as $\ell_{P,Q} : y + 93x + 85$, $\ell_{P,P} : y + 127x + 90$, $\ell_{Q,Q} : y + 13x + 16$. Let $D_1 = 2(R) + (S)$, $D_2 = 3(R) - 3(S)$ and $D_3 = (R) + (S) - 2(O)$. We can compute $\ell_{P,Q}(D_1) = (y_R + 93x_R + 85)^2(y_S + 93x_S + 85) = 122$, or $\ell_{P,P}(D_2) = (y_R + 127x_R + 90)^3 / (y_S + 127x_S + 90)^3 = 53$, but we can not evaluate any of these functions at D_3 , since $O \in \text{supp}(D_3)$, and O is also in the supports of $(\ell_{P,Q})$, $(\ell_{P,P})$, $(\ell_{Q,Q})$. Let $\ell'_{P,P} = 17\ell_{P,P}$ so that $\ell'_{P,P} = 17y + 40x + 63$, and that $\ell'_{P,P}(D_2) = (17y_R + 40x_R + 63)^3 / (17y_S + 40x_S + 63)^3 = 53 = \ell_{P,P}(D_2)$. This is true in general, i.e. that if $g = cf$ for some constant $c \in \mathbb{F}_q$, then $f(D) = g(D)$. If D has degree zero; the constant c will cancel out because $\deg(D) = 0$ implies the numerator and denominator of $f(D)$ (identically $g(D)$) have the same total degree.

We conclude the current subsection with *Example 2.5.8* by acknowledging in general that if $g = cf$ for some constant $c \in \mathbb{F}_q$, then $f(D) = g(D)$. If D has the constant c will cancel out because $\deg(D) = 0$ implies the numerator and denominator of $f(D)$ have the same total degree.

In the following *Section 2.6* we are discussing cryptographic pairings as bilinear maps from two elliptic curve groups to a third (finite field) group, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, based on [5, 17, 28]. As we will see, to be able to define a useful pairing on $\mathbb{G}_1$ and $\mathbb{G}_2$ we must be able to define more than one subgroup in the r -torsion of E where the most useful case from the cryptographic perspective will be when r is a large prime.

2.6 Elliptic curves as pairing groups

A bilinear map on an abelian group (refer *Definition 2.1.2*) M can be represented as in *Equation 2.56*, by taking values in some other abelian group R .

$$\langle \cdot, \cdot \rangle : M \times M \rightarrow R. \quad (2.56)$$

Suppose that the binary group operations in M and R are denoted by $+$, $\cdot$, then applying *Equation 2.56* bilinearity property of map for $x, y, z \in M$ means we have the map $\langle \cdot, \cdot \rangle$ linear in both inputs as follows:

$$\begin{aligned} \langle x + y, z \rangle &= \langle x, z \rangle \cdot \langle y, z \rangle \\ \langle x, z + y \rangle &= \langle x, y \rangle \cdot \langle x, z \rangle. \end{aligned}$$

Our primary objective in this section is to define the two suitable groups $\mathbb{G}_1$ and $\mathbb{G}_2$ for pairing. However, for our purposes we also find it advantageous to slightly relax the condition that the two arguments in the map come from the same group, and allow them to come from cyclic groups of the same order (which are therefore isomorphic), the notation commonly used for the bilinear map is described as *Equation 2.57*:

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T \quad (2.57)$$

Let $\mathbb{F}_{q^k}$ be some finite extension of $\mathbb{F}_q$ with $k \leq 1$. The groups $\mathbb{G}_1, \mathbb{G}_2$ both (with the same order) defined in $E(\mathbb{F}_{q^k})$ and the target group $\mathbb{G}_T$ defined over the multiplicative group of $\mathbb{F}_{q^k}^*$. In the following we write $\mathbb{G}_1, \mathbb{G}_2$ additively, while $\mathbb{G}_T$ multiplicatively, as can be seen in *Equation 2.58*:

$$\begin{aligned} e(P + P', Q) &= e(P, Q) \cdot e(P', Q), \\ e(P, Q + Q') &= e(P, Q) \cdot e(P, Q') \end{aligned}$$

$$e([a]P, [b]Q) = e(P, [b]Q)^a = e([a]P, Q)^b = e(P, Q)^{ab} = e([b]P, [a]Q) \quad (2.58)$$

Example 2.6.1 (Pairing computation example). Let $q = 7691$ and let $E/\mathbb{F}_q : y^2 = x^3 + 1$. Suppose $\mathbb{F}_{q^2}$ is constructed $\mathbb{F}_{q^2} = \mathbb{F}_q(u)$ where $u^2 + 1 = 0$. Let $P = (2693, 4312) \in E(\mathbb{F}_q)$ and $Q = (633u + 6145, 7372u + 109) \in E(\mathbb{F}_{q^2})$. $\#E(\mathbb{F}_q) = 2^2 \cdot 3 \cdot 641$ and $\#E(\mathbb{F}_{q^2}) = 2^4 \cdot 3^2 \cdot 641^2 = \#E(\mathbb{F}_q)^2$.

P and Q were especially chosen (we will see why later) to be in different subgroups of the same prime order $r = |\langle P \rangle| = |\langle Q \rangle| = 641$. The Weil pairing $e(\cdot, \cdot)$ of P and Q is $e(P, Q) = 6744u + 5677 \in \mathbb{F}_{q^2}^*$.

In fact, $r \mid \#E(\mathbb{F}_{q^2})$, and $e(P, Q)$ actually lies in a subgroup of $\mathbb{F}_{q^2}$, namely the r -th roots of unity $\mu_r \subset \mathbb{F}_{q^2}$, meaning that $e(P, Q)^r = 1$.

We are now in a position to illustrate some examples of *bilinearity*. Thus, take any $a \in \mathbb{Z}_r$ and $b \in \mathbb{Z}_r$, say $a = 403$ and $b = 135$, and see that $[a]P = (4903, 2231)$ and $[b]Q =$

$(5806u + 1403, 6091u + 2370)$. We can compute $e([a]P, Q) = 3821u + 7025$ and verify that $e([a]P, Q) = 3821u + 7025 = (6744u + 5677)^{403} = e(P, Q)^a$. On the other hand if we compute $e(P, [b]Q) = 248u + 5$, then we see that $e(P, [b]Q) = 248u + 5 = (6744u + 5677)^{135} = e(P, Q)^b$.

Note that since $e(P, Q) \neq 1 \in \mu_r$, $e([a]P, [b]Q)$ will only be trivial if $r \mid ab$, which implies $r \mid a$ or $r \mid b$, meaning either (or both) of $[a]P$ or $[b]Q$ must be $\mathcal{O}$. Thus, $e(P, Q) \neq 1$ guarantees non-trivial pairings for non-trivial arguments; this is a cryptographically necessary property that is called *non-degeneracy*.

Following from *Example 2.6.1* above, if a pairing e is bilinear, non-degenerate and efficiently computable, e is called an *admissible pairing*. Currently, the only known instantiations of pairings suitable for cryptography are the *Weil* and *Tate* pairings (refer to 3.2.2 and *Section 3.2.3*) on divisor class groups of algebraic curves (recall from *Subsection 2.5.1*), and in the simplest and most efficient cases, on elliptic curves.

For cryptographic applications we need more properties than the three which constitute an admissible pairing. The *bilinearity* property represented in *Equation 2.58* gives pairing-based primitives increased functionality over traditional primitives and it is considered useless unless discrete logarithm related problems (refer to *Section 2.3.2*) within all three groups remain intractable.

Secondly, *Example 2.6.1* gives an admissible pairing, but because the small sizes of $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ clearly offer no resistance in regards to their respective discrete logarithm problems, such pairing instance would never be used in practice. Let us consider *Example 2.6.2* as an illustration.

Example 2.6.2 (Bilinearity property). Let $r > 1$ be an integer. Suppose $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ has $\mathbb{G}_1 = \mathbb{G}_T = \mathbb{Z}_r^*$ and $\mathbb{G}_2 = \mathbb{Z}_{r-1}^+$, and is defined by $e : (g, a) \mapsto g^a$.

For $g, g' \in \mathbb{G}_1$ we have $e(g \cdot g', a) = e(g, a) \cdot e(g', a)$, and for $a, a' \in \mathbb{G}_2$ we have $e(g, a + a') = e(g, a) \cdot e(g, a')$. Although e is bilinear, the discrete logarithm problem in $\mathbb{G}_2$ is easy, so the power of the bilinear map becomes somewhat redundant. Previously in *Subsection 2.3.2* we defined the discrete logarithm problem in terms of elliptic curves. Now, if we set r to be a large prime, then the standard discrete logarithm problem can be formulated for pairing groups: $g \in \mathbb{G}_1$, $h \in \mathbb{G}_T$, find $a \in \mathbb{G}_2$ such that $e(g, a) = h$.

2.6.1 The r -torsion subgroup, embedding degree, Frobenius endomorphism

We can now turn our focus towards defining the groups $\mathbb{G}_1$ and $\mathbb{G}_2$ more precisely. Having not yet seen how pairings are computed, we will need to make some statements regarding what we need out of $\mathbb{G}_1$ and $\mathbb{G}_2$ that will really only tie together when the definitions of the Weil and Tate pairings come in the following *Section 3.1*.

The main such statement is that computing the pairing $e(P, Q)$, in either the Weil or Tate sense, requires that P and Q come from disjoint cyclic subgroups (refer *Definition 2.1.9*) of the same prime order r .

At this point we can only hint towards Weil reciprocity (refer to *Appendix ??*) and claiming that if P and Q are in the same cyclic subgroup, then the pairing computation essentially fails because supports of the associated divisors are forced to undesirably coincide.

As outlined in *Example 2.6.1* the way we obtain two distinct order- r subgroups in general is to find the smallest extension $\mathbb{F}_{q^k}$ of $\mathbb{F}_q$ such that $E(\mathbb{F}_{q^k})$ captures more points of order r . The integer $k \geq 1$ that achieves this is called the *embedding degree*, and it plays a crucial role in pairing computation due to its properties (refer to *Subsection 4.1*). Let us introduce the entire group of points of order r on $E(\mathbb{F}_q)$, called the *r -torsion*, which is denoted $E[r]$ and defined in *Equation 2.59*. (We note, that in *Definition 2.1.10* a generalized formula were given, from now on we would refer such subgroups as r -torsion if we are talking about pairings, specifically.)

$$E[r] = \{P \in E : [r]P = \mathcal{O}\} \quad (2.59)$$

The result in *Equation 2.59* is quite remarkable; it tells us not only the cardinality of $E[r]$, but its structure too. If K is any field with characteristic zero or prime to r , we have

$$E[r] \cong \mathbb{Z}_r \times \mathbb{Z}_r. \quad (2.60)$$

This means that in general $\#E[r] = r^2$. Furthermore, since the point at infinity overlaps into all order- r subgroups, *Equation 2.60* implies that (for prime r) the r -torsion consists of $r+1$ cyclic subgroups of order r . The following equivalent conditions for the embedding degree k also tell us precisely where $E[r]$ lies in its entirety. We note that the embedding degree is actually a function $k(q, r)$ of q and r , but we simply write k since the context is usually clear:

- k is the smallest positive integer such that $r \mid (q^k - 1)$;
- k is the smallest positive integer such that $\mathbb{F}_{q^k}$ contains all of the r -th roots of unity in $\overline{\mathbb{F}}_q$ (i.e. $\mu_r \subset \mathbb{F}_{q^k}$);
- k is the smallest positive integer such that $E[r] \subset E(\mathbb{F}_{q^k})$.

If $r \nmid \#E(\mathbb{F}_q)$ (i.e. $r \nmid \#E(\mathbb{F}_q)$ but $r^2 \nmid \#E(\mathbb{F}_q)$), then the r -torsion subgroup in $E(\mathbb{F}_q)$ is unique. In this case, $k > 1$ and *Equation 2.60* implies that $\mathbb{F}_{q^k}$ is the smallest field extension of $\mathbb{F}_q$ which produces any more r -torsion points belonging to $E(\mathbb{F}_{q^k}) \setminus E(\mathbb{F}_q)$.

In other words, once the extension field is big enough to find one more point of order r (that is not defined over the base field), then we actually find all of the points in $E[r] \cong \mathbb{Z}_r \times \mathbb{Z}_r$ as illustrated in *Example 2.6.3*

Example 2.6.3 (Find r -torsion subgroups). Let $q = 11$, and consider $E/\mathbb{F}_q : y^2 = x^3 + 4$. $E(\mathbb{F}_q)$ has 12 points, so take $r = 3$ and note (from *Equation (2.60)*) that there are 9 points in the 3-torsion. Only 3 of them are found in $E(\mathbb{F}_q)$, namely $(0, 2)$, $(0, 9)$ and $\mathcal{O}$, which agrees with the fact that the embedding degree $k \neq 1$, since $(q^1 - 1) \not\equiv 0 \pmod{r}$. However, $(q^2 - 1) \equiv 0 \pmod{r}$, which means that the embedding degree is $k = 2$, so we form $\mathbb{F}_{q^2} = \mathbb{F}_q(u)$, with $u^2 + 1 = 0$. Thus, we are guaranteed to find the whole 3-torsion in $E(\mathbb{F}_{q^2})$, and it is structured as 4 cyclic subgroups of order 3; $\mathcal{O}$ overlaps into all of them. We point out that although $\mathcal{O}$ is in the 3-torsion, it does not have order 3, but rather order 1 — points of order $d \mid r$ are automatically included in the r -torsion.

Now pick points $P, Q \in E[3] \setminus \{\mathcal{O}\}$ that are not in the same subgroup, neither of which are $\mathcal{O}$. The translation of *Equation (2.60)* is that any other point in $E[3]$ can be

obtained as $[i]P + [j]Q$, for $i, j \in \{0, 1, 2\}$. Fixing $P \neq \mathcal{O}$ and letting j run through $0, 1, 2$ lands $P + [j]Q$ in the other three subgroups of $E[3]$ (that are not $\langle Q \rangle$ — this corresponds to $P = \mathcal{O}$).

Since we are working over finite extension fields of $\mathbb{F}_q$, we can also rely on *Galois theory* [5], namely, the *trace map* of the point $P = (x, y) \in E(\mathbb{F}_{q^k})$ can be defined as follows (Equation 2.61):

$$\text{Tr}(P) = \sum_{\sigma \in \text{Gal}(\mathbb{F}_{q^k}/\mathbb{F}_q)} \sigma(P) = \sum_{i=0}^{k-1} \pi^i(P) = \sum_{i=0}^{k-1} (x^{q^i}, y^{q^i}) \quad (2.61)$$

where π denotes the q -power Frobenius endomorphism, recall Definition 2.1.20 from Subsection 2.1.3. Galois theory tells us that $\text{Tr} : E(\mathbb{F}_{q^k}) \rightarrow E(\mathbb{F}_q)$, so when $r \parallel \#E(\mathbb{F}_q)$ (which will always be the case from now on), this map — which is actually a group homomorphism — sends all torsion points into one subgroup of the r -torsion.

Example 2.6.4. We take $q = 11$ again, but this time with $E/\mathbb{F}_q : y^2 = x^3 + 7x + 2$. $E(\mathbb{F}_q)$ has 7 points, so take $r = 7$. We already have $E(\mathbb{F}_q)[r]$, but to collect $E[r]$ in its entirety we need to extend $\mathbb{F}_q$ to $\mathbb{F}_{q^k}$. This time, the smallest integer k such that $(q^k - 1) \bmod 7 \equiv 0$ is $k = 3$, so we form $\mathbb{F}_{q^3} = \mathbb{F}_q(u)$ with $u^3 + u + 4 = 0$, and we are guaranteed that $E[7] \subset E(\mathbb{F}_{q^3})$. The entire 7-torsion has cardinality 49 and splits into 8 cyclic subgroups.

To fit the points in, we use the power representation of elements in $\mathbb{F}_{q^3} = \mathbb{F}_q(u)$. In this case, for $P \in E(\mathbb{F}_{q^3})$, the trace map on E is $\text{Tr}(P) = (x, y) + (x^q, y^q) + (x^{q^2}, y^{q^2})$. For the unique torsion subgroup $E(\mathbb{F}_q)[r]$, the Frobenius endomorphism is trivial ($\pi(P) = P$) so the trace map clearly acts as multiplication by k , i.e. $\text{Tr}(P) = [k]P$.

However, Tr will send every other element in the torsion into $E(\mathbb{F}_q)[r]$. For example, for $Q = (u^{481}, u^{1049})$ (in the subgroup pointing upwards), we have $\text{Tr}(Q) = (8, 8)$; for $R = (u^{423}, u^{840})$ (the lower right subgroup), we have $\text{Tr}(R) = (10, 7)$; for $S = (u^{1011}, u^{1244})$, we have $\text{Tr}(S) = (8, 3)$.

There is one other peculiar subgroup in $E[7]$ however, for which the trace map sends each element to $\mathcal{O}$. In our case this subgroup is the group containing $T = (u^{1315}, u^{1150})$, i.e. $\text{Tr}(T) = \mathcal{O}$, so $\text{Tr} : \langle T \rangle \rightarrow \{\mathcal{O}\}$. One final point to note is that the embedding degree $k = 3$ also implies that the (six) non-trivial 7-th roots of unity are all found in $\mathbb{F}_{q^3}$ (but not before), i.e. $\mu_7 \setminus \{1\} \in \mathbb{F}_{q^3} \setminus \mathbb{F}_{q^2}$.

We now give a general depiction of the r -torsion $E[r]$. First, we have to discuss a few assumptions that apply most commonly to the scenarios we will be encountering. We assume that $r \parallel \#E(\mathbb{F}_q)$ is prime and the embedding degree k (with respect to r) satisfies $k > 1$. Thus, there is a unique subgroup of order r in $E[r]$ which is defined over $\mathbb{F}_q$, called the *base-field subgroup*, and it will be denoted by G_1 .

Since the Frobenius endomorphism π acts on G_1 , but nowhere else in $E[r]$, it can be characterised as

$$G_1 = E[r] \cap \ker(\pi - [1]),$$

meaning, that G_1 is the $[1]$ -eigenspace of π restricted to $E[r]$. There is another subgroup of $E[r]$ that can be expressed using an eigenspace of π . Namely, we can easily deduce that the other eigenvalue of π is q , and we define another subgroup G_2 of $E[r]$ as

$$G_2 = E[r] \cap \ker(\pi - [q]).$$

It turns out that this subgroup is precisely the peculiar subgroup can be seen in *Example 2.6.4*. We call $\mathbb{G}_2$ the *trace zero subgroup*, since all $P \in \mathbb{G}_2$ satisfy $\text{Tr}(P) = \mathcal{O}$, based on the results of *Lemma IX.16* from [30].

As we mentioned previously in the introduction of *Section 2.6*, our primary goal in this section is to find suitable groups to define our pairing. As $\mathbb{G}_1$ and $\mathbb{G}_2$ can be defined as any of the $r + 1$ groups in $E[r]$, they are promising candidates. However there are many reasons we would like to specifically set $\mathbb{G}_1 = G_1$ and $\mathbb{G}_2 = G_2$.

One specific reason for that is the existence of maps to and from the different torsion subgroups affects certain functionalities that cryptographers desire in a pairing-based protocol. More specifically, from any subgroup in $E[r]$ that is not G_1 or G_2 , we can always map into either G_1 or G_2 via the trace and anti-trace maps respectively.

If E is ordinary (just like in our case as can be seen later in *Subsection ??*), we do not have computable maps out of G_1 or G_2 ; otherwise, if E is supersingular then the distortion map ϕ is a homomorphic map out of these two subgroups.

2.6.2 Pairing types

The distinguishability among pairing types arise from observing the practical implications of placing $\mathbb{G}_1$ and $\mathbb{G}_2$ in different subgroups of $E[r]$. In fact, as it turned out [5] it is always best to set $\mathbb{G}_1 = G_1$, so the four types of pairing will be tied to the definition of $\mathbb{G}_2$. The main factors affecting primarily the classification are the ability to hash and/or randomly sample elements of $\mathbb{G}_2$ and the existence of $\psi : \mathbb{G}_2 \mapsto \mathbb{G}_1$ isomorphism. The latter is often required to make security proofs work and (as always) issues concerning storage and efficiency (see [31] for more details).

Historically, Galbraith et al. [31] were the first to identify that all of the potentially desirable properties in a protocol cannot be achieved simultaneously, thus they classified pairings into three certain types, but a fourth type was by Shacham [32].

Since we are using ordinary curve families later in this thesis, we are mentioning *Type 1 pairings*, but not considering the introduction of it. The remaining three types are defined over ordinary elliptic curves, thus are no restrictions imposed on the choice of elliptic curve that lead to a loss of efficiency.

In all situations we set $\mathbb{G}_1 = G_1$ and points of $P_1 = P_1$ (where hashing is relatively easy), so we only need to discuss the choices for $\mathbb{G}_2$ and P_2 . Since the thesis is about *BLS curves* (refer to *Chapter 4*) we will discuss the relevant pairing type for that particular curve family, on the other hand we note, that nowadays all of the state-of-the-art implementations of pairings take place on ordinary curves that assume the *Type 3* scenario, where the only potential sacrifice is the map $\psi : \mathbb{G}_2 \mapsto \mathbb{G}_1$.

In *Type 3* pairings we take $\mathbb{G}_2 = G_2$, the trace zero subgroup. The drawback in this case is that the only subgroup (besides $\mathbb{G}_1$) that we can hash into is also the only subgroup we cannot find a map out of. An isomorphism $\psi : \mathbb{G}_2 \rightarrow \mathbb{G}_1$ trivially exists, we just do not have an efficient way to compute it. Thus, security proofs that rely on the existence of such a ψ are no longer applicable, unless the underlying problem(s) remains hard when the adversary is allowed oracle access to ψ [33].

Before moving our focus to the algorithm for computing pairings, we have to discuss a way to represent elements in $\mathbb{G}_2$, which is happens to be also the most efficient way to hash into. In the following *Subsection 2.6.3* the discussion brings up the notion of *twists* of elliptic curves, which was first applied to pairings by Barreto et al. in [34].

2.6.3 Twist of elliptic curves

Given elliptic curve $E'/\mathbb{F}_q$ we observe that the map Ψ^{-1} defined by Equation 2.62

$$\Psi^{-1} : (x, y) \mapsto (-x, iy) \quad (2.62)$$

takes points from E to E' , thus $\Psi^{-1} : E \rightarrow E'$. Restricting Ψ^{-1} to $\mathbb{G}_2$ therefore gives a map that takes elements defined over $\mathbb{F}_{q^2}$ to elements defined over $\mathbb{F}_q$. The convention in accordance with [5] is to write Ψ for the reverse map $\Psi : E' \rightarrow E$, which in this case as follows (Equation 2.63):

$$\Psi : (x', y') \mapsto (-x', y'/i) = (-x', -y'i) \quad (2.63)$$

We call E' a *twist* of elliptic curve E . Every twist has a degree d , which tells us the extension field of $\mathbb{F}_q$ over which E and E' become isomorphic. For our purposes, d is also the degree of the field of definition of E' as a subfield of $\mathbb{F}_{q^k}$, i.e. a degree d twist E' of E will be defined over $\mathbb{F}_{q^{k/d}}$. In the general case, the twist will pull computations back into the subfield $\mathbb{F}_{q^{k/d}}$ of $\mathbb{F}_{q^k}$. For example, if the embedding degree is $k = 12$, then a quadratic twist ($d = 2$) would allow computations in $\mathbb{G}_2$ to be performed in $\mathbb{F}_{q^6}$ rather than $\mathbb{F}_{q^{12}}$, whilst a sextic twist ($d = 6$) would allow us to instead work in $\mathbb{F}_{q^2}$. Thus, we prefer the degree d of the twist to be as high as possible. The following Example 2.6.5 shows how to work with twists introduced in Equation 2.63.

Example 2.6.5 (Twist of elliptic curve). Let $q = 103$ and consider $E/\mathbb{F}_q : y^2 = x^3 + 72$, which has $\#E(\mathbb{F}_q) = 84$, so let $r = 7$. The embedding degree (with respect to r) is $k = 6$, so form $\mathbb{F}_{q^6} = \mathbb{F}_q(u)$ with $u^6 + 2 = 0$. The trace zero subgroup $\mathbb{G}_2 = E[r] \cap \ker(\pi - [q])$ is defined over $\mathbb{F}_{q^6}$, and is generated by $(35u^4, 42u^3)$ (see Figure 4.12). We define the degree $d = 6$ sextic twist E' of E as $E' : y^2 = x^3 + 72u^6$, where the back-and-forth isomorphisms are defined as

$$\Psi : E' \rightarrow E, \quad (x', y') \mapsto (x'/u^2, y'/u^3),$$

$$\Psi^{-1} : E \rightarrow E', \quad (x, y) \mapsto (u^2x, u^3y).$$

We observe that Ψ^{-1} maps elements in $\mathbb{G}_2 \in E(\mathbb{F}_{q^k})[r] = E(\mathbb{F}_{q^6})[r]$ to elements in $E'(\mathbb{F}_{q^{k/d}})[r] = E'(\mathbb{F}_q)[r]$. Thus, when performing group operations in $\mathbb{G}_2 = \mathbb{G}_2$, we gain the advantage of working over $\mathbb{F}_q$ instead of $\mathbb{F}_{q^6}$, a huge improvement in computational complexity.

Recall in our case that the elliptic curve is given by $E : y^2 = x^3 + ax + b$, hence the twist of it is given by

$$E' : y^2 = x^3 + a\omega^4x + b\omega^6,$$

with $\Psi : E' \rightarrow E : (x', y') \mapsto (x'/\omega^2, y'/\omega^3)$, where $\omega \in \mathbb{F}_{q^k}$. Therefore we can only achieve specific degrees of d through combinations of zero and non-zero values for a and b , yielding us with four possibilities of twists (refer from Definition 2.6.1 up to Definition 2.6.4) based on the degree of d .

Definition 2.6.1. Quadratic twists (degree $d = 2$) are available on any elliptic curve, so if $E/\mathbb{F}_q : y^2 = x^3 + ax + b$, then a quadratic twist is given by $E'/\mathbb{F}_{q^{k/2}} : y^2 = x^3 + a\omega^4x + b\omega^6$, with $\omega \in \mathbb{F}_{q^k}$ but $\omega^2 \in \mathbb{F}_{q^{k/2}}$. Since $\omega^3 \in \mathbb{F}_{q^k}$, the isomorphism $\Psi : E' \rightarrow E$ defined

by $\Psi : (x', y') \mapsto (x'/\omega^2, y'/\omega^3)$ will take elements in $E'(\mathbb{F}_{q^{k/2}})$ to elements in $E(\mathbb{F}_{q^k})$, whilst Ψ^{-1} will do the reverse.

Definition 2.6.2. Cubic twists (degree $d = 3$) twists can only occur when $a = 0$, so if $E/\mathbb{F}_q : y^2 = x^3 + b$, then $E'/\mathbb{F}_{q^{k/3}} : y^2 = x^3 + b\omega^6$, with $\omega^3, \omega^6 \in \mathbb{F}_{q^{k/3}}$, but $\omega^2 \in \mathbb{F}_{q^k} \setminus \mathbb{F}_{q^{k/3}}$. Thus, the isomorphism $\Psi : E' \rightarrow E$ (defined as usual) will take elements in $E'(\mathbb{F}_{q^{k/3}})$ to elements in $E(\mathbb{F}_{q^k})$, whilst Ψ^{-1} does the opposite.

Definition 2.6.3. Quartic twists (degree $d = 4$) twists are available when $b = 0$, so if $E/\mathbb{F}_q : y^2 = x^3 + ax$, then $E'/\mathbb{F}_{q^{k/4}} : y^2 = x^3 + a\omega^4x$, with $\omega^4 \in \mathbb{F}_{q^{k/4}}$, $\omega^2 \in \mathbb{F}_{q^{k/2}}$ and $\omega^3 \in \mathbb{F}_{q^k} \setminus \mathbb{F}_{q^{k/2}}$. Thus, Ψ will move elements in $E'(\mathbb{F}_{q^{k/4}})$ up to elements in $E(\mathbb{F}_{q^k})$, whilst Ψ^{-1} will move elements from $E(\mathbb{F}_{q^k})$ down to $E'(\mathbb{F}_{q^{k/4}})$.

Definition 2.6.4. Sextic twists (degree $d = 6$) are only available when $a = 0$, so if $E/\mathbb{F}_q : y^2 = x^3 + b$, then $E'/\mathbb{F}_{q^{k/6}} : y^2 = x^3 + b\omega^6$, with $\omega^6 \in \mathbb{F}_{q^{k/6}}$, $\omega^3 \in \mathbb{F}_{q^{k/3}}$ and $\omega^2 \in \mathbb{F}_{q^{k/2}}$. Thus, Ψ pushes elements in $E'(\mathbb{F}_{q^{k/6}})$ up to $E(\mathbb{F}_{q^k})$, whilst Ψ^{-1} pulls elements from $E(\mathbb{F}_{q^k})$ all the way down to $E'(\mathbb{F}_{q^{k/6}})$.

As a remark a specific twist can only be applied if the curve is of the corresponding form above and the embedding degree k has d as a factor. Thus, attractive embedding degrees are those which have any of $d \in \{2, 3, 4, 6\}$ as factors, but preferably $d = 4$ or $d = 6$ (which is also the maximum for genus 1 curves) for increased performance.

Chapter 3

Computational Aspects of Pairings

3.1 Miller's algorithm for the Weil and Tate pairings

In the current chapter we review the theoretical foundations of Weil and Tate pairings with regards to Miller's algorithm on how to compute them. The theoretical introductions (refer to *Subsection 3.1.1* and *Subsection 3.1.2*) use a more generalized notations based on [27]. The practical examples presented later in the chapter (refer 3.2.2 and *Subsection 3.2.3*) are motivated by [5], therefore we will restrict the analogues notations to be aligned with [5]. We note that the definitions in our context are apply to elliptic curves over finite fields, but the more general definitions are analogous, see [35, 30] for more detail.

3.1.1 Theoretic overview of Weil Pairing

Let E be an elliptic curve containing n points over a field $\mathbb{F}_q$. Let G be a cyclic subgroup of $E(\mathbb{F}_q)$ of order r with r, q coprime. Let k be the smallest positive integer such that $E(\mathbb{F}_{q^k})$ contains all of $E[r]$.

We define the Weil pairing $f : E[r] \times E[r] \rightarrow \mathbb{F}_{q^k}^*$ as follows. For a pair of points $P, Q \in E[r]$, choose any $R, S \in E(\mathbb{F}_{q^k})$ such that $S \neq R, P + R, P + R - Q, R - Q$. Let f_P be a rational function with divisor $(f_P) = (P + R)^r / (R)^r$, (In other words we have r zeroes at $P + R$ and r poles at R . In standard notation this is $r\langle P + R \rangle - r\langle R \rangle$.) and similarly let f_Q be a rational function with divisor $(f_Q) = (Q + S)^r / (S)^r$, then

$$f(P, Q) = \frac{f_P(Q + S) / f_P(S)}{f_Q(P + R) / f_Q(R)} \quad (3.1)$$

It can be shown that the value in *Equation 3.1* is independent of the choices for R, S . The conditions for choosing R, S ensure we never evaluate f_P nor f_Q at a zero or pole. Also both f_P, f_Q are only unique up to a constant, but this has no effect on the final answer since quotients of these functions are computed and the constants cancel out. Refer *Definition 6* in *Appendix Subsection B.1.1* for more information regarding the provable properties of Weil pairing.

We note that finding explicit expressions for these functions is infeasible for cryptographically useful pairings, hence we will use Miller's algorithm [36, 37] instead, that evaluates these functions in at the required points. His algorithm resembles a

typical exponentiation routine that performs square and multiply as we continues in the subsequent *Subsection 3.1.1*

Miller's algorithm for the Weil pairing

For the Weil pairing we need to evaluate some rational function f_P with divisor $(P + R)^r / (R)^r$. For any points U, V , let $L_{U,V}(X, Y)$ be an equation for a line through U and V , let $T_U(X, Y)$ be an equation for the tangent through U , let $V_U(X, Y)$ be an equation for a vertical line through U (e.g. $X - x$, where $U = (x, y)$).

Since $T_U = L_{U,U}$ and $V_U = L_{U,-U}$, defining T_U and V_U is unnecessary but they aid comprehension. Note if $U = -U$ then T_U is the same as V_U .

For an integer k , let f_k be a rational function with divisor $(f_k) = \frac{(P+R)^k}{(R)^k(kP)}$. Observe $f_r = f_P$, since $rR = O$. It can be checked that $\left(f_a \cdot f_b \cdot \frac{L_{aP,bP}}{V_{(a+b)P}}\right) = (f_{a+b})$, with computation on the divisors:

$$\frac{(P+R)^a}{(R)^a(aP)} \cdot \frac{(P+R)^b}{(R)^b(bP)} \cdot \frac{(aP)(bP)(-(a+b)P)}{(a+b)P(-(a+b)P)} = \frac{(P+R)^{a+b}}{(R)^{a+b}((a+b)P)}.$$

Which yields us with the formula: $f_k^2 \cdot \frac{T_{kP}}{V_{2kP}} = (f_{2k})$. Thus we may compute $f_P(Q) = f_r(Q)$ using *Algorithm 1* where the binary representation of r is $r_t \dots r_0$.

Input: Binary representation of r is $r_t \dots r_0$

Output: $x = f_P(Q)$

$x_1 \leftarrow V_{P+R}(Q)/L_{P,R}(Q);$

$x \leftarrow x_1;$

$Z \leftarrow P;$

for $i = t - 1$ **to** 0 **do**

$x \leftarrow x^2 \cdot T_Z(Q)/V_{2Z}(Q);$

$Z \leftarrow 2Z;$

if $r_i = 1$ **then**

$x \leftarrow x \cdot x_1 \cdot L_{Z,P}(Q)/V_{Z+P}(Q);$

$Z \leftarrow Z + P;$

end

end

return $x;$

Algorithm 1: Miller's algorithm for Weil pairing [38]. $x = f_P(Q)$

On termination, we have $x = f_P(Q)$ (and $Z = rP = O$). Throughout the algorithm we may multiply the equation of any line L, T, V by an arbitrary constant, thus this algorithm really computes $f_P(Q)$ for one possible choice of f_P .

3.1.2 Theoretic overview of Tate Pairing

We now consider the Tate pairing. Recall we wish to compute $f_P(Q)$ where f_P has divisor $(P)^r$. We can view our current situation as a special case of the above with $R = O$. Thus define the intermediate functions f_k by $(f_k) = (P)^k / (kP)$.

We have $(f_r) = (f_P)$. The following identities can be obtained by simplifying earlier formulas using $R = O$, but it is easy enough to check them directly.

For example, we can show $(f_k) = \left(\prod_{i=1}^{k-1} \frac{L_{iP}}{V_{(i+1)P}} \right)$ via

$$(f_k) = \frac{(P)(P)}{(2P)} \cdot \frac{(P)(2P)}{(3P)} \cdots \frac{(P)((k-1)P)}{(kP)}.$$

We could compute $f_r(Q)$ using this formula, but this is clearly impractical for large r .

Miller's algorithm for the Tate pairing

In [38] is found that $(f_{2k}) = (f_k^2 \cdot T_{kP}/V_{2kP})$ which can be shown with direct calculation:

$$\frac{(P)^{2k}}{(2kP)} = \frac{(P)^{2k}}{(kP)^2} \cdot \frac{(kP)^2(-(2kP))}{(2kP)(-(2kP))}$$

and similarly it can be shown $(f_{k+1}) = (f_k \cdot L_{kP,P}/V_{(k+1)P})$. Thus leading us to the following 2 *Algorithm* that computes $f_P(Q)$ given points P, Q (where P has order r). Let the binary representation of r be $r_t \dots r_0$.

Input: Binary representation of r is $r_t \dots r_0$

Output: $x = f_P(Q)$

$x \leftarrow 1;$

$Z \leftarrow P;$

for $i = t - 1$ **to** 0 **do**

$x \leftarrow x^2 \cdot T_Z(Q)/V_{2Z}(Q);$

$Z \leftarrow 2Z;$

if $r_i = 1$ **then**

$x \leftarrow x \cdot L_{Z,P}(Q)/V_{Z+P}(Q);$

$Z \leftarrow Z + P;$

end

end

return $x;$

Algorithm 2: Miller's algorithm for Tate pairing. $x = f_P(Q)$

When the algorithm finishes we have $x = f_r(Q)$ (and $Z = rP = O$). Note this algorithm can be viewed as the previous *Algorithm 1* with $x_1 = 1$.

Building upon 3.1.1, *Subsection 3.1.2* and the introduced definitions from *Section 2.5* we examine further in *Subsection 3.2* how the Weil and Tate pairing may be used in a practical setting through examples motivated by [5].

3.2 Merging the Theory with Practice

First of all, in this section we will use the notation $w_r(P, Q)$ for the (order r) Weil pairing of P and Q and $t_r(P, Q)$ for their (order r) Tate pairing, as this will help when discussing differences and relationships between them. However after this section, it

will be distinguishable which pairing we mean and what the value of r is (the largest prime factor of $\#E(\mathbb{F}_q)$), thus we will return to the notation most commonly seen in the literature by simply write $e(P, Q)$.

Both pairings make use of a special case of the following fact we recall from *Section 2.5* : a divisor $D = \sum_P n_P(P)$ is principal (i.e. the divisor of a function) if and only if $\sum_P n_P = 0$ and $\sum_P [n_P]P = \mathcal{O}$ on E . For any $m \in \mathbb{Z}$ and $P \in E$, it follows that there exists a function $f_{m,P}$ with divisor

$$(f_{m,P}) = m(P) - ([m]P) - (m-1)(\mathcal{O}), \quad (3.2)$$

where we note that for $m = 0$, one can take $f_{0,P} = 1$ with $(f_{0,P})$ the zero divisor.

Thus, if $P \in E[r]$, then $f_{r,P}$ has divisor

$$(f_{r,P}) = r(P) - r(\mathcal{O}). \quad (3.3)$$

It can be observed that $(f_{m+1,P}) - (f_{m,P}) = (P) + ([m]P) - ([m+1]P) - (\mathcal{O})$, which is exactly the divisor of the function $\ell_{[m]P,P}/v_{[m+1]P}$, where $\ell_{[m]P,P}$ and $v_{[m+1]P}$ are the sloped and vertical lines used in the chord-and-tangent addition of the point $[m]P$ and P (see Figure 5.1). This means we can build $f_{m+1,P}$ from $f_{m,P}$ via

$$f_{m+1,P} = f_{m,P} \cdot \frac{\ell_{[m]P,P}}{v_{[m+1]P}}. \quad (3.4)$$

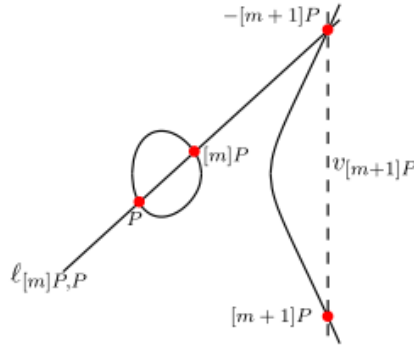


Figure 3.1: Geometrical interpretation of: $\left(\frac{\ell_{[m]P,P}}{v_{[m+1]P}}\right) = (\ell_{[m]P,P}) - (v_{[m+1]P}) = (P) + ([m]P) - ([m+1]P) - (\mathcal{O})$.

3.2.1 Miller's Algorithm for Weil and Tate pairing

In practical cryptographic application r will be huge, thus for any points P and Q belonging to distinct subgroups in $E[r]$, and since $f_{r,P}$ is a function of degree approximately r , computing this function as we did in *Equation 3.1* is impossible. In this subsection we describe Miller's algorithm[37], which makes this computation feasible.

Following *Equation 3.2*, the divisor $(f_{m,P}) = m(P) - ([m]P) - (m-1)(\mathcal{O})$ can be updated to the divisor $(f_{m+1,P}) = (m+1)(P) - ([m+1]P) - m(\mathcal{O})$ by adding the divisor $(\ell_{[m]P,P}/v_{[m+1]P}) = (P) + ([m]P) - ([m+1]P) - (\mathcal{O})$. This corresponds to the multiplication of functions $f_{m+1,P} = f_{m,P} \cdot \ell_{[m]P,P}/v_{[m+1]P}$.

Starting with $f_{2,P} = 2(P) - ([2]P) - (\mathcal{O})$ then, we can repeat this process roughly $r - 1$ times to obtain the desired function $f_{r,P} = r(P) - ([r]P) - (r - 1)(\mathcal{O}) = r(P) - r(\mathcal{O})$. We note that for the last step, when $m = r - 1$ we will have $f_{r-1,P} = (r - 1)(P) - ([r - 1]P) - (r - 2)(\mathcal{O})$. Therefore the required divisor is $(P) + ([r - 1]P) - 2(\mathcal{O})$ which corresponds to the vertical line $v_{[r-1]P} = v_{-P} = v_P$. We note that this is the same vertical line that appears on the denominator of $l_{[r-2]P,P}/v_{[r-1]P}$, yielding us with the pairing evaluation function $f_{r,P}$ as the product (Equation 3.5) :

$$f_{r,P} = l_{[r-2]P,P} \cdot \prod_{i=1}^{r-3} \frac{l_{[i]P,P}}{v_{[i+1]P}} \quad (3.5)$$

The first four sloped lines $l_{[i]P,P}$ and corresponding vertical lines $v_{[i+1]P}$ from the numerator and denominator of the product in Equation 3.5 are depicted in Figure 3.2. As can be followed, the product (Equation 3.5) is built up incrementally by absorbing each of the $v_{[m+1]P}$ and $f_{m,P}$ to increment into $f_{m+1,P}$, eventually arriving at $f_{r,P}$. Alternatively, it can help to see the divisor sum written out in full, to see the contributions of each of the functions $v_{[i+1]P}$ in the product all at once.

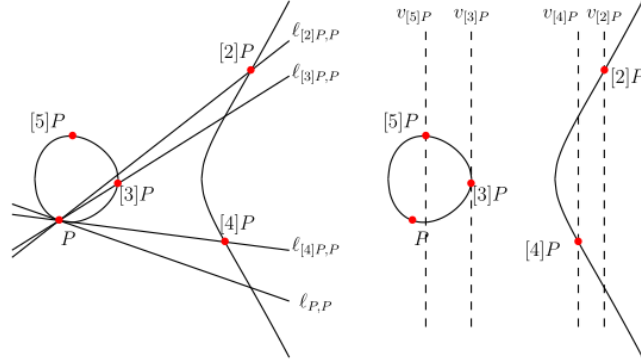


Figure 3.2: One the left, we can view the first four sloped lines, while on there are the first four vertical lines as arising from the product in Equation 3.5.[5]

$$\begin{aligned} l_{P,P}/v_{[2]P} &: (P) + (P) - ([2]P) - (\mathcal{O}) \\ l_{[2]P,P}/v_{[3]P} &: (P) + ([2]P) - ([3]P) - (\mathcal{O}) \\ l_{[3]P,P}/v_{[4]P} &: (P) + ([3]P) - ([4]P) - (\mathcal{O}) \\ &\vdots \\ l_{[r-4]P,P}/v_{[r-3]P} &: (P) + ([r-4]P) - ([r-3]P) - (\mathcal{O}) \\ l_{[r-3]P,P}/v_{[r-2]P} &: (P) + ([r-3]P) - ([r-2]P) - (\mathcal{O}) \\ l_{[r-2]P,P} &: (P) + ([r-2]P) - ([r-1]P) - 3(\mathcal{O}) \end{aligned}$$

When summing all of the above divisors, most of the inner terms cancel out with one another to leave $(r - 1)(P) + (-[r - 1]P) - r(\mathcal{O})$, and since $[r - 1]P = -P$, we get the divisor of the product being $r(P) - r(\mathcal{O})$.

Roughly speaking, $f_{r,P} = g(x, y)/h(x, y)$, where g and h are degree r functions on E . The above method computes $f_{r,P}$ by increasing the degrees of g and h by one each time $f_{m,P}$ is incremented.

The function $f_{m,P}$ has m zeros at P and $(m-1)$ poles at $\mathcal{O}$. Rather than adding one zero and one pole via multiplying $f_{m,P}$ by linear functions, we can double the number of zeros at P and the number of poles at $\mathcal{O}$ if we instead square $f_{m,P}$. Observe that since $(f_{m,P}) = m(P) - ([m]P) - (m-1)(\mathcal{O})$, then

$$(f_{m,P}^2) = 2m(P) - 2([m]P) - 2(m-1)(\mathcal{O}),$$

which is almost the same as $f_{2m,P}$, whose divisor is

$$(f_{2m,P}) = 2m(P) - ([2m]P) - (2m-1)(\mathcal{O});$$

the difference between the two divisors being $(f_{2m,P}) - (f_{m,P}^2) = 2([m]P) - ([2m]P) - (\mathcal{O})$, which corresponds to a function with two zeros at $[m]P$, a pole at $[2m]P$ and another pole at $\mathcal{O}$. The function represents the quotient of the tangent line at $[m]P$ and the vertical line at $[2m]P$ – the lines used to double the point $[m]P$. Thus, we can advance from $f_{m,P}$ to $f_{2m,P}$ via

$$f_{2m,P} = f_{m,P}^2 \cdot \frac{l_{[m]P,[m]P}}{v_{[2m]P}}.$$

We note that the degree of $f_{m,P}$ grows linearly in the size of m , so the function $f_{m,P}$ becomes too large to store explicitly. Thus, the last step in Miller's derivation of the pairing algorithm was to evaluate $f_{m,P}$ at the given divisor (at every stage), i.e. $f_{m,P}(D_Q)$. This means that at any intermediate stage of the algorithm we will not be storing an element of the function field $f_{m,P} \in \mathbb{F}_{q^k}(E)$, but rather its evaluation at D_Q which is the value $f_{m,P}(D_Q) \in \mathbb{F}_{q^k}$. At each stage then, the updates that build the function are evaluated at D_Q before being absorbed into intermediate pairing value that is carried through the routine as summarised in *Algorithm 3*.

Input: $P \in E(\mathbb{F}_{q^k})[r]$, $D_Q \sim (Q) - (\mathcal{O})$ with support disjoint from $(f_{r,P})$ and $r = (r_{n-1} \dots r_1 r_0)_2$ with $r_{n-1} = 1$

Output: $f \leftarrow f_{r,P}(D_Q)$

$R \leftarrow P$, $f \leftarrow 1$;

for $i = n-2$ **to** 0 **do**

 Compute the line functions $\ell_{R,R}$ and $v_{[2]R}$ for doubling R ;

$R \leftarrow [2]R$;

$f \leftarrow f^2 \cdot \frac{\ell_{R,R}}{v_{[2]R}}(D_Q)$;

if $r_i = 1$ **then**

 Compute the line functions $\ell_{R,P}$ and v_{R+P} for adding R and P ;

$R \leftarrow R + P$;

$f \leftarrow f \cdot \frac{\ell_{R,P}}{v_{R+P}}(D_Q)$;

end

end

return f ;

Algorithm 3: Miller's Algorithm

3.2.2 Working example for Weil Pairing

Let $P, Q \in E(\mathbb{F}_{q^k})[r]$ and let D_P and D_Q be degree zero divisors with disjoint supports such that $D_P \sim (P) - (\mathcal{O})$ and $D_Q \sim (Q) - (\mathcal{O})$. There exist functions f and g such that $(f) = rD_P$ and $(g) = rD_Q$. The Weil pairing w_r is a map $w_r : E(\mathbb{F}_{q^k})[r] \times E(\mathbb{F}_{q^k})[r] \rightarrow \mu_r$, defined as

$$w_r(P, Q) = \frac{f(D_Q)}{g(D_P)}.$$

Among other properties, the Weil pairing is bilinear and non-degenerate. An important point to note is that we cannot simply define the Weil pairing as $w_r(P, Q) = f_{r,P}(D_Q)/f_{r,Q}(D_P)$, because $(f_{r,P}) = r(P) - r(\mathcal{O})$ and $(f_{r,Q}) = r(Q) - r(\mathcal{O})$; this corresponds to the divisors $D_P = (P) - (\mathcal{O})$ and $D_Q = (Q) - (\mathcal{O})$, which does not adhere to the requirement that D_P and D_Q have disjoint supports.

Example 3.2.1 (Weil Pairing computational example.). Let $q = 23$, and consider $E/\mathbb{F}_q : y^2 = x^3 - x$, which (is supersingular and therefore) has $\#E(\mathbb{F}_q) = q + 1 = 24$. The point $P = (2, 11)$ is a point of order $r = 3$ and the embedding degree with respect to r is $k = 2$. Take $\mathbb{F}_{q^2} = \mathbb{F}_q(i)$ with $i^2 + 1 = 0$, from which we obtain a point Q of order 3 (that is not in $\langle P \rangle$) as $Q = (21, 12i)$, which is actually in the trace zero subgroup, i.e. $\pi(Q) = [q]Q$. Suppose we wish to compute the Weil pairing $w_r(P, Q)$ of P and Q , which have divisors $(f_{r,P}) = 3(P) - 3(\mathcal{O})$ and $(g_{r,P}) = 3(Q) - 3(\mathcal{O})$ respectively. We need to find divisors D_P and D_Q that have distinct supports but which are respectively equivalent to $(P) - (\mathcal{O})$ and $(Q) - (\mathcal{O})$.

Note that only one of these divisors needs to be updated (so that its support does not contain $\mathcal{O}$), but we will update both in the name of symmetry. Thus, take two more (random) points in $E(\mathbb{F}_{q^2})$ as $R = (17i, 2i + 21)$ and $S = (10i + 18, 13i + 13)$, and set $D_P = (P + R) - (R)$ and $D_Q = (Q + S) - (S)$. We find f as a function with divisor D_P and g as a function with divisor D_Q as $f = f_{r,P}/(\ell_{P,R}/v_{P+R})^3$ and $g = g_{r,Q}/(\ell_{Q,S}/v_{Q+S})^3$ respectively, where $\ell_{P,R}/v_{P+R}$ is the quotient of the chord between P and R and the vertical line through $P + R$. We can now compute the Weil pairing $w_r(P, Q) = f(D_Q)/g(D_P)$

$$= \frac{f(Q + S) \cdot g(R)}{f(S) \cdot g(P + R)} = 15i + 11$$

3.2.3 Working example for Tate Pairing

The formal definition of the Tate pairing requires that only one argument comes from the r -torsion, while the other argument can be any point of $E(\mathbb{F}_{q^k})$. In general it is still advantageous to choose both points from (distinct subgroups in) the r -torsion.

In order to define the Tate pairing correctly though, we need to properly define the groups involved. We assume the standard setting that is of most interest to us: $k > 1$, $r \nmid \#E(\mathbb{F}_q)$ and, since there are r^2 points in the subgroup $E(\mathbb{F}_{q^k})[r]$, we usually have $r^2 \nmid \#E(\mathbb{F}_{q^k})$. Thus, let $h = \#E(\mathbb{F}_{q^k})/r^2$ be the cofactor that sends points in $E(\mathbb{F}_{q^k})$ to points in $E(\mathbb{F}_{q^k})[r]$. Let $rE(\mathbb{F}_{q^k})$ be the coset of points in $E(\mathbb{F}_{q^k})$ defined by Equation 3.6.

$$rE(\mathbb{F}_{q^k}) = \{[r]P : P \in E(\mathbb{F}_{q^k})\} \quad (3.6)$$

The number of elements in $rE(\mathbb{F}_{q^k})$ is denoted as h and it also contains $\mathcal{O}$. Following [39], we can obtain another distinct coset of $E(\mathbb{F}_{q^k})$ by adding a random element R (not in $E[r]$) to each element of rE . In this way we can obtain precisely r^2 distinct, order h cosets. Consequently, the quotient group E/rE is the group whose elements are these cosets. We note that elements belonging to each coset do not have the same order, nor do they form a (sub)group.

In the quotient group E/rE , points belonging to the same coset (group element) can be used to represent the coset, thus any two points in the same coset differ from one another by an element in rE . One can think of E/rE as the set of equivalence classes of points in $E(\mathbb{F}_{q^k})$ under the equivalence relation $P_1 \equiv P_2$ if and only if $P_1 - P_2 \in rE$ in accordance with [30].

In our case $E[r]$ and the quotient group E/rE both have r^2 elements, however, if $r^2 \nmid \#E(\mathbb{F}_{q^k})$, then $E[r] \cap rE = \mathcal{O}$, which means that adding a unique torsion element to all of the elements in rE will generate a unique coset in E/rE . That is, $r^2 \nmid \#E(\mathbb{F}_{q^k})$ implies that $E[r]$ does represent E/rE and this will always be the case for us.

When it comes to defining the Tate pairing this observation is particularly since the second group in the (order r) Tate pairing is E/rE . Therefore $E[r]$ represents E/rE , which allows us to take both groups from the r -torsion. Now, let us consider the case when we are operating over finite fields (*Definition 3.2.1*).

Definition 3.2.1 (The Tate pairing over finite fields.). Let $P \in E(\mathbb{F}_{q^k})[r]$, from which it follows that there is a function f whose divisor is $(f) = r(P) - r(\mathcal{O})$. Let $Q \in E(\mathbb{F}_{q^k})$ be any representative in any equivalence class in $E(\mathbb{F}_{q^k})/rE(\mathbb{F}_{q^k})$, and let D_Q be a degree zero divisor defined over $\mathbb{F}_{q^k}$ that is equivalent to $(Q) - (\mathcal{O})$, but whose support is disjoint to that of (f) . The Tate pairing t_r is a map

$$t_r : E(\mathbb{F}_{q^k})[r] \times E(\mathbb{F}_{q^k})/rE(\mathbb{F}_{q^k}) \rightarrow \mathbb{F}_{q^k}^* / (\mathbb{F}_{q^k}^*)^r,$$

$$t_r(P, Q) = f(D_Q).$$

Example 3.2.2 (Computing Tate pairing over finite field.). We are building upon parameters from *Example 5.2.1* in [5]. Let $P = (3, 2) \in E[r]$ and let $Q = (i+1, 4i+2) \in E(\mathbb{F}_{q^k})$. The function $f : y + 2x + 2 = 0$ on E has divisor $3(P) - 3(\mathcal{O})$, so to compute the Tate pairing we need to find an appropriate $D_Q \sim (Q) - (\mathcal{O})$ but with $P, \mathcal{O} \notin \text{supp}(D_Q)$.

After taking R randomly as $R = (2i, i+2)$, and let $D_Q = (Q+R) - (R)$, where $Q+R = (3i+1, 2)$. The Tate pairing is computed as follows:

$$t_r(P, Q) = f(D_Q) = \frac{f(Q+R)}{f(R)} = \frac{2+2 \cdot (3i+1)+2}{(i+2)+2 \cdot 2i+2} = 4i+4$$

Furthermore to illustrate the bilinearity property, computing $t_r(P, [2]Q)$ with $D_{[2]Q} = ([2]Q+R) - (R)$ where $[2]Q+R = (i+2, i)$ yields us:

$$t_r(P, [2]Q) = f(D_{[2]Q}) = \frac{f([2]Q+R)}{f(R)} = \frac{i+2 \cdot (i+2)+2}{(i+2)+2 \cdot 2i+2} = 2i+4$$

or computing $t_r([2]P, Q)$, where $\tilde{f} = y + 3x + 3$ has divisor $(\tilde{f}) = r([2]P) - r(\mathcal{O})$, gives

$$t_r([2]P, Q) = f(D_Q) = \frac{f(Q+R)}{f(Q)} = \frac{2+3 \cdot (3i+1)+3}{(i+2)+3 \cdot 2+3} = 3i+2$$

We note that the standard Tate pairing has an (undesirable) property that its output lies in an equivalence class, rather than being a unique value. For the Tate pairing to be useful in cryptography is that different parties must compute the exact same value under the bilinearity property, rather than values which are the same under the above notion of equivalence. Thus, to be suitable in practice, the definition of the Tate pairing has to be updated, to make sure the mapping produces unique values as follows in *Definition 3.2.2*.

Definition 3.2.2 (The reduced Tate pairing). Let P, Q, f and D_Q . Over finite fields, the reduced Tate pairing T_r is a map $T_r : E(\mathbb{F}_{q^k})[r] \times E(\mathbb{F}_{q^k})/rE(\mathbb{F}_{q^k}) \rightarrow \mu_r$, which defined as

$$T_r(P, Q) = t_r(P, Q)^{\#\mathbb{F}_{q^k}/r} = f_{r,P}(D_Q)^{(q^k-1)/r},$$

where the final exponentiation allows you to pick D_Q as $(Q+R) - (R)$ with R in a subfield, so it vanishes and the Miller function can be evaluated directly on R .

Exponentiating elements in $\mathbb{F}_{q^k}^*/(\mathbb{F}_{q^k}^*)^r$ to the power of $(q^k-1)/r$ sends the paired value to an exact r -th root of unity in μ_r . From now on we will also take the second argument of the (reduced) Tate pairing from the r -torsion. We will further assume a Type 3 pairing (refer *Subsection 2.6.2*), therefore, in the pairing of P and Q , we will assume $P \in \mathbb{G}_1 = E[r] \cap \ker(\pi - [1])$ and $Q \in \mathbb{G}_2 = E[r] \cap \ker(\pi - [q])$.

By fixing P and letting Q run through $\langle Q \rangle$ (which has order r) will give each value in μ_r , hence for any $\tilde{P}, \tilde{Q}$ pair chosen from anywhere in the torsion, there exists a scalar $0 \leq a \leq r-1$ such that $T_r([a]P, Q) = T_r(P, [a]Q) = T_r(\tilde{P}, \tilde{Q})$.

We note that the t_r pairings on the top level are equivalent in $\mathbb{F}_{q^k}^*/(\mathbb{F}_{q^k}^*)^r$, also T_r ensures that each of the pairings (that should be equivalent) take exactly the same value in $\mu_r \subset \mathbb{F}_{q^k}$. From now on, when we say Tate pairing, we mean the reduced Tate pairing T_r as introduced in *Definition 3.2.2*.

In the following *Chapter 4* review the Weil and Tate pairings over Type 3 (refer to *Subsection 2.6.2*) pairing-friendly BLS curve on the 128-bit security levels based on [40, 41, 34, 13, 42, 43] with some examples motivated by [5, 43, 44]. However, before we dive into the practical application, there are a few topics we have to recap in more details, such as the embedding degree (see *Subsection 2.6.1*), type 3 pairings (refer to *Subsection 2.6.2*) as well as the sextic twist (refer to *Definition 2.6.4*) of an elliptic curve.

Chapter 4

BLS curves for high-security pairings

4.1 Pairing-friendly curve selection and the balancing act

Let E be an elliptic curve defined over a finite field K . Let $P \in E(K)$ be a point of prime order r and $G = \langle P \rangle$, since we intend to use cryptosystems that operate in G . Hence the field arithmetic will be performed in K during the cryptographic operations, while the security of such resulting systems partly depends on the size of r (which is a factor of $\#E(K)$). We focus on the cases when $\#E(K)$ is a small multiple of r , on the other hand relaxing this later condition permits us to use a wider variety of pairings, as we discuss later.

Recall the size of K is roughly the size of $\#E(K)$, thus if both r and $\#E(K)$ are about l -bits in length, we obtain a pairing-based cryptosystem with l -bit security (refer to *Table 2.1*) in G whose running times depend on operations on l -bit numbers.

Recall the Tate pairing (refer to *Subsection 3.1.2*) must be computed in some field extension L of K that contains the r -th roots of unity (so r divides $|L| - 1$). On the other hand, the Weil pairing (refer to *Subsection 3.1.1*) is defined on $E[r]$, the r -torsion points of E , which lie in $E(L')$ for some field extension L' of K . In order for the Tate pairing to be efficiently computable, operations must be efficient in L , and similarly, in the case of L' for the Weil pairing. Thus we have to seek fields (L or L') that are small enough so that field operations are still fast.

It turns out that such a field L' will always contain the n -th roots of unity, thus we always have $L \subseteq L'$. Although the converse is not true: it is possible for a field L to contain the r -th roots of unity but at the same time to have $E[r]$ contained in $E(L')$ and not $E(L)$, consequently for some L' strictly bigger than L , refer to [45] for more details.

We recall the embedding degree definition from *Subsection 2.6.2*, which can be thought of as measuring the size of L compared to the field K (refer to *Table 2.1*). Then, suppose $k > 1$ and $r \mid q^k - 1$, furthermore consider the subgroup T of $E[r]$ consisting of all points of trace zero, that is

$$T = \{Q \in E[r] : \text{tr } Q = Q + \Phi(Q) + \cdots + \Phi^{k-1}(Q) = \mathcal{O}\},$$

where Φ denote the Frobenius map as introduced in *Definition 2.1.20*. We note that the group T may be explicitly constructed by using the map $P \mapsto P - \Phi(P)$ on points

of $E(\mathbb{F}_{q^k})$. Now we have $\Phi(T) = T$ with T not contained in $E(\mathbb{F}_q)$ by assumption.

Recall from *Subsection 2.6.1* and [46] that $\mathbb{G}_2 \subset E(\mathbb{F}_{q^k})[r]$ is the eigenspace of the Frobenius endomorphism π associated with the eigenvalue q , thus it is the complementary subspace of $\mathbb{G}_1$ (which is the eigenspace for the eigenvalue 1). As a result, $\mathbb{G}_2$ is the image of the efficient endomorphism ψ of $E(\mathbb{F}_{q^k})[r]$. Since the eigenvalues of Φ must be 1 and q , consequently T must be the q -eigenspace of Φ and hence

$$\Phi^k(Q) = q^k Q = Q,$$

since $r \mid q^k - 1$. Thus T , like $E(\mathbb{F}_q)$ is fixed under Φ^k , and since these groups are linearly independent they generate all of $E[r]$, implying that all of $E[r]$ is fixed under Φ^k . Hence $E[r] \subset E(\mathbb{F}_{q^k})$. In general, for a cyclic subgroup $G \subset \#E(\mathbb{F}_q)$ the embedding degree k of $E(\mathbb{F}_q)$ may not be the smallest positive integer such that $r \mid q^k - 1$, but in practice, when r is a large prime factor of $\#E(\mathbb{F}_q)$ the probability that k is not the integer we seek is considered negligible.

4.1.1 The balancing act

Pairings are fundamentally different to traditional number-theoretic primitives, in that they require multiple groups that are defined in different settings. Namely, $\mathbb{G}_1$ and $\mathbb{G}_2$ are elliptic curve groups, whilst $\mathbb{G}_T$ is a multiplicative subgroup of a finite field.

All three groups must be secure against the respective instances of the discrete logarithm problem, which means attackers can break the system by solving either the DLP (refer to *Subsection 2.3.1*) in $\mathbb{G}_T$ or the ECDLP (refer to *Definition 2.3.2*) in $\mathbb{G}_1$ or $\mathbb{G}_2$. Therefore to achieve the same security, a finite field group needs to have a much greater cardinality than an elliptic curve group.

Let us consider the following example

Example 4.1.1 (The balancing act. Motivated by [5]). Let $E/\mathbb{F}_{q_1} : y^2 = x^3 + 14$ be the curve with order $\#E(\mathbb{F}_{q_1})$ having large prime factor r , where q_1 and r_1 are given as

$$\begin{aligned} q_1 &= 4\,219\,433\,269\,001\,672\,285\,392\,043\,949\,141\,038\,139\,415\,112 \\ &\quad 441\,532\,591\,511\,251\,381\,287\,775\,317\,505\,016\,692\,408\,034 \\ &\quad 796\,798\,044\,263\,154\,903\,329\,667 \\ &\quad (369 \text{ bits}), \\ r_1 &= 2\,236\,970\,611\,075\,786\,789\,503\,882\,736\,578\,627\,885\,610\,300 \\ &\quad 038\,964\,062\,133\,851\,391\,137\,376\,569\,980\,702\,677\,867 \\ &\quad (271 \text{ bits}). \end{aligned}$$

The embedding degree is $k = 9$ ($q_1^9 - 1 \equiv 0 \pmod{r_1}$) thus, the two elliptic curve groups $\mathbb{G}_1 \in E[r_1]$ and $\mathbb{G}_2 \in E[r_1]$ have an order of 271 bits. Although $\mathbb{G}_T$ is a subgroup of order r (in $\mathbb{F}_{q_1}^*$), the attack complexity is determined by the full size of the field $\mathbb{F}_{q_1^9}$, which is 3248 bits in this case, therefore meeting the requirements for 128-bit security.

In the second case, we choose $E/\mathbb{F}_{q_2} : y^2 = x^3 + 2$ be the curve with prime order $r_2 = \#E(\mathbb{F}_{q_2})$, where q_2 and r_2 are given as

$$\begin{aligned}
q_2 &= 2875788016482373728402120498006552346737 \\
&\quad 721998351309856542751926351376964733335173, \\
r_2 &= 2875788016482373728402120498006552346737 \\
&\quad 668371977047909896314898406560560716472109.
\end{aligned}$$

The embedding degree is $k = 12$, i.e. $q_2^{12} - 1 \equiv 0 \pmod{r_2}$, giving $\mathbb{F}_{q_2^{12}}$ as a 3248-bit field, which is the same size as the $k = 9$ curve above.

As highlighted in *Example 4.1.1* $\mathbb{G}_1$, $\mathbb{G}_2$ and $\mathbb{G}_T$ have orders of the same bit-lengths, but using this curve instead means that arithmetic in $\mathbb{F}_q$ will be substantially faster; a 271-bit field in this case, compared to 369-bit field in the last. An important parameter associated with a curve that is suitable for pairings is the ratio between the field size q and the large prime group order r , which we call the ρ -value and can be computed as in *Equation 4.1*.

The ρ -value indicates how much (ECDLP) security a curve offers for its field size, and since we generally prefer the largest prime divisor r of $\#E$ to be as large as possible, $\rho = 1$ is as good as we can get.

$$\rho = \frac{\log q}{\log r} \quad (4.1)$$

Another metric commonly used in comparison of security levels is the ratio between group sizes: $\rho \cdot k$, for instance $\left(\frac{\log q^k}{\log r} = k \cdot \frac{\log q}{\log r}\right)$. Where $\rho \cdot k$ balances the attack complexities of the DLP and ECDLP refer to [5, 47] for details about attacks against ECC cryptosystems.

Lastly, referring to [48] we say that a curve is pairing-friendly there is a prime $r \geq \sqrt{q}$ dividing $\#E(\mathbb{F}_q)$ (i.e. $\rho \leq 2$), and the embedding degree k with respect to r is less than $\log_2(r)/8$.

In the following *Subsection 4.1.2* we briefly review the complex multiplication method (CM for short) of Atkin and Morain [14], which can be used to find the elliptic curve equations provided the CM discriminant (denoted as D).

4.1.2 Review the construction of ordinary pairing-friendly curves

The methods for elliptic curve constructions can be found in the literature are essentially following the same idea: Set k to a fixed value then compute integers t, r, q such that there is an elliptic curve $E/\mathbb{F}_q$ with trace of Frobenius t , with a subgroup of prime order r , and an embedding degree k . The complex multiplication method (CM method) of Atkin and Morain [14] can then be used to find the equation of E . The other two most general methods, are the *Cocks-Pinch* [49] and *Dupont-Enge-Morain* [50] algorithms, which produce curves with $\rho = 2$. However, we note that this output often not the desired one, especially compared to the ρ -values obtained by the complex multiplication method. In *Equation 4.2* the CM discriminant (denoted as D) can be followed of E for some f integer, as the square-free part of $4q - t^2$:

$$Df^2 = 4q - t^2 \quad (4.2)$$

In 2001, Miyaji, Nakabayashi and Takano [51] gave the first constructions of ordinary pairing-friendly elliptic curves, commonly referred in the literature by MNT criteria. Their method has since been greatly extended and generalized. In certain cases they found that if k is the desired embedding degree, then $r \mid q^k - 1$ implies $r \mid \Phi_k(q)$, since the k -th cyclotomic polynomial $\Phi_k(x)$ is the factor of $x^k - 1$ that does not appear as a factor of any polynomial $(x^i - 1)$ with $i < k$ (refer to IX.15.2 [30]).

For the mentioned special cases they parameterized the curves by writing t , r and q as polynomials $t(x)$, $r(x)$ and $q(x)$ in terms of a parameter. They primarily focused on embedding degrees $k = 3, 4, 6$ and assumed that the group order was to be prime, i.e. $r(x) = q(x) + 1 - t(x)$. Lately they proved that the only possibilities for $t(x)$ and $q(x)$ are:

$$\begin{aligned} k = 3 : t(x) &= -1 \pm 6x \quad \text{and} \quad q(x) = 12x^2 - 1; \\ k = 4 : t(x) &= -x, x + 1 \quad \text{and} \quad q(x) = x^2 + x + 1; \\ k = 6 : t(x) &= 1 \pm 2x \quad \text{and} \quad q(x) = 4x^2 + 1. \end{aligned}$$

We note that, in general parameterised families therefore, we use the degrees of $q(x)$ and $r(x)$ to state $\rho = \frac{\deg(q(x))}{\deg(r(x))}$ (refer to *Definition 4.1*).

After the publication of MNT paper a number of works followed on the generalisation of their results. In particular, we continue with the work by Barreto et al. [52], Scott and Barreto [53], with mentioning Galbraith et al. [54]. All of the mentioned generalizations obtain more parameterised families by simply relaxing the condition regarding the group order, by fixing it as prime and allowing for small cofactors so that $\#E = hr$.

Now, we can recall observation made by Barreto et al. that somewhat simplifies the process is the following: $r \mid \Phi_k(q)$ and $q + 1 - t \equiv 0 \pmod{r}$ combine to give that $\Phi_k(t - 1) \equiv 0 \pmod{r}$ [52, Lemma 1]. Substituting $hr = q + 1 - t$ into the CM equation in *Equation 4.2* yields us with:

$$Df^2 = 4hr - (t - 2)^2$$

In [52], Barreto et al. obtained more parameterised families for various k by considering a special case of the above equation with $t(x) = x + 1$, $D = 3$. Since $r \mid \Phi_k(x)$ finding $f(x)$ and $m(x)$ to fit *Equation 4.3*.

$$3f(x)^2 = 4m(x)\Phi_k(x) - (x - 1)^2 \tag{4.3}$$

In the following *Subsection 4.1.3* we continue with the introduction of BLS curves families based on [13, 43, 55, 56]

4.1.3 Overview of Barreto-Lynn-Scott Curve family

As we reviewed previously pairing is a kind of bilinear map defined over two elliptic curves E and E' . A Barreto-Naehrig Curve (BN) curve is one of the instantiations of pairing-friendly curves proposed in 2005. Around the time BN curves were introduced, Schirokauer warned that the special form of p may reduce the difficulty of the extension field discrete logarithm problem on which the security of these pairings are based.

In response to these developments many researchers have turned instead to the BLS curves, which were actually the first pairing friendly curves of this type to be discovered. Furthermore the BLS curves are in fact a “family of families”, that support members with embedding degrees of 12, 24 and 48 (hereafter referred to as BLS12, BLS24 and BLS48 curves respectively), that are suitable at different security levels.

In contrast to BN curves, $E(F_p)$ does not have a prime order, instead its order is divisible by a large parameterized prime r and denoted by $h \cdot r$ with cofactor h . The pairing will be defined on the r -torsions points. More formally a BLS curve family considers elliptic curves E and E' parameterized by a well chosen integer t . E is defined over a finite field $\mathbb{F}_p$ by an equation of the form $E : y^2 = x^3 + b$, and its twisted curve, $E' : y^2 = x^3 + b'$, where b is an element of multiplicative group of order p . Similarly to BN curves BLS curves can also be categorized as D-type and M-type (refer to [13]) whether $b' = b/m$ or $b' = b \cdot m$ respectively.

For embedding degree $k = 24$, the Barreto-Lynn-Scott (BLS) family is given by the following parametrization (refer to [56]):

$$\begin{aligned} p(x) &= (x-1)^2(x^8 - x^4 + 1)/3 + x, \\ n(x) &= (x-1)^2(x^8 - x^4 + 1)/3, \\ f(x) &= (x-1)(2x-1)/3, \\ r(x) &= x^8 - x^4 + 1, \\ t(x) &= x + 1. \end{aligned}$$

Finding a specific BLS curve is achieved by running through integer values $x_0 \equiv 1 \pmod{3}$ until $p(x_0)$ and $r(x_0)$ are both prime (note that $x_0 \equiv 1 \pmod{3}$ leads to all involved parameters being integers). For each set of parameters, there exists an elliptic curve E over $\mathbb{F}_p$ such that $\#E(\mathbb{F}_p) = n(x_0)$. The correct curve E can be found by trying different values for b (for example using different twists) and checking for the right group order. Since $r(x_0) \mid n(x_0)$, there is a subgroup of $E(\mathbb{F}_p)$ of prime order $r(x_0)$. The CM discriminant is $D = 3$ because $4p(x_0) - t(x_0)^2 = 3f(x_0)^2$. The family has a ρ -value of $\rho = \deg(p)/\deg(r) = 1.25$.

The following *Table 4.1* summarizes the salient parameters of BLS12 and BLS24 families of elliptic curves. All these curves are parameterized by a positive integer z , are defined by an equation of the form $Y^2 = X^3 + b$, and have a twist of order $d = 6$ (refer to *Subsection 2.6.4*).

Table 4.1: Important parameters for the BLS12 and BLS24 families.

Curve Family	k	ρ	Parameters
BLS12	12	≈ 1.5	$p(z) = \frac{(z-1)^2(z^4 - z^2 + 1)}{3} + z,$ $r(z) = z^4 - z^2 + 1,$ $t(z) = z + 1$
BLS24	24	≈ 1.25	$p(z) = \frac{(z-1)^2(z^8 - z^4 + 1)}{3} + z,$ $r(z) = z^8 - z^4 + 1,$ $t(z) = z + 1$

On the other hand 4.2 lists the important parameters of the particular curves suitable for implementing pairing-based protocols at the 192-bit security level. The requirements for this security level are that the bitlength of r be at least 384 (in order to resist Pollard's rho attack (refer to [47]) on the discrete logarithm problem in $\mathbb{G}_1$), and that the bitlength of p^k should be at least 7680 (in order to resist the number field sieve attack [11] on the discrete logarithm problem in $\mathbb{F}_{p^k}^*$).

Table 4.2: Important parameters for BLS12 and BLS24 curves.

Curve	k	z	$\lceil \log_2 p \rceil$	$\lceil \log_2 r \rceil$	ρ	$\lceil \log_2 q \rceil$	$\lceil \log_2 p^k \rceil$
BLS12	12	$-2^{107} + 2^{105} + 2^{93} + 2^5$	638	427	1.49	1276	7656
BLS24	24	$-2^{48} + 2^{45} + 2^{31} - 2^7$	477	383	1.25	1914	11482

As pointed out in *Subsection 3.2.1*, Miller functions are essential to the definition of most cryptographic pairings. Although all pairings can be defined individually in formal terms, it is more instructive to give the following constructive definitions in *Table 4.3* based on [57], assuming an underlying curve $E/\mathbb{F}_q$ containing a group $E(\mathbb{F}_q)[n]$ of prime order n with embedding degree k and letting $z := (q^k - 1)/n$.

Table 4.3: Definitions of commonly used pairings on elliptic curves.

Pairing	Definition
Weil pairing	$w(P, Q) = (-1)^n \frac{f_{n,P}(Q)}{f_{n,Q}(P)}$
Tate pairing	$\tau(P, Q) = f_{n,P}(Q)^z$
Eta pairing	$\eta(P, Q) = f_{\lambda,P}(Q)^z$, where $\lambda^d \equiv 1 \pmod{n}$
Ate pairing	$a(P, Q) = f_{t-1,Q}(P)^z$, where t is the trace of Frobenius
Optimized Ate pairing	$a_c(P, Q) = f_{(t-1)c \bmod n,Q}(P)^z$, for some $0 < c < k$
Twisted Ate pairing	$\eta_c(P, Q) = f_{\lambda c \bmod n,P}(Q)^z$, for some $0 < c < k$
Optimal Ate pairing	$a_{\text{opt}}(P, Q) = f_{\ell,Q}(P)^z$, where $\log_2 \ell \approx \frac{\log_2 n}{\phi(k)}$

Optimal pairings achieve the shortest loop length among all of the pairings presented in *Table 4.3*. To obtain unique values, most of these pairings (the Weil pairing is an exception) are reduced via the final exponentiation by z . We note that the computation of z is the subject of research, refer to [58].

Before we dive into the computation of line function let us recall the optimal ate pairing on BLS curves (in this case for $k = 24$) and the arithmetic that is involved in its computation. Let $E/\mathbb{F}_p$ be a BLS curve with parameters constructed as above given an integer $x_0 \equiv 1 \pmod{3}$, i.e. $p = p(x_0)$, $r = r(x_0)$ and so forth. Then there exists a unique twist $E'/\mathbb{F}_{p^4}$ with $r \mid \#E'(\mathbb{F}_{p^4})$ with twisting isomorphism $\psi : E' \rightarrow E$ over $\mathbb{F}_{p^4}$. Let us define the usual groups $G_1 = E(\mathbb{F}_p)[r]$ and $G_2 = \ker(\phi_p - [p]) \subset E(\mathbb{F}_{p^4})[r]$ as the 1- and p -eigenspaces of the Frobenius endomorphism ϕ_p on $E[r]$. Let the group $G'_2 = \psi^{-1}(G_2)$ be the preimage of G_2 under ψ . Let $T = t - 1$. Then we can either compute the original ate pairing

$$a_T : G'_2 \times G_1 \rightarrow \mathbb{F}_{p^{24}}^*, \quad (Q', P) \mapsto f_{T, \psi(Q')}(P)^{\frac{p^{24}-1}{r}},$$

by “untwisting” Q' for the computation of $f_{T,\psi(Q')}$; or we can compute

$$a'_T : G'_2 \times G_1 \rightarrow \mathbb{F}_{p^{24}}^*, \quad (Q', P) \mapsto f_{T,Q'}(\psi^{-1}(P))^{\frac{p^{24}-1}{r}},$$

i.e. compute entirely on the twist by “twisting” P to E' .

A function $f_{T,R}(S)$ for points R, S on E or E' as above is computed with Miller’s algorithm. It involves curve arithmetic in $\mathbb{G}'_2$, i.e. doubling and addition of points on the curve E' over the field $\mathbb{F}_{p^4}$ and the computation of line functions from points in $\mathbb{G}'_2$ evaluated at a point in $\mathbb{G}_1$ or its preimage under ψ .

The function $f_{T,R}(S)$ is built up by accumulating the line function values. This requires efficient squaring and multiplication-by-line-function operations in the full extension field $\mathbb{F}_{p^{24}}$. The final exponentiation on elements in $\mathbb{F}_{p^{24}}^*$ has a fixed exponent and it is possible to use specialized more efficient squaring operations as well as other optimizations. Full extension field arithmetic needs to be particularly efficient as pairing efficiency strongly depends on it. It is therefore important to work with a well-chosen extension field tower, refer to [56].

4.1.4 Line function

The optimal ate pairing definition from *Table 4.3*. Such pairing calculates $w = e(Q, P)$, where $Q \in \mathbb{G}_2$, $P \in \mathbb{G}_1$ are points in elliptic curve groups of order r , and $w \in \mathbb{G}_T$ is an element in the k -th degree extension, in this case $\mathbb{F}_{p^{12}}$, also of order r . The elliptic curve groups $\mathbb{G}_1$ and $\mathbb{G}_2$ must be distinct groups of order r , taken over the extension $\mathbb{F}_{p^{12}}$. However a common optimization is to choose $\mathbb{G}_1$ as the group on the curve $E(\mathbb{F}_p)$ taken over the base field $\mathbb{F}_p$, and $\mathbb{G}_2$ is represented on $E'(\mathbb{F}_{p^2})$ as the sextic twist of the trace-zero group, which can be instantly untwisted when needed to a point on $E(\mathbb{F}_{p^{12}})$.

As we presented before the Miller loop (refer to *Algorithm 3*) needs to evaluate so-called line functions. These are elements in $\mathbb{F}_{p^{12}}$ that arise during the multiplication of Q by u using a standard double-and-add algorithm, and can be considered as a distance metric between these multiples of Q and the second stationary point P .

Assume Q is represented using homogeneous projective coordinates, and that its current multiple has coordinates (X, Y, Z) , and assume that the coordinates of the fixed point P are provided in affine form as (x_P, y_P) . The Miller loop calculates what is essentially the product of many line function evaluations. Refer to [55] for more about the sparsity exploitation.

In *Algorithm 4*, T represents the multiples of Q that arise, and $l_{T,S}$ represents the line function that arises when adding point S to point T . It is coordinates of T and P that are used to calculate these line functions. Now we consider line 6 of the algorithm when the i -th bit of u equals 1, that is $u_i = 1$. In this loop iteration after squaring f , the value of f is updated as $f \leftarrow f \cdot l_{T,T}(P) \cdot l_{2T,Q}(P)$. If calculated as described in this standard algorithm this would cost two dense-sparse multiplications, that is 26M. However by calculating $l_{T,T}(P) \cdot l_{2T,Q}(P)$ separately, and only then multiplying f by this product, the cost will be only 23M, as the sparse-sparse multiplication costs 6M and the multiplication of the dense f by their product costs 17M.

Input: $Q \in \mathbb{G}_2, P \in \mathbb{G}_1$, curve parameter u

Output: $f \in \mathbb{F}_{p^{12}}$

$f \leftarrow 1$;

$T \leftarrow Q$;

for $i \leftarrow \lfloor \log_2(u) \rfloor - 1$ **to** 0 **do**

$f \leftarrow f^2$;

$f \leftarrow f \cdot l_{T,T}(P), T \leftarrow 2T$;

if $u_i = 1$ **then**

$f \leftarrow f \cdot l_{T,Q}(P), T \leftarrow T + Q$;

end

end

$f \leftarrow f^{(p^6-1)(p^2+1)(p^4-p^2+1)/r}$;

return f

Algorithm 4: Ate pairing on BLS12 curve, refer to [55]

4.2 Optimizations

As mentioned earlier, optimization techniques are currently the subject of intensive research in their own right; for this reason, in this section we will mention only a few methods based on [5].

4.2.1 Denominator elimination

Let us review a refined version of Miller's algorithm for pairings over large prime fields, which is mostly due to improvements suggested by Barreto, Kim, Lynn and Scott [59], often referred as BKLS-GHS method. They started with an observation that allows us to conveniently replace the divisor D_Q with the point Q in the Tate pairing definition (refer to *Subsection 3.1.2*). Consequently long as $k > 1$ and P and Q are linearly independent, then $f_{r,P}(D_Q)^{(q^k-1)/r} = f_{r,P}(Q)^{(q^k-1)/r}$.

This saves us defining a divisor equivalent to $D_Q = (Q) - (\mathcal{O})$ with support disjoint to $(f_{r,P})$, but more importantly it allows us to simply evaluate the intermediate Miller function at the point Q (rather than two points) in each iteration of Algorithm *Algorithm 3*.

Example 4.2.1. Let $q = 47$, $E/\mathbb{F}_q : y^2 = x^3 + 21x + 15$, $\#E(\mathbb{F}_q) = 51$, $r = 17$, $k = 4$, $\mathbb{F}_{q^4} = \mathbb{F}_q(u)$ with $u^4 - 4u^2 + 5 = 0$, $P = (45, 23) \in G_1$ and $Q = (31u^2 + 29, 35u^3 + 11u) \in G_2$. Thus, the Tate pairing is $e(P, Q) = f_{r,P}(Q)^{(q^k-1)/r} = (32u^3 + 17u^2 + 43u + 12)^{287040} = 33u^3 + 43u^2 + 45u + 39$, which is the same value we got when instead computing $f_{r,P}(D_Q) = f_{r,P}([2]Q)/f_{r,P}(Q)$.

Now we can describe optimisation regarding the denominator elimination. Barreto et al. [59] noticed that $q - 1 \mid (q^k - 1)/r$, since if $r \mid q - 1$ then the embedding degree would be $k = 1$. This allows us to write the final exponent as $(q^k - 1)/r = (q - 1) \cdot c$, which gives $f_{r,P}(Q)^{(q^k-1)/r} = (f_{r,P}(Q)^{q-1})^c$, meaning that any elements of $\mathbb{F}_q$ contributing to $f_{r,P}(Q)$ will be mapped to one under the final exponentiation. Thus, one can freely multiply or divide $f_{r,P}(Q)$ by an element of $\mathbb{F}_q$ without affecting the pairing value [59, Corr. 1].

Therefore, the vertical lines appearing on the denominators of Miller's algorithm for the Tate pairing are entirely defined over $\mathbb{F}_q$: the line is a function $x - x_R$ that depends on $P \in E(\mathbb{F}_q)[r]$, which is evaluated at $x_Q \in \mathbb{F}_q$. Thus, in this case the contribution of (each of) the denominators to $f_{r,P}(Q)$ ends up being mapped to 1 under the final exponentiation, so these denominators (the v 's in the ℓ/v 's refer to Steps 5 and 9 in *Algorithm 3*) can be removed from the Miller loop.

Example 4.2.2. Again, we will continue on from *Example 4.2.1* for the sake of a convenient comparison. We simply give an updated table that details the intermediate Miller functions and pairing values subject to denominator elimination. Therefore, $e(P, Q) = f_{r,P}(Q)^{(q^k-1)/r} = (9u^3 + 10u^2 + 32u + 36)^{(q^k-1)/r} = 33u^3 + 43u^2 + 45u + 39$, which agrees with the Tate pairing value.

Lastly, we now refine Miller's algorithm for the Tate pairing computation subject to the BKLS-GHS improvements. The denominators that were on lines 5 and 9 have now gone (under the assumption that k is even), and that the second input is now the point Q , rather than a divisor equivalent to D_Q . Furthermore we have necessarily included the final exponentiation in *Algorithm 2.2* since this is what facilitates the modifications. We note that Type 3 pairing were assumed so the coordinates of P and Q lie in fields that allow for denominator elimination. Recall that the vertical line joining $(r-1)P = -P$ and P in the last iteration can also be omitted. Thus, an optimised Tate pairing computation will execute the main loop from $i = n-2$ to $i = 1$ before performing a doubling-only iteration to finish; we left the main loop to $i = 0$ for simplicity as can be followed in *Algorithm 5*.

Input: $P \in G_1, Q \in G_2$ (Type 3 pairing) and $r = (r_{n-1} \dots r_1 r_0)_2$ with $r_{n-1} = 1$
Output: $f_{r,P}(Q)^{(q^k-1)/r} \leftarrow f$
 $R \leftarrow P, f \leftarrow 1;$
for $i = n-2$ **to** 0 **do**
 Compute the sloped line function $\ell_{R,R}$ for doubling R ;
 $R \leftarrow [2]R$;
 $f \leftarrow f^2 \cdot \ell_{R,R}(Q)$;
 if $r_i = 1$ **then**
 Compute the sloped line function $\ell_{R,P}$ for adding R and P ;
 $R \leftarrow R + P$;
 $f \leftarrow f \cdot \ell_{R,P}(Q)$;
 end
end
return $f \leftarrow f^{(q^k-1)/r};$

Algorithm 5: The BKLS-GHS version of Miller's algorithm for the Tate pairing.

4.2.2 Low hamming weights

Regardless of the pairing-based protocol, the loop length of the pairing is known publicly; therefore, unlike ECC where we try to avoid special choices of scalars that might give attackers unnecessary advantage, in PBC there is no problem in specialising

the choice of the loop length. In this light, it is advantageous to use curves where the loop length has a low Hamming weight, thus minimising the number of additions incurred in Miller's algorithm.

Example 4.2.3 (BLS curve low hamming weight. Motivated by [5]). Both $x = 258419657403767392$ and $x = 144115188109674496$ are 58-bit values that result in $k = 24$ BLS curves suitable for pairings at the 224-bit security level. The former was found by kick-starting the search at a random value between 2^{57} and 2^{58} , and as such, has a Hamming weight of 28, as we would expect. On the other hand, the second value is actually $2^{57} + 2^{25} + 2^{18} + 2^{11}$, which has Hamming weight 4. Thus, we would much prefer the second value since this would result in 24 less additions through the Miller loop. Another nice alternative that gives similar parameter sizes is $x = 2^{56} + 2^{40} - 2^{20}$, which does not have a low Hamming weight, but rather a low NAF-weight (weight in the signed binary representation), for which Miller's algorithm can be updated to take advantage of.

4.2.3 Implementation Summary: Reduced Tate Pairing on BLS12-381

The SageMath implementation (refer to *Appendix A.36*) realizes the reduced Tate pairing $\tau : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ on the BLS12-381 curve, directly instantiating the theoretical constructions surveyed in the preceding sections. Therefore curve is defined by $E : y^2 = x^3 + 4$ over $\mathbb{F}_p$, with embedding degree $k = 12$, subgroup order $q = x^4 - x^2 + 1$, and BLS seed $x = -15132376222941642752$, satisfying the BLS12 polynomial family (refer to *Table 4.1*). The p -value is $p \approx 1.5$, and the 381-bit prime field together with $k = 12$ yields a 4572-bit target group $\mathbb{F}_{p^{12}}^*$, meeting the requirements for 128-bit security outlined in *Table 2.1*. The implementation constructs the extension field tower $\mathbb{F}_p \subset \mathbb{F}_{p^2} \subset \mathbb{F}_{p^{12}}$ following the strategy reviewed in *Subsection 4.1.3*. The group $G_1 = E(\mathbb{F}_p)[q]$ is the $\mathbb{F}_p$ -rational q -torsion, while G_2 is represented on the sextic twist $E'(\mathbb{F}_{p^2})$ and lifted to $E(\mathbb{F}_{p^{12}})$ via the untwist isomorphism $\psi^{-1}(x', y') = (x'/w^2, y'/w^3)$, keeping G_2 arithmetic in the cheaper quadratic field as described in *Subsection 4.1.4*.

We note that a further optimization regarding the Miller loop could be BKLS-GHS version outlined in *Algorithm 5*. On the other the computation may be more efficiently by exploiting the low Hamming weight of x (as discussed in *Subsection 4.2.2*) deferring to the final exponentiation.

The exponent $(p^{12} - 1)/q$ is decomposed as $(p^6 - 1)(p^2 + 1) \cdot (p^4 - p^2 + 1)/q$, separating the cheap *easy part* (two Frobenius maps, one inversion, two multiplications) from the *hard part*. The hard part is computed via a Frobenius-based decomposition $f^{(p^4 - p^2 + 1)/q} = f^{\lambda_0} \cdot (f^{\lambda_1})^p \cdot (f^{\lambda_2})^{p^2} \cdot (f^{\lambda_3})^{p^3}$, where each λ_i is a polynomial in x , requiring only five short exponentiations by the BLS seed, a strategy corresponding to Algorithm 1 of [57]. Both a trivial reference mode ($f^{(p^{12}-1)/q}$ directly) and the optimised two-phase mode are implemented and cross-validated, with the optimised result satisfying $\tau_{\text{opt}}(P, Q) = \tau(P, Q)^3$, verified by the assertion `mu\_r\_tate\_1**3 == mu\_r\_tate\_2`. Recall the pairing properties from *Section 2.6*, the implementation includes axiomatic test beyond the membership assertion $\tau(P, Q)^q = 1$, to establishing all defining properties of the pairing as can be followed in *Pseudo-code 4.1*:

- **Non-degeneracy:** $\tau(P, Q) \neq 1$ for independent generators $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$.

- **Bilinearity in P :** $\tau(aP, Q) = \tau(P, Q)^a$.
- **Bilinearity in Q :** $\tau(P, aQ) = \tau(P, Q)^a$.
- **Joint and cross bilinearity:** $\tau(aP, bQ) = \tau(P, Q)^{ab} = \tau(bP, aQ)$.

```

1  # e(P, Q) must not be 1 for non-trivial independent generators
2  Non-degeneracy: OK
3
4  Bilinearity in P:
5  e(aP, Q) == e(P, Q)^a OK
6
7  Bilinearity in Q:
8  e(P, aQ) == e(P, Q)^a OK
9
10 Joint bilinearity:
11 e(aP, bQ) == e(P, Q)^(ab) OK
12
13 Cross bilinearity:
14 e(aP, bQ) == e(bP, aQ) OK

```

Listing 4.1: Axiomatic test initiated to ensure Tate pairing is correctly implemented. (refer to *Code Snippet A.37*)

Although the reduced Tate pairing implementation presented above is experimentally verified, it leaves substantial room for optimization at the algorithmic level. Three principal directions are mentioned below, each reducing the dominant cost (the Miller loop length) by exploiting the structure of the BLS12-381 parameters. The first and most immediate improvement follows from replacing the Tate pairing loop scalar $q \approx 2^{255}$ with the trace of Frobenius $T = t - 1 = x$, where $|x| \approx 2^{63}$ for BLS12-381. The *ate pairing* (refer to *Table 4.3*) is defined as

$$a(Q, P) = f_{T, Q}(P)^{(p^{12}-1)/q},$$

swapping the roles of P and Q so that the Miller loop iterates over $Q \in G_2$ with $P \in G_1$ as the evaluation point. Since $\log_2 |T| \approx 63$ compared to $\log_2 q \approx 255$, the loop length is reduced by a factor of approximately $4\times$, giving a proportional reduction in the number of $\mathbb{F}_{p^{12}}$ squarings and line-function multiplications. The final exponentiation remains unchanged. Correctness follows from the identity $f_{T, Q}(P)^{(p^{12}-1)/q} = f_{q, P}(Q)^{(p^{12}-1)/q}$, which holds modulo the kernel of the final exponentiation map.

The *r-ate pairing* generalizes the ate pairing by allowing the loop scalar to be any element of the lattice $\Lambda = \{s \in \mathbb{Z} : s \equiv 0 \pmod{q}, |s| < p^{k/2}\}$ reduced modulo q . For BLS12-381 one may take $s = x^2 - 1$ or similar short combinations of powers of x , yielding loop lengths as short as $\log_2 |s| \approx 126$ bits. The key theoretical requirement is that $s \not\equiv 0 \pmod{q}$ and that the resulting Miller function still lies in the correct coset of $(\mathbb{F}_{p^{12}}^*)^q$, which is guaranteed when s belongs to the Optimal Ate admissibility conditions (refer to [60]).

The *optimal Ate pairing* achieves the theoretical lower bound on loop length, namely $\log_2 \ell \approx \log_2(q)/\phi(k)$ (refer to *Table 4.3*), where $\phi(k) = 4$ for $k = 12$, giving $\ell \approx 2^{64}$. For BLS12-381 this is realised by the scalar $\ell = x$, recovering precisely the BLS seed, so that the optimal Ate loop length coincides with that of the ate pairing. Combined

with the denominator elimination of *Subsection 4.2.1* , sparse-sparse line function multiplication (saving 3M per bit-1 iteration as noted in *Subsection 4.1.4*), and the lazy-reduction strategy for $\mathbb{F}_{p^{12}}$ arithmetic, the optimal Ate pairing on BLS12-381 represents the state-of-the-art implementation target, reducing the total Miller loop cost relative to the Tate implementation above by a factor exceeding $4\times$ before any field-level optimizations are applied.

Conclusion

In this thesis, we have studied pairing computations over elliptic curves of the BLS family, with a particular focus on the Weil and Tate pairings. We began by reviewing the theoretical foundations in, which established the algebraic hierarchy of groups, rings, and fields, with special attention to Galois fields and their indispensable role in cryptographic applications. Finite field arithmetic was examined in the following subsection with respect to the properties that make it amenable to efficient computation, including the existence and uniqueness of field extensions and the significance of field characteristics. The connection between pairing-based cryptography and elliptic curve cryptosystems was formalised through the review of discrete logarithm-based hardness assumptions to ensure the secure application of pairing-based protocols.

In the following section elliptic curve arithmetic over finite fields was covered noticing the difference between ordinary and supersingular curves, with regard to affine and projective point representations, and the operations of point addition and doubling. The theory of divisors provided us the algebraic language to handle rational functions on curves, culminating in the definition of the r -torsion subgroup, the embedding degree, and the four standard pairing type classifications. The notion of elliptic curve twists was introduced as the mechanism by which computations over the full extension field $\mathbb{F}_{q^k}$ can be pulled back into the subfield $\mathbb{F}_{q^{k/d}}$, yielding significant savings in field arithmetic.

The theoretical framework was translated into computational procedures in the following chapter, in which the Weil and Tate pairing were introduced first through their defining divisor-theoretic expressions. We noted that the infeasibility of computing explicit rational function expressions for cryptographically sized inputs motivated the adoption of Miller’s algorithm throughout, and the chapter concluded with worked examples illustrating the pairing properties of bilinearity and non-degeneracy in concrete group settings.

The last chapter started by addressing the problem of pairing-friendly curves selection, framing it as a balancing act among three simultaneous security requirements: the ECDLP hardness in $\mathbb{G}_1$ and $\mathbb{G}_2$, and the DLP hardness in the target group $\mathbb{G}_T \subset \mathbb{F}_{p^k}^*$. The ρ -value was revisited as a measure of the efficiency of a curve’s group order relative to its field size. The complex multiplication method of Atkin and Morain was reviewed as the principal technique for constructing ordinary pairing-friendly curves from a prescribed CM discriminant, and the BLS curve family was derived as a leading instantiation of this methodology. The BLS12-381 curve, with embedding degree $k = 12$ and a 381-bit prime field, was selected as the implementation of the reduced Tate pairing with balance between security and efficiency at the 128-bit security level.

Bibliography

- [1] Antoine Joux. “A One Round Protocol for Tripartite Diffie–Hellman.” In: *Journal of Cryptology* 17.3 (2004), pp. 263–276.
- [2] Alfred Menezes. “An Introduction to Pairing-Based Cryptography.” In: *Recent Trends in Cryptography*. Ed. by I. Luengo. Vol. 477. Contemporary Mathematics. American Mathematical Society and Real Sociedad Matemática Española, 2009, pp. 47–65. URL: <https://www.math.uwaterloo.ca/~ajmeneze/publications/pairings.pdf>.
- [3] Dan Boneh and Matthew Franklin. “Identity-Based Encryption from the Weil Pairing.” In: *SIAM Journal on Computing* 32.3 (2003), pp. 586–615. DOI: 10.1137/S0097539701398521. eprint: <https://doi.org/10.1137/S0097539701398521>. URL: <https://doi.org/10.1137/S0097539701398521>.
- [4] Nir Bitansky et al. “From Extractable Collision Resistance to Succinct Non-Interactive Arguments of Knowledge, and Back Again.” In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference (ITCS 2012)*. ACM, 2012, pp. 326–349. DOI: 10.1145/2090236.2090263. URL: <https://doi.org/10.1145/2090236.2090263>.
- [5] Craig Costello. *Pairings for Beginners*. [Online; accessed 17. January. 2026]. 2012. URL: <https://static1.squarespace.com/static/5fdbb09f31d71c1227082339/t/5ff394720493bd28278889c6/1609798774687/PairingsForBeginners.pdf>.
- [6] Lawrence C. Washington. *Elliptic Curves: Number Theory and Cryptography*. 2nd ed. Discrete Mathematics and Its Applications. Second edition. Boca Raton, FL: CRC Press, Taylor & Francis Group, 2008, p. 536. ISBN: 978-1-4200-7146-7.
- [7] Craig Costello. *Fast Formulas for Computing Cryptographic Pairings*. [Online; accessed 17. January. 2026]. 2012. URL: https://eprints.qut.edu.au/61037/1/Craig_Costello_Thesis.pdf.
- [8] The Sage Developers. *SageMath: Open-Source Mathematics Software*. <https://www.sagemath.org/>. Accessed: 2026-01-13. 2026.
- [9] Bert den Boer. “Diffie-Hellman is as Strong as Discrete Log for Certain Primes.” In: *Advances in Cryptology — CRYPTO ’88, 8th Annual International Cryptology Conference, Santa Barbara, California, USA, August 21-25, 1988, Proceedings*. Vol. 403. Lecture Notes in Computer Science. Springer, 1988, pp. 530–539. DOI: 10.1007/0-387-34799-2_38.

- [10] Ueli M. Maurer and Stefan Wolf. “The Relationship Between Breaking the Diffie-Hellman Protocol and Computing Discrete Logarithms.” In: *SIAM Journal on Computing* 28.5 (1999), pp. 1689–1721. DOI: 10.1137/S0097539796302749.
- [11] Diego F. Aranha, Georgios Fotiadis, and Aurore Guillevic. “A short-list of pairing-friendly curves resistant to the Special TNFS algorithm at the 192-bit security level.” In: *IACR Communications in Cryptology* 1.3 (2024). hal-04666521v2, p. 44. DOI: 10.62056/angyl86bm.
- [12] National Security Agency. *Commercial National Security Algorithm Suite 2.0 Cybersecurity Advisory*. Technical Report. Cybersecurity Advisory outlining quantum-resistant algorithm requirements for National Security Systems. U.S. National Security Agency, Sept. 2022. URL: https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF.
- [13] Yumi Sakemi, Satoru Kanno, and Riad S. Wahby. *Pairing-Friendly Curves*. Internet-Draft draft-irtf-cfrg-pairing-friendly-curves-12. Work in Progress. Internet Engineering Task Force, Nov. 2025. 54 pp. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-pairing-friendly-curves/12/>.
- [14] A. O. L. Atkin and François Morain. “Elliptic curves and primality proving.” In: *Mathematics of Computation* 61 (1993), pp. 29–68. DOI: 10.1090/S0025-5718-1993-1183434-8.
- [15] Steven H. Weintraub. *An Introduction to Abstract Algebra: Sets, Groups, Rings, and Fields*. Singapore: World Scientific Publishing, 2022, pp. 29–54. ISBN: 978-981-124-755-2.
- [16] Joyce K. Reynolds. *Assigned Numbers: RFC 1700 is Replaced by an On-line Database*. Accessed: 2025-03-28. 2002. URL: <https://datatracker.ietf.org/doc/html/rfc3232>.
- [17] Darrel Hankerson, Alfred Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. 46-65, 76-86, 171-172. Springer Professional Computing. eBook ISBN: 978-0-387-21846-5; Softcover ISBN: 978-1-4419-2929-7. New York, NY: Springer, 2004, pp. 1–6. ISBN: 978-0-387-95273-4. DOI: 10.1007/b97644.
- [18] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*. <https://csrc.nist.gov/pubs/fips/197/final>. FIPS PUB 197, updated May 9, 2023. 2023. DOI: 10.6028/NIST.FIPS.197-upd1.
- [19] National Institute of Standards and Technology. *The Keyed-Hash Message Authentication Code (HMAC)*. Tech. rep. FIPS PUB 198-1. National Institute of Standards and Technology, July 2008. DOI: 10.6028/NIST.FIPS.198-1. URL: http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf.
- [20] W. Diffie and M. Hellman. “New directions in cryptography.” In: *IEEE Transactions on Information Theory* 22.6 (1976), pp. 644–654. DOI: 10.1109/TIT.1976.1055638.
- [21] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. *An Introduction to Mathematical Cryptography*. 2nd ed. Undergraduate Texts in Mathematics. Springer-Verlag, 2014, pp. 45, 75. ISBN: 978-1-4939-1710-5. DOI: 10.1007/978-1-4939-1711-2.

- [22] ECRYPT II. *D.MAYA.6: Final Report on Main Computational Assumptions in Cryptography*. European Commission FP7, ICT-2007-216676. ECRYPT II Project, 2013. URL: <https://cordis.europa.eu/docs/projects/cnect/6/216676/080/deliverables/001-DMAYA6.pdf>.
- [23] Nigel P. Smart. *ECRYPT II Yearly Report on Algorithms and Key Sizes (2009–2010)*. Tech. rep. D.SPA.13. Published as deliverable D.SPA.13. ECRYPT II – European Network of Excellence in Cryptology, EU FP7, ICT-2007-216676, 2010. URL: <http://www.ecrypt.eu.org/documents/D.SPA.13.pdf>.
- [24] National Institute of Standards and Technology. *Recommendation for Key Management: Part 1 – General*. NIST Special Publication 800-57 Part 1 Rev. 5. Security strength equivalences for cryptographic algorithms. 2020.
- [25] T. Lange D. J. Bernstein. *Explicit-formulas database*. [Online; accessed 17. February. 2026]. 2007. URL: <http://www.hyperelliptic.org/EFD>.
- [26] Huseyin Hisil et al. *Faster Group Operations on Elliptic Curves*. Cryptology ePrint Archive, Paper 2007/441. 2007. URL: <https://eprint.iacr.org/2007/441>.
- [27] Ben Lynn. “On the Implementation of Pairing-based Cryptosystems.” PhD Dissertation. <https://crypto.stanford.edu/pbc/thesis.pdf>. PhD thesis. Stanford University, June 2007.
- [28] Alfred Menezes. *Elliptic Curve Public Key Cryptosystems*. 1st ed. The Springer International Series in Engineering and Computer Science. New York, NY: Springer, 1993, pp. 30–32. ISBN: 978-0-7923-9368-9. DOI: 10.1007/978-1-4615-3198-2.
- [29] Steven D. Galbraith. *Mathematics of Public Key Cryptography*. Cambridge University Press, 2012, p. 135.
- [30] Steven D. Galbraith. *Pairings*. Ed. by Ian F. Blake, Gadiel Seroussi, and Nigel P. Smart. Vol. 317. London Mathematical Society Lecture Note Series. Cambridge: Cambridge University Press, 2005. Chap. 9, pp. 183–213. ISBN: 9780521604154.
- [31] Steven D. Galbraith, Kenneth G. Paterson, and Nigel P. Smart. “Pairings for Cryptographers.” In: *Discrete Applied Mathematics* 156.16 (2008), pp. 3113–3121. DOI: 10.1016/j.dam.2007.12.010.
- [32] Hovav Shacham. “New Paradigms in Signature Schemes.” PhD thesis. Stanford University, Dec. 2005.
- [33] Nigel P. Smart and Frederik Vercauteren. “On Computable Isomorphisms in Efficient Asymmetric Pairing-Based Systems.” In: *Discrete Applied Mathematics* 155.4 (2007), pp. 538–547. DOI: 10.1016/j.dam.2006.03.022.
- [34] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. “On the Selection of Pairing-Friendly Groups.” In: *Selected Areas in Cryptography*. Ed. by Mitsuru Matsui and Robert J. Zuccherato. Vol. 3006. Lecture Notes in Computer Science. Springer, 2004, pp. 17–25. DOI: 10.1007/978-3-540-24654-1_2.
- [35] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*. 2nd ed. Vol. 106. Graduate Texts in Mathematics. New York: Springer, 2009. ISBN: 978-0-387-09494-6.

- [36] Victor S. Miller. “Short Programs for Functions on Curves.” Unpublished manuscript. 1986.
- [37] Victor S. Miller. “The Weil Pairing, and Its Efficient Calculation.” In: *J. Cryptology* 17 (2004), pp. 235–261. DOI: 10.1007/s00145-004-0315-8.
- [38] Ben Lynn. “On the Implementation of Pairing-Based Cryptosystems.” PhD Dissertation. Stanford, CA: Stanford University, June 2007.
- [39] Michael Scott. *Understanding the Tate Pairing*. Personal webpage. Accessed online. 2004. URL: <http://www.computing.dcu.ie/~mike/tate.html>.
- [40] Diego F. Aranha et al. “Faster Explicit Formulas for Computing Pairings over Ordinary Curves.” In: *Advances in Cryptology – EUROCRYPT 2011*. Vol. 6632. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2011, pp. 48–68. DOI: 10.1007/978-3-642-20465-4_4. URL: <https://eprint.iacr.org/2010/526>.
- [41] Diego Aranha et al. “Implementing Pairings at the 192-Bit Security Level.” In: Jan. 2013, pp. 177–195. ISBN: 978-3-642-36333-7. DOI: 10.1007/978-3-642-36334-4_11.
- [42] Riad S. Wahby and Dan Boneh. *Fast and simple constant-time hashing to the BLS12-381 elliptic curve*. Cryptology ePrint Archive, Paper 2019/403. 2019. DOI: 10.13154/tches.v2019.i4.154-179. URL: <https://eprint.iacr.org/2019/403>.
- [43] Craig Costello. “Fast Formulas for Computing Cryptographic Pairings.” PhD thesis. Brisbane, Australia: Queensland University of Technology, 2012.
- [44] Vincent Guillevi. *pairings-192*. GitLab repository. Accessed: 2026-06-12. 2024. URL: <https://gitlab.inria.fr/guillevi/pairings-192>.
- [45] R. Balasubramanian and Neal Koblitz. “The Improbability That an Elliptic Curve Has Subexponential Discrete Log Problem under the Menezes-Okamoto-Vanstone Algorithm.” In: *Journal of Cryptology* 11.2 (1998), pp. 141–145.
- [46] Nadia El Mrabet and Marc Joye, eds. *Guide to Pairing-Based Cryptography*. Chapman & Hall/CRC Cryptography and Network Security Series. Boca Raton, FL: CRC Press, 2017. ISBN: 978-1-4987-2950-5.
- [47] Junfeng Fan and Ingrid Verbauwhede. “An Updated Survey on Secure ECC Implementations: Attacks, Countermeasures and Cost.” In: *Cryptography and Security: From Theory to Applications*. Ed. by David Naccache. Vol. 6805. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012, pp. 265–282. DOI: 10.1007/978-3-642-28368-0_18.
- [48] David Freeman, Michael Scott, and Edlyn Teske. *A taxonomy of pairing-friendly elliptic curves*. Cryptology ePrint Archive, Paper 2006/372. 2006. URL: <https://eprint.iacr.org/2006/372>.
- [49] Clifford Cocks and Richard G. E. Pinch. “ID-Based Cryptosystems Based on the Weil Pairing.” Unpublished manuscript. 2001.
- [50] R. Dupont, A. Enge, and F. Morain. “Building curves with arbitrary small MOV degree over finite prime fields.” In: *Journal of Cryptology* 18.2 (2005), pp. 79–89. DOI: 10.1007/s00145-004-0146-7.

- [51] Atsuko Miyaji, Masaki Nakabayashi, and Shunzou Takano. “New explicit conditions of elliptic curve traces for FR-reduction.” In: *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* E84-A.5 (2001), pp. 1234–1243.
- [52] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. *Constructing Elliptic Curves with Prescribed Embedding Degrees*. 2002. IACR: 2002/088.
- [53] Michael Scott and Paulo S. L. M. Barreto. “Generating more MNT elliptic curves.” In: *Designs, Codes and Cryptography* 38.2 (2006), pp. 209–217.
- [54] Steven D. Galbraith, James F. McKee, and Paulo C. Valença. “Ordinary abelian varieties having small embedding degree.” In: *Finite Fields and Their Applications* 13.4 (2007), pp. 800–814. DOI: 10.1016/j.ffa.2006.12.004.
- [55] Michael Scott. *Pairing Implementation Revisited*. Cryptology ePrint Archive, Paper 2019/077. Available at <https://eprint.iacr.org/2019/077>. 2019. URL: <https://eprint.iacr.org/2019/077>.
- [56] Craig Costello, Kristin E. Lauter, and Michael Naehrig. “Attractive Subfamilies of BLS Curves for Implementing High-Security Pairings.” In: *Progress in Cryptology – INDOCRYPT 2011*. Ed. by Daniel J. Bernstein and Sanjit Chatterjee. Vol. 7107. Lecture Notes in Computer Science. Springer, 2011, pp. 320–342. DOI: 10.1007/978-3-642-25578-6_23.
- [57] Diego F. Aranha et al. *The Realm of the Pairings*. Cryptology ePrint Archive, Paper 2013/722. <https://eprint.iacr.org/2013/722>. 2013.
- [58] Michael Scott et al. “On the Final Exponentiation for Calculating Pairings on Ordinary Elliptic Curves.” In: *Pairing-Based Cryptography – Pairing 2009*. Vol. 5671. LNCS. Springer, 2009, pp. 78–88. DOI: 10.1007/978-3-642-03298-1_6.
- [59] Paulo S. L. M. Barreto et al. “Efficient Algorithms for Pairing-Based Cryptosystems.” In: *Advances in Cryptology – CRYPTO 2002*. Ed. by Moti Yung. Vol. 2442. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2002, pp. 354–368. ISBN: 978-3-540-44050-5. DOI: 10.1007/3-540-45708-9_23.
- [60] Florian Hess. “Pairing Lattices.” In: *Pairing-Based Cryptography – Pairing 2008*. Vol. 5209. Lecture Notes in Computer Science. Springer, 2008, pp. 18–38.

